Московский политехнический университет

Отчёт

по дисциплине «Программирование криптографических алгоритмов»

Выполнил студент
группы 241-372:
Сергеев Н.А.
Преподаватель:
Бутакова Н.Г.

Москва, 2026 г.

Содержание

Содержание 2
Аннотация 3
Общий интерфейс 4
Общие функции. ..... **Ошибка! Закладка не определена.**
Блок A: Шифры однозначной замены 8
1 Шифр Атбаш 8
Блок B: Шифры многозначной замены 12
5. Шифр Белазо 12
Блок C: Шифры блочной замены 18
8. Матричный шифр 18
Блок D: Шифры перестановки 25
10. Вертикальная перестановка 25
Блок E: Шифры гаммирования 33
13. Одноразовый блокнот Шеннона 33
Блок F: Поточные шифры 40
15. Поточный шифр A5/1 40
Блок G: Комбинационные шифры 50
19. Шифр AES 50
Блок H: Ассиметричные шифры 57
21. Шифр RSA 57
Блок I: алгоритмы цифровых подписей 66
24. Цифровая подпись RSA 66
Блок J: Стандарты цифровых подписей 73
26. Цифровая подпись ГОСТ 34.10-94 73
Блок K: обмен ключами 82
28. Обмен ключами по алгоритму Диффи-Хеллмана 82

Аннотация

Среда программирования: PyCharm

Язык программирования: python

Процедуры для запуска программ: скачать папку с программами, открыть файл main.py, запустить его, при необходимости выбрать интерпретатор, откроется меню со всеми шифрами, дальше действовать по требованиям программы.

Пословица - текст по варианту 1: Леопард не может изменить своих пятен.

Текст на 1000 знаков для проверки:

Более всего на свете прокуратор ненавидел запах розового масла, и все теперь предвещало нехороший день, так как запах этот начал преследовать прокуратора с рассвета. Прокуратору казалось, что розовый запах источают кипарисы и пальмы в саду, что к запаху кожаного снаряжения и пота от конвоя примешивается проклятая розовая струя. От флигелей в тылу дворца, где расположилась пришедшая с прокуратором в Ершалаим первая когорта Двенадцатого Молниеносного легиона, заносило дымком в колоннаду через верхнюю площадку сада, и к горьковатому дыму, свидетельствовавшему о том, что кашевары в кентуриях начали готовить обед, примешивался все тот же жирный розовый дух. «Да, нет сомнений, это она, опять она, непобедимая, ужасная болезнь гемикрания, при которой болит полголовы от нее нет средств, нет никакого спасения попробую не двигать головой».

Введение

Цель работы — изучить реализацию криптографических алгоритмов и проверить их работу на практических примерах. В отчёте рассматриваются шифрование и расшифрование, формирование и проверка цифровой подписи, а также согласование общего ключа.

Для этого реализации объединены общим консольным интерфейсом. Для алгоритмов приведены описание, блок-схемы, код и результаты выполнения; параметры и сообщения вводятся согласно логике соответствующей программы.

Общий интерфейс

Данный интерфейс объединяет все программы с шифрами в одну программу.

Для создания приложения выбран консольный интерфейс. Пользователь выбирает шифр, после чего пользователю предоставляется выбор: зашифровать, расшифровать, выход из программы, а также в индивидуальном порядке ввести какие-то дополнительные данные.

Код программы:

```
import runpy
from pathlib import Path

BASE_DIR = Path(__file__).parent

def run_script(relative_path):
    try:
        path = BASE_DIR / relative_path
        print("\n" + "=" * 60)
        print(f"Запуск: {path.name}")
        print("=" * 60 + "\n")
        runpy.run_path(str(path), run_name="__main__")
    except FileNotFoundError:
        print(f"\nФайл не найден:\n{path}")
    except Exception as e:
        print(f"\nОшибка:\n{e}")
    input("\nНажмите Enter для продолжения...")

# ==========================
# КЛАССИЧЕСКИЕ ШИФРЫ
# ==========================
def classic_menu():
    while True:
        print("\n" + "=" * 60)
        print("КЛАССИЧЕСКИЕ ШИФРЫ")
        print("=" * 60)
```

```
        print("1 - Атбаш")
        print("2 - Цезарь")
        print("3 - Полибий")
        print("4 - Тритемий")
        print("5 - Виженер")
        print("6 - Вертикальная перестановка")
        print("7 - Матричный шифр")
        print("8 - Плейфер")
        print("0 - Назад")
        choice = input("\nВаш выбор: ")
        if choice == "1":
            run_script("Лабы Тема 1/Лаб 1_1/1.Шифр Атбаш.ру")
        elif choice == "2":
            run_script("Лабы Тема 1/Лаб 1_1/2.Шифр Цезаря.ру")
        elif choice == "3":
            run_script("Лабы Тема 1/Лаб 1_1/3.Квадрат Полибия.ру")
        elif choice == "4":
            run_script("Лабы Тема 1/Лаб 1_2/4.Шифр Тритемия.ру")
        elif choice == "5":
            run_script("Лабы Тема 1/Лаб 1_2/5.Шифр Виженера.ру")
        elif choice == "6":
            run_script("Лабы Тема 1/Лаб 1_4/10.Вертикальная перестановка.ру")
        elif choice == "7":
            run_script("Лабы Тема 1/Лаб 1_3/8.Матричный шифр.ру")
        elif choice == "8":
            run_script("Лабы Тема 1/Лаб 1_3/9.Шифр Плейфера.ру")
        elif choice == "0":
            return

# ==========================
# СИММЕТРИЧНЫЕ АЛГОРИТМЫ
# ==========================
def symmetric_menu():
    while True:
        print("\n" + "=" * 60)
        print("СИММЕТРИЧНЫЕ АЛГОРИТМЫ")
        print("=" * 60)
        print("1 - МАГМА (ГОСТ Р 34.12-2015)")
        print("2 - Блокнот Шеннона")
        print("3 - A5/1")
        print("4 - A5/2")
        print("5 - AES")
        print("6 - Кузнечик (ГОСТ Р 34.12-2015)")
        print("0 - Назад")
        choice = input("\nВаш выбор: ")
        if choice == "1":
            run_script("Лабы Тема 1/Лаб 1_7/17.MAGMA.ру")
        elif choice == "2":
            run_script("Лабы Тема 1/Лаб 1_5/13.Одноразовый блокнот К.Шеннона.ру")
        elif choice == "3":
            run_script("Лабы Тема 1/Лаб 1_6/15.A5_1.ру")
        elif choice == "4":
            run_script("Лабы Тема 1/Лаб 1_6/16.A5_2.ру")
        elif choice == "5":
            run_script("Лабы Тема 1/Лаб 1_7/19.AES.ру")
        elif choice == "6":
            run_script("Лабы Тема 1/Лаб 1_7/20.KUZNECHIK.ру")
        elif choice == "0":
            return

# ==========================
# АСИММЕТРИЧНЫЕ
# ==========================
def asymmetric_menu():
    while True:
        print("\n" + "=" * 60)
        print("АСИММЕТРИЧНЫЕ АЛГОРИТМЫ")
        print("=" * 60)
        print("1 - RSA (шифрование)")
        print("2 - Диффи-Хеллман")
        print("0 - Назад")
        choice = input("\nВаш выбор: ")
```

```
        if choice == "1":
            run_script("Лабы Тема 2/Лаб 2_1/21-23.RSA_Elgamal_ECC.py")
        elif choice == "2":
            run_script("Лабы Тема 4/Лаб 4_1/28.Диффи-Хеллман.py")
        elif choice == "0":
            return

# ==========================
# ЭЛЕКТРОННАЯ ПОДПИСЬ
# ==========================
def signature_menu():
    while True:
        print("\n" + "=" * 60)
        print("ЭЛЕКТРОННАЯ ПОДПИСЬ")
        print("=" * 60)
        print("1 - RSA подпись")
        print("2 - ГОСТ Р 34.10-94")
        print("3 - ГОСТ Р 34.10-2012")
        print("0 - Назад")
        choice = input("\nВаш выбор: ")
        if choice == "1":
            run_script("Лабы Тема 3/Лаб 3_1/24.RSA.py")
        elif choice == "2":
            run_script("Лабы Тема 3/Лаб 3_2/26.ГОСТ_Р_34.10-94.py")
        elif choice == "3":
            run_script("Лабы Тема 3/Лаб 3_2/27.ГОСТ_Р_34.10-2012.py")
        elif choice == "0":
            return

# ==========================
# ГЛАВНОЕ МЕНЮ
# ==========================
def main():
    while True:
        print("\n")
        print("=" * 60)
        print("ГЛАВНОЕ МЕНЮ КРИПТОГРАФИЧЕСКИХ АЛГОРИТМОВ")
        print("=" * 60)
        print("1 - Классические шифры")
        print("2 - Симметричные алгоритмы")
        print("3 - Асимметричные алгоритмы")
        print("4 - Электронная подпись")
        print("0 - Выход")
        choice = input("\nВаш выбор: ")
        if choice == "1":
            classic_menu()
        elif choice == "2":
            symmetric_menu()
        elif choice == "3":
            asymmetric_menu()
        elif choice == "4":
            signature_menu()
        elif choice == "0":
            print("\nДо свидания!")
            break
        else:
            print("\nНеверный выбор")

if __name__ == "__main__":
    main()
```

Рисунок 1. Общий интерфейс.

Блок-схема:

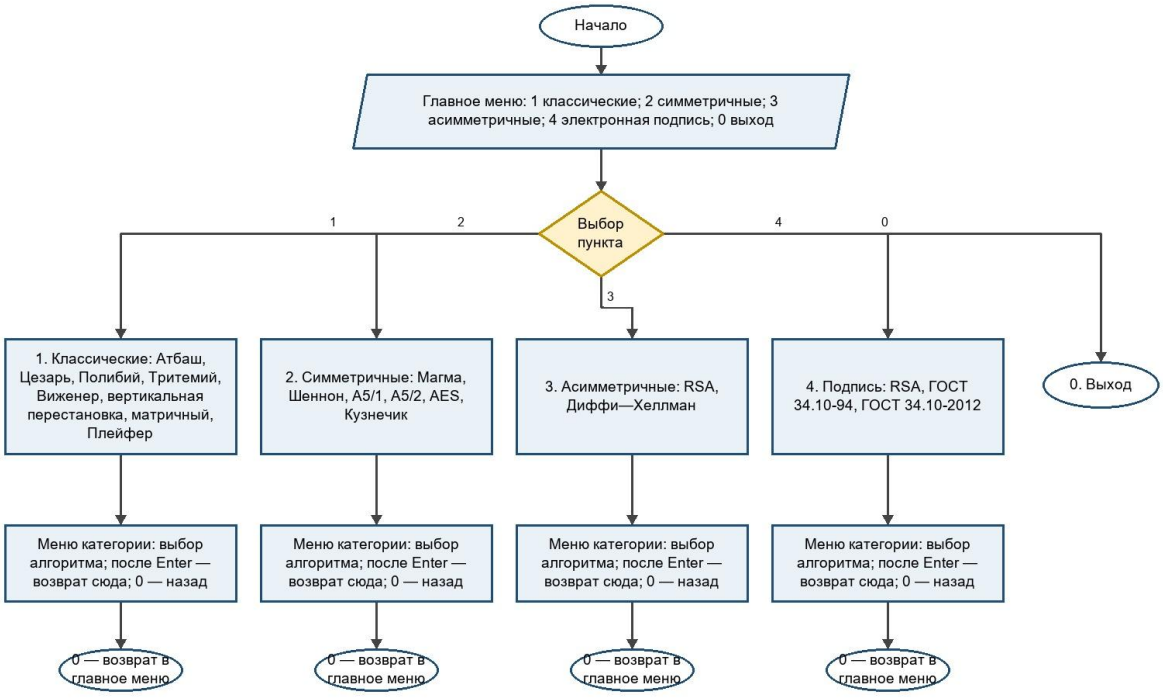


Рисунок 2. Блок-схема интерфейса всех шифров.

Блок А: Шифры однозначной замены

1. Шифр Атбаш

Шифр АТБАШ - шифр простой замены. Алгоритм представляет собой замену i-ой буквы алфавита на n-i+1 букву алфавита, где n – размерность алфавита.

Блок-схема:

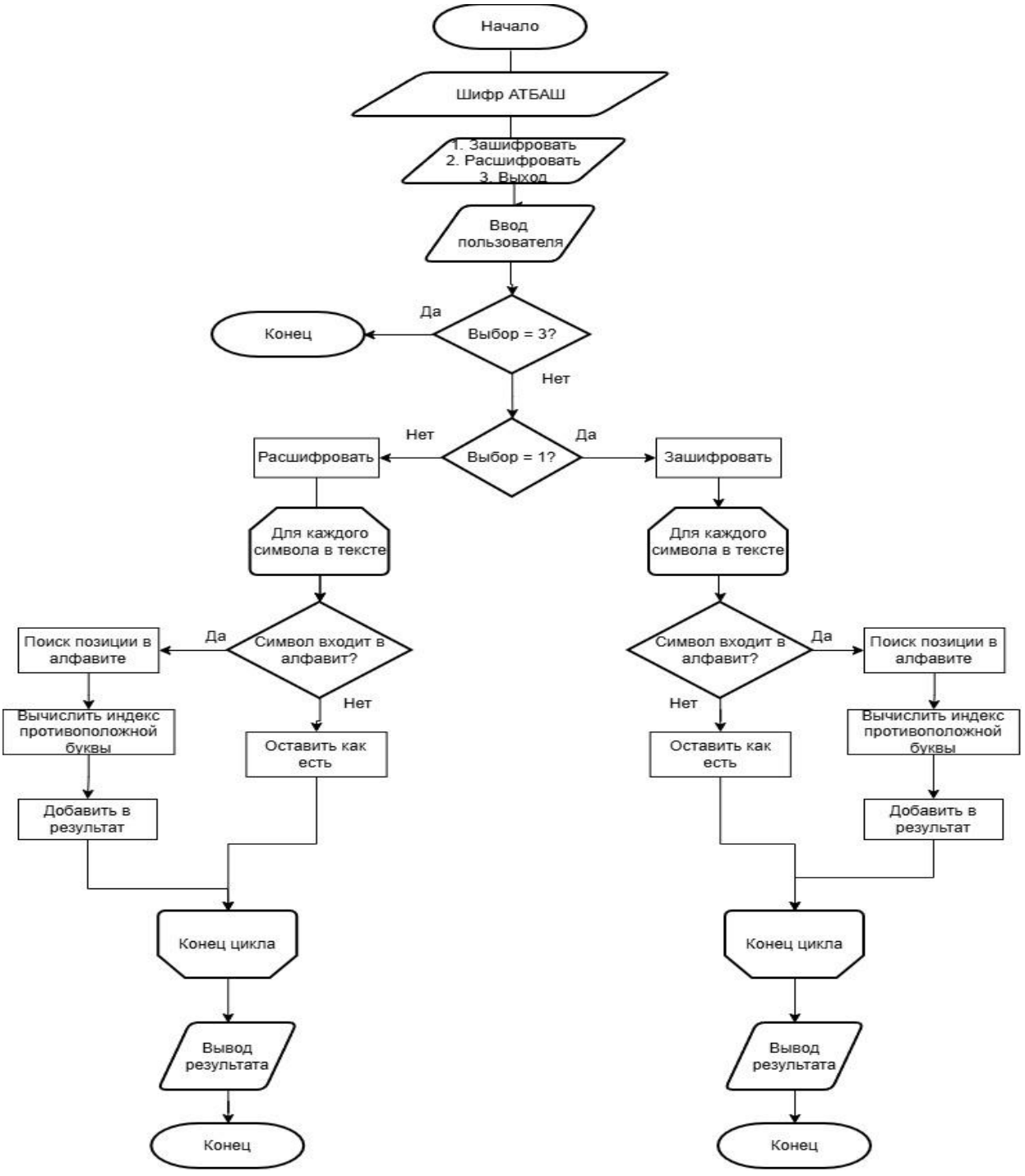


Рисунок 4. Блок-схема АТБАШ

Код программы:

```
"""
ШИФР АТБАШ
Русский алфавит (33 буквы с Ъ)
Первая буква алфавита заменяется последней, вторая — предпоследней и т.д.
"""

ALPHABET = 'АБВГДЕЁЖЗИЙКЛМНОПРСТУФХЦЧШЩЪЫЬЭЮЯ'
N = len(ALPHABET)

char_to_num = {char: i for i, char in enumerate(ALPHABET)}
num_to_char = {i: char for i, char in enumerate(ALPHABET)}

def encrypt(text):
    text = text.upper()
    result = []

    for char in text:
        if char == ' ':
            result.append(' ')
        elif char in char_to_num:
            m = char_to_num[char]
            L = (N - 1 - m) % N
            result.append(num_to_char[L])
        else:
            result.append(char)

    return ''.join(result)

def decrypt(ciphertext):
    return encrypt(ciphertext)  # Атбаш симметричен

while True:
    print("\n" + "=" * 50)
    print("ШИФР АТБАШ")
    print("=" * 50)
    print("1. Зашифровать")
    print("2. Расшифровать")
    print("3. Выход")

    choice = input("\nВыберите действие: ")

    if choice == "1":
        text = input("Введите текст для шифрования: ")
        encrypted = encrypt(text)

        print("\nРезультат:")
        print(encrypted)

    elif choice == "2":
        text = input("Введите текст для расшифрования: ")
        decrypted = decrypt(text)

        print("\nРезультат:")
        print(decrypted)

    elif choice == "3":
        print("Выход из программы...")
        break

    else:
        print("Неверный выбор!")
```

Результат работы программы:

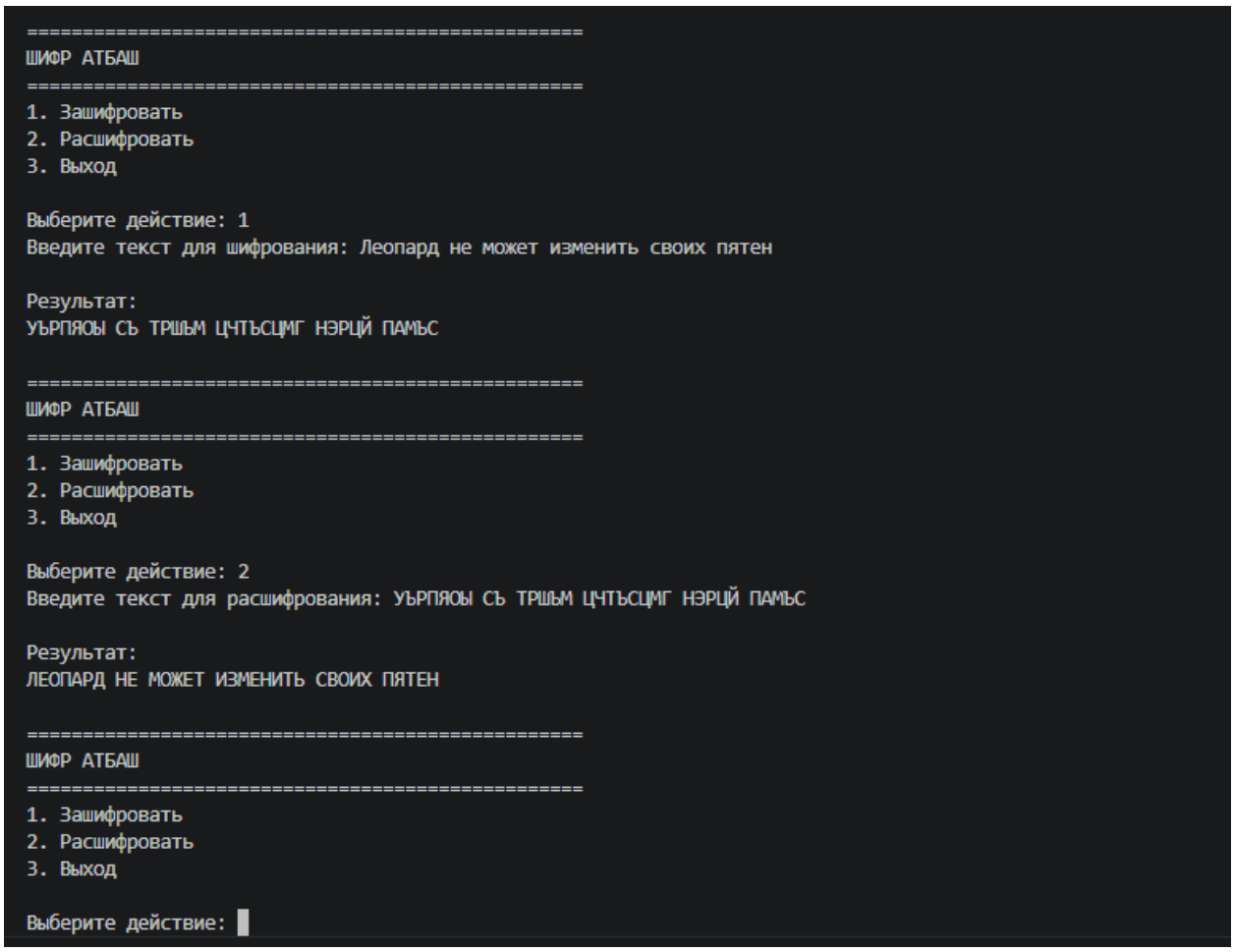
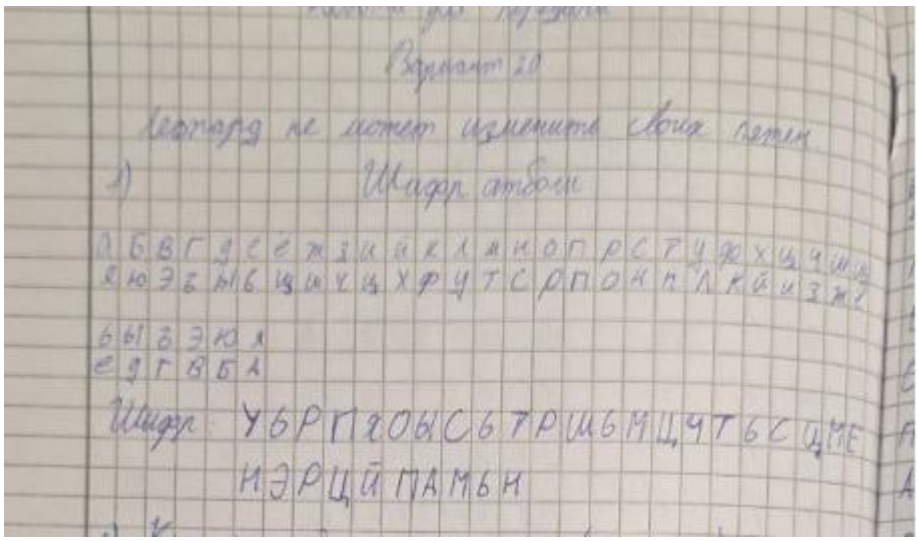


Рисунок 5. Результат выполнения шифра АТБАШ



Тест на 1000 символов:

```
Выполнение задачи: C:\Users\NIKITA\AppData\Local\Microsoft\WindowsApps\python3.11.exe c:\Users\NIKITA\Desktop\Папочки\Лабы\КРИПТО\main.py

==================================================
ШИФР АТБАШ
==================================================
1. Зашифровать
2. Расшифровать
3. Выход

Выберите действие: 1
Введите текст для шифрования: Более всего на свете прокуратор ненавидел запах розового масла, и все теперь предвещало нехороший день, так как запах этот начал преследовать прокуратора с рассвета. Прокур
атору казалось, что розовый запах источают кипарисы и пальмы в саду, что к запаху кожаного снаряжения и пота от конвоя примешивается проклятая розовая струя. От флигелей в тылу дворца, где расположилась
 пришедшая с прокуратором в Ершалаим первая когорта Двенадцатого Молниеносного легиона, заносило дымком в колоннаду через верхнюю площадку сада, и к горьковатому дыму, свидетельствовавшему о том, что ка
шевары в кентуриях начали готовить обед, примешивался все тот же жирный розовый дух. «Да, нет сомнений, это она, опять она, непобедимая, ужасная болезнь гемикрания, при которой болит полголовы от нее не
т средств, нет никакого спасения попробую не двигать головой».

Результат:
КРУЬЪ ЭНЬЬР СЯ НЭЬМЬ ПОРФЛОЯМРО СЪСЯЭЦЫЬУ ЧЯПЯЙ ОРЧРЭРЬР ТЯНУЯ, Ц ЭНЬ МЫПЪОГ ПОЫЫЭЪЁЯУР СЪЙРОРЖЦХ ЫЬСГ, МЯФ ФЯФ ЧЯПЯЙ ВМРМ СЯЗЯУ ПОЫНУЫРЭЯМГ ПОРФЛОЯМРОЯ Н ОЯННЭЬМЯ. ПОРФЛОЯМРОЛ ФЯЧЯУРНГ, ЭМР ОРЧРЭДХ ЧЯ
ПЯЙ ЦНМРЗЯБМ ФЦПЯОЦНД Ц ПЯУГТД Э НЯЫЛ, ЭМР Ф ЧЯПЯЙЛ ФРШЯСРЬР НСЯОАШЬСЦА Ц ПРМЯ РМ ФРСЭРА ПОЦТЪЖЦЭЯЬМНА ПОРФУАМЯА ОРЧРЭЯА НМОЛА. РМ КУЦЬЪУЬХ Э МДУЛ ЫЭРОИЯ, ЫЫЬ ОЯНПРУРШЦУЯНГ ПОЦКЫЖЯА Н ПОРФЛОЯМРОРТ Э ЪО
ЖЯУЯЦТ ПЬОЭЯА ФРЬРОМЯ ЫЭЪСЯЫИЯМРЬР ТРУСЦЬСРНСРЬР УЬЬЦРСЯ, ЧЯСРНЦУР ЫДТФРТ Э ФРУРССЯЫЛ ЗЬОЬЧ ЭЬОЙСББ ПУРЁЯЫФЛ НЯЫЯ, Ц Ф ЪРОГФРЭЯМРТЛ ЫДТЛ, НЭЦЫЫМЬУГНМЭРЭЯЭЖЬТЛ Р МРТ, ЭМР ФЯЖЪЭЯОД Э ФЪСМЛОЦАЙ СЯЗЯУЦ ЬРМР
ЭЦМГ РЮЫ, ПОЦТЪЖЦЭЯУНА ЭНЬ МРМ ШЬ ШЦОСДХ ОРЧРЭДХ ЫЛЙ. «ЫЯ, СЪМ НРТСЪСЦХ, ВМР РСЯ, РПАМГ РСЯ, СЪПРЮЫЦТЯА, ЛШЯНСЯА КРУЬЧСГ ЬЪТЦФОЯСЦА, ПОЦ ФРМРОРХ КРУЦМ ПРУЬРУРЭД РМ СЪЬ СЪМ НОЫНМЭ, СЪМ СЦФЯРЬР НПЯНЬСЦА ПРПОРЮЛБ СЪ ЫЭЦЬЯМГ ЬРУРЭРХ».
```

Рисунок 7. Шифрование АТБАШ на 1000 символов

```
==================================================
ШИФР АТБАШ
==================================================
1. Зашифровать
2. Расшифровать
3. Выход

Выберите действие: 2
Введите текст для расшифрования: КРУЬЪ ЭНЬЬР СЯ НЭЬМЬ ПОРФЛОЯМРО СЪСЯЭЦЫЬУ ЧЯПЯЙ ОРЧРЭРЬР ТЯНУЯ, Ц ЭНЬ МЫПЪОГ ПОЫЫЭЪЁЯУР СЪЙРОРЖЦХ ЫЬСГ, МЯФ ФЯФ ЧЯПЯЙ ВМРМ СЯЗЯУ ПОЫНУЫРЭЯМГ ПОРФЛОЯМРОЯ Н ОЯННЭЬМЯ. ПОР
ФЛОЯМРОЛ ФЯЧЯУРНГ, ЭМР ОРЧРЭДХ ЧЯПЯЙ ЦНМРЗЯБМ ФЦПЯОЦНД Ц ПЯУГТД Э НЯЫЛ, ЭМР Ф ЧЯПЯЙЛ ФРШЯСРЬР НСЯОАШЬСЦА Ц ПРМЯ РМ ФРСЭРА ПОЦТЪЖЦЭЯЬМНА ПОРФУАМЯА ОРЧРЭЯА НМОЛА. РМ КУЦЬЪУЬХ Э МДУЛ ЫЭРОИЯ, ЫЫЬ ОЯНПРУРШЦУ
ЯНГ ПОЦКЫЖЯА Н ПОРФЛОЯМРОРТ Э ЪОЖЯУЯЦТ ПЬОЭЯА ФРЬРОМЯ ЫЭЪСЯЫИЯМРЬР ТРУСЦЬСРНСРЬР УЬЬЦРСЯ, ЧЯСРНЦУР ЫДТФРТ Э ФРУРССЯЫЛ ЗЬОЬЧ ЭЬОЙСББ ПУРЁЯЫФЛ НЯЫЯ, Ц Ф ЪРОГФРЭЯМРТЛ ЫДТЛ, НЭЦЫЫМЬУГНМЭРЭЯЭЖЬТЛ Р МРТ, ЭМР
ФЯЖЪЭЯОД Э ФЪСМЛОЦАЙ СЯЗЯУЦ ЬРМРЭЦМГ РЮЫ, ПОЦТЪЖЦЭЯУНА ЭНЬ МРМ ШЬ ШЦОСДХ ОРЧРЭДХ ЫЛЙ. «ЫЯ, СЪМ НРТСЪСЦХ, ВМР РСЯ, РПАМГ РСЯ, СЪПРЮЫЦТЯА, ЛШЯНСЯА КРУЬЧСГ ЬЪТЦФОЯСЦА, ПОЦ ФРМРОРХ КРУЦМ ПРУЬРУРЭД РМ СЪЬ
СЪМ НОЫНМЭ, СЪМ СЦФЯРЬР НПЯНЬСЦА ПРПОРЮЛБ СЪ ЫЭЦЬЯМГ ЬРУРЭРХ».

Результат:
БОЛЕЕ ВСЕГО НА СВЕТЕ ПРОКУРАТОР НЕНАВИДЕЛ ЗАПАХ РОЗОВОГО МАСЛА, И ВСЕ ТЕПЕРЬ ПРЕДВЕЩАЛО НЕХОРОШИЙ ДЕНЬ, ТАК КАК ЗАПАХ ЭТОТ НАЧАЛ ПРЕСЛЕДОВАТЬ ПРОКУРАТОРА С РАССВЕТА. ПРОКУРАТОРУ КАЗАЛОСЬ, ЧТО РОЗОВЫЙ ЗА
ПАХ ИСТОЧАЮТ КИПАРИСЫ И ПАЛЬМЫ В САДУ, ЧТО К ЗАПАХУ КОЖАНОГО СНАРЯЖЕНИЯ И ПОТА ОТ КОНВОЯ ПРИМЕШИВАЕТСЯ ПРОКЛЯТАЯ РОЗОВАЯ СТРУЯ. ОТ ФЛИГЕЛЕЙ В ТЫЛУ ДВОРЦА, ГДЕ РАСПОЛОЖИЛАСЬ ПРИШЕДШАЯ С ПРОКУРАТОРОМ В ЕР
ШАЛАИМ ПЕРВАЯ КОГОРТА ДВЕНАДЦАТОГО МОЛНИЕНОСНОГО ЛЕГИОНА, ЗАНОСИЛО ДЫМКОМ В КОЛОННАДУ ЧЕРЕЗ ВЕРХНЮЮ ПЛОЩАДКУ САДА, И К ГОРЬКОВАТОМУ ДЫМУ, СВИДЕТЕЛЬСТВОВАВШЕМУ О ТОМ, ЧТО КАШЕВАРЫ В КЕНТУРИЯХ НАЧАЛИ ГОТО
ВИТЬ ОБЕД, ПРИМЕШИВАЛСЯ ВСЕ ТОТ ЖЕ ЖИРНЫЙ РОЗОВЫЙ ДУХ. «ДА, НЕТ СОМНЕНИЙ, ЭТО ОНА, ОПЯТЬ ОНА, НЕПОБЕДИМАЯ, УЖАСНАЯ БОЛЕЗНЬ ГЕМИКРАНИЯ, ПРИ КОТОРОЙ БОЛИТ ПОЛГОЛОВЫ ОТ НЕЕ НЕТ СРЕДСТВ, НЕТ НИКАКОГО СПАСЕН
ИЯ ПОПРОБУЮ НЕ ДВИГАТЬ ГОЛОВОЙ».

==================================================
ШИФР АТБАШ
==================================================
1. Зашифровать
2. Расшифровать
3. Выход

Выберите действие:
```

Рисунок 8. Расшифровывание АТБАШ на 1000 символов

Блок B: Шифры многозначной замены

5. Шифр Белазо

Шифр Белазо — это шифр подстановки. Он является усовершенствованием шифра Цезаря и предшественником шифра Виженера. В отличие от шифра Цезаря, где сдвиг постоянен для всего сообщения, шифр Белазо использует ключевое слово, которое определяет переменный сдвиг для каждого символа.

Обозначения:

P - открытый текст

C - зашифрованный текст

K - ключ

m - мощность алфавита

i - позиция символа в тексте (начало 0)

$C_i = (P_i + K\ \{i \bmod |K|\}) \bmod m$ - формула для шифра Белазо

Блок-схема:

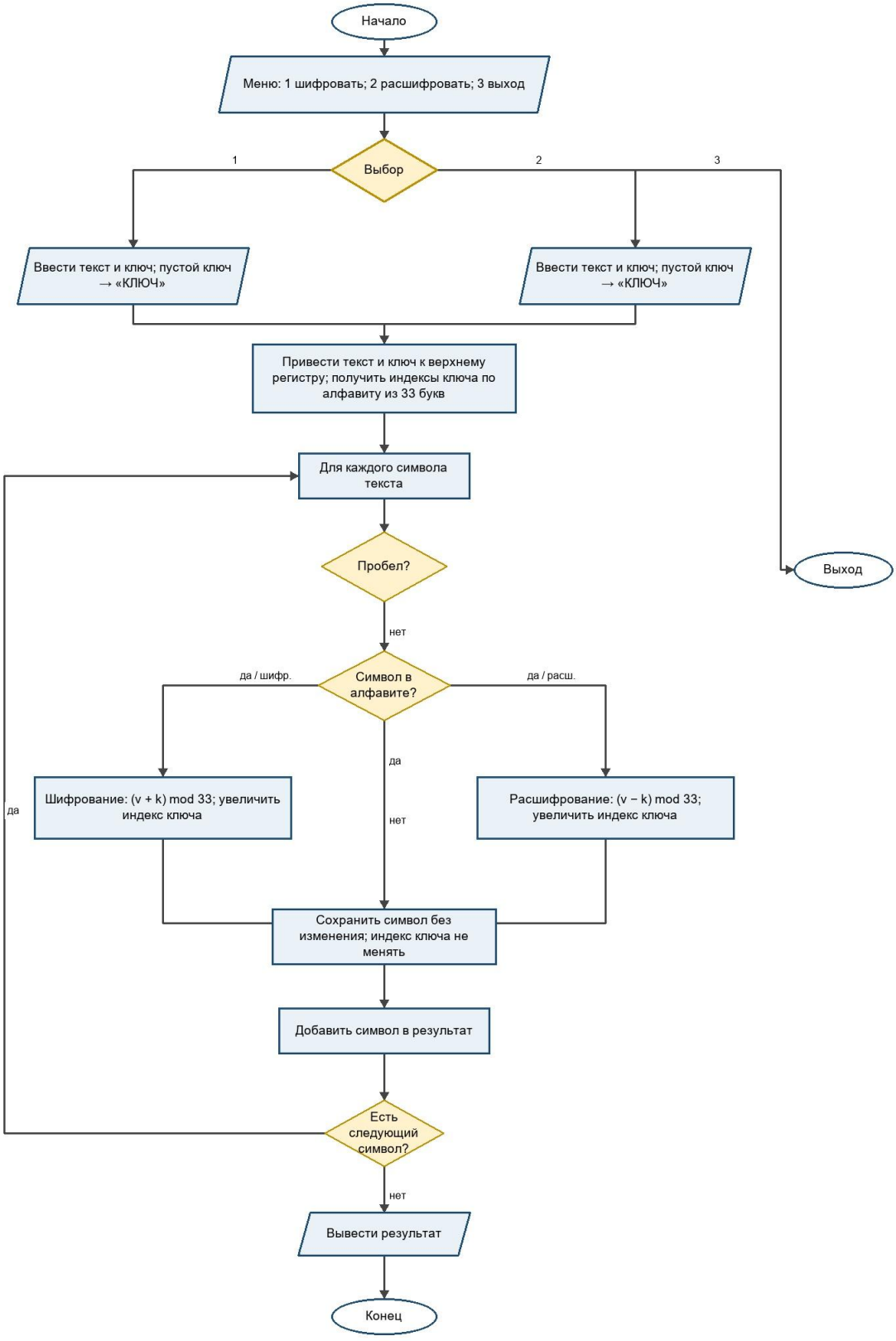


Рисунок 9. Блок-схема шифра Белазо.

Код программы:

```
# -*- coding: utf-8 -*-
"""
ПРОСТОЙ ШИФР БЕЛАЗО
"""
ALPHABET = 'АБВГДЕЁЖЗИЙКЛМНОПРСТУФХЦЧШЩЪЫЬЭЮЯ'
N = 33
char_to_num = {char: i for i, char in enumerate(ALPHABET)}
num_to_char = {i: char for i, char in enumerate(ALPHABET)}
KEYWORD = "КЛЮЧ"
def get_key_nums(keyword):
    return [char_to_num[char] for char in keyword.upper() if char in char_to_num]
def encrypt(text, keyword=KEYWORD):
    text = text.upper()
    key_nums = get_key_nums(keyword)
    key_length = len(key_nums)
    result = []
    key_idx = 0
    for char in text:
        if char == ' ':
            result.append(' ')
        elif char in char_to_num:
            vi = char_to_num[char]
            ki = key_nums[key_idx % key_length]
            wi = (vi + ki) % N
            result.append(num_to_char[wi])
            key_idx += 1
        else:
            result.append(char)
    return ''.join(result)
def decrypt(ciphertext, keyword=KEYWORD):
    ciphertext = ciphertext.upper()
    key_nums = get_key_nums(keyword)
    key_length = len(key_nums)
    result = []
    key_idx = 0
    for char in ciphertext:
        if char == ' ':
            result.append(' ')
        elif char in char_to_num:
            wi = char_to_num[char]
            ki = key_nums[key_idx % key_length]
            vi = (wi - ki) % N
            result.append(num_to_char[vi])
            key_idx += 1
        else:
            result.append(char)
    return ''.join(result)
while True:
    print("\n" + "=" * 50)
    print("ПРОСТОЙ ШИФР БЕЛАЗО")
    print("=" * 50)
    print("1. Зашифровать")
    print("2. Расшифровать")
    print("3. Выход")
    choice = input("\nВыберите действие: ")
    if choice == "1":
        text = input("Введите текст для шифрования: ")
        key = input("Введите ключ (Enter = КЛЮЧ): ").strip()
        if not key:
            key = KEYWORD
        encrypted = encrypt(text, key)
        print("\nРезультат шифрования:")
        print(encrypted)
    elif choice == "2":
        text = input("Введите текст для расшифрования: ")
        key = input("Введите ключ (Enter = КЛЮЧ): ").strip()
        if not key:
            key = KEYWORD
        decrypted = decrypt(text, key)
```

```
            print("\nРезультат расшифрования:")
            print(decrypted)
        elif choice == "3":
            print("Выход из программы...")
            break
        else:
            print("Неверный выбор!")
```

Результат работы программы:

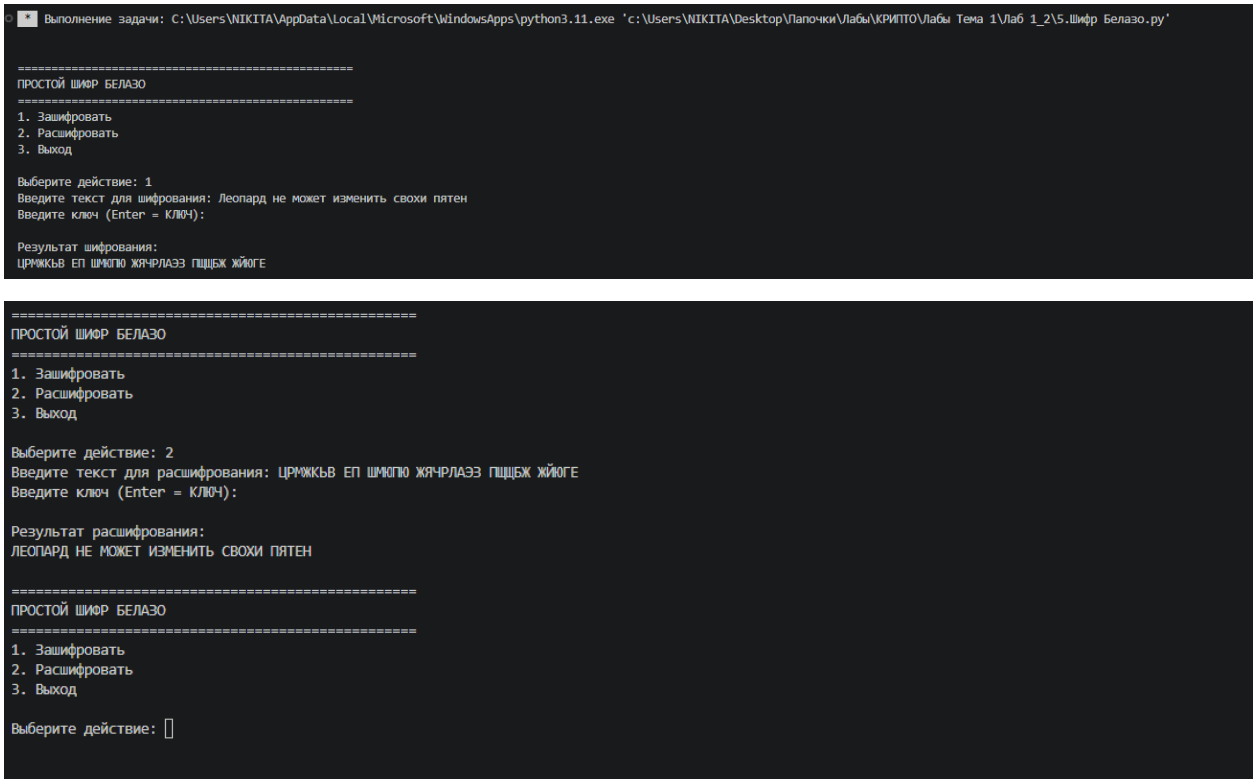


Рисунок 10. Результат работы шифра Белазо.

Тест на 1000 знаков:

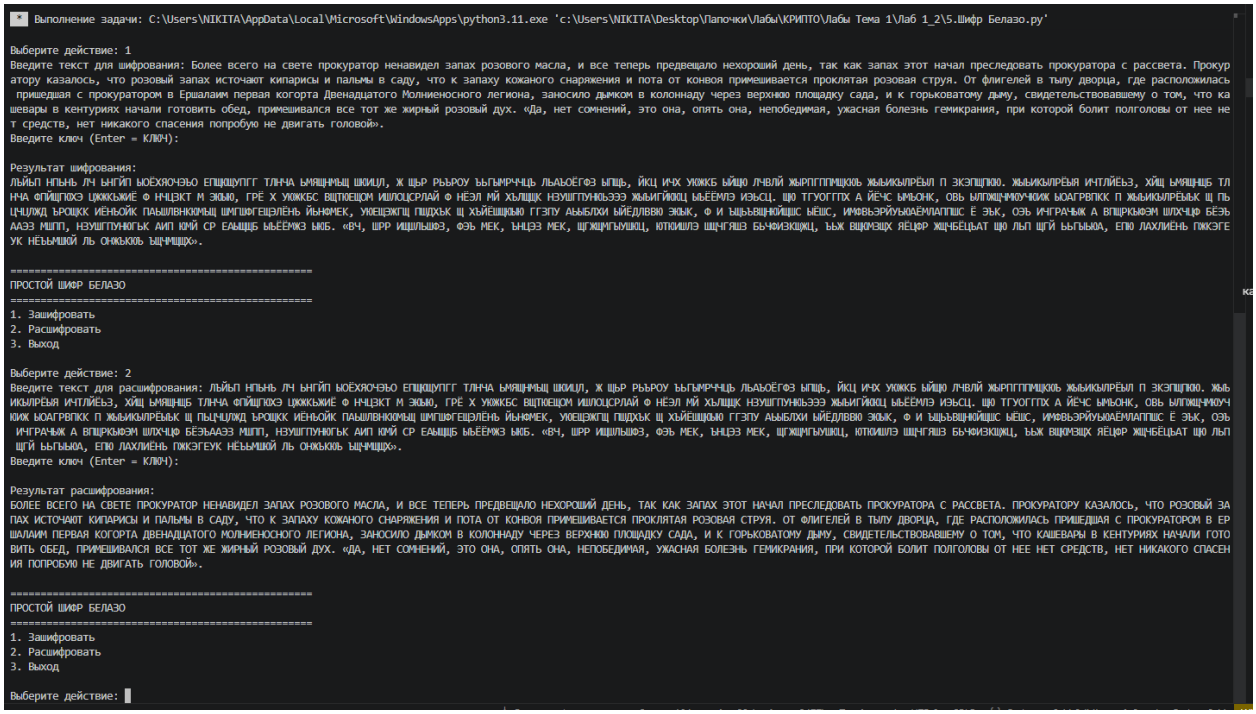
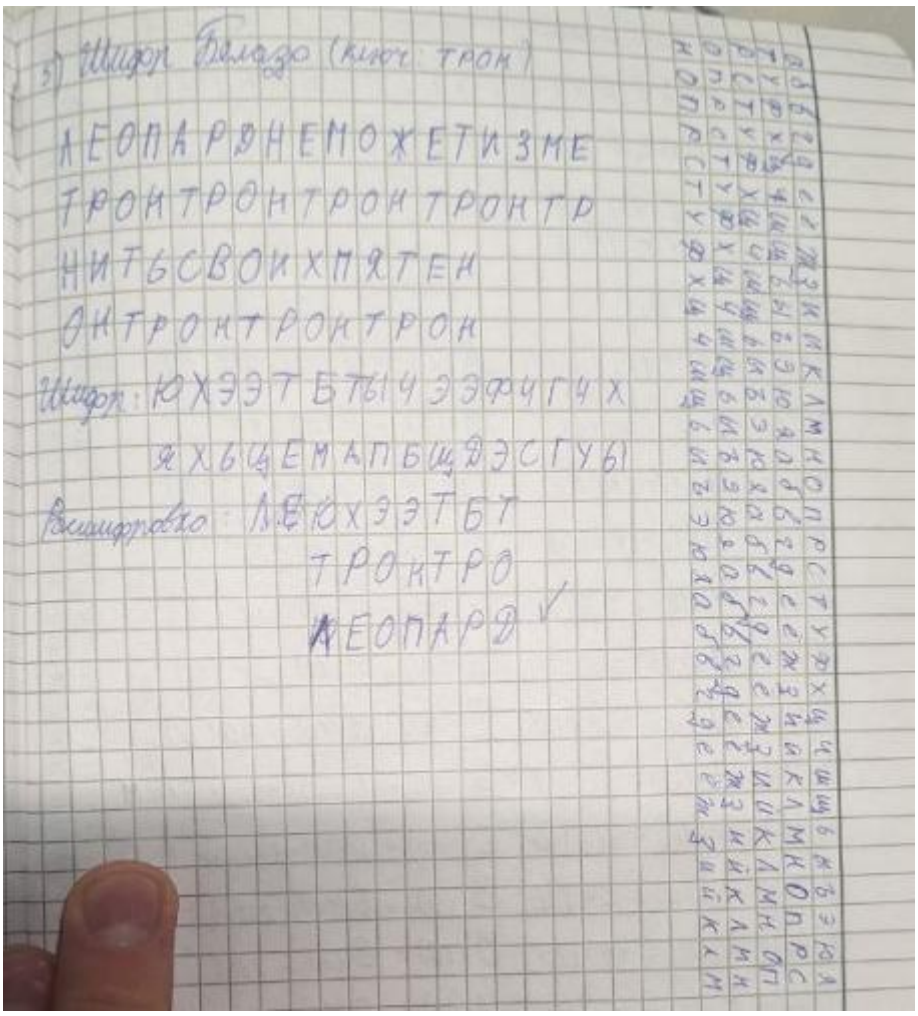


Рисунок 12. Тест на 1000 символов шифра Белазо.


5) Шифр Белазо (ключ: трон)
АЕОПАРДНЕМОЖЕТИЗМЕ
ТРОНТРОНТРОНТРОНТРО
НИТЬСВОИХМЯТЕН
ОНТРОНТРОНТРОН
Шифр: ЮХЭЭТБТЫЧЭЭФЧГЧХ
ЯХЬЦЕМАПЬЩДЭСГУЬ
Расшифровка: АЕЮХЭЭТБТ
ТРОНТРО
АЕОПАРД

Блок С: Шифры блочной замены

8. Матричный шифр

Матричный шифр (также известен как шифр Хилла) — это полиграммный шифр подстановки. Он является одним из первых шифров, которые могли работать с блоками символов (биграммами, триграммами).

Ключевая особенность: Шифр Хилла использует линейную алгебру и матричное умножение для шифрования целых блоков символов одновременно.

Обозначения:

$C = (K \times P) \bmod m$

P — вектор-столбец открытого текста (размер n×1)

K — ключевая матрица (размер n×n)

C — вектор-столбец зашифрованного текста (размер n×1)

m — мощность алфавита (32 для русского языка)

mod m — операция взятия остатка по модулю m

Блок-схема:

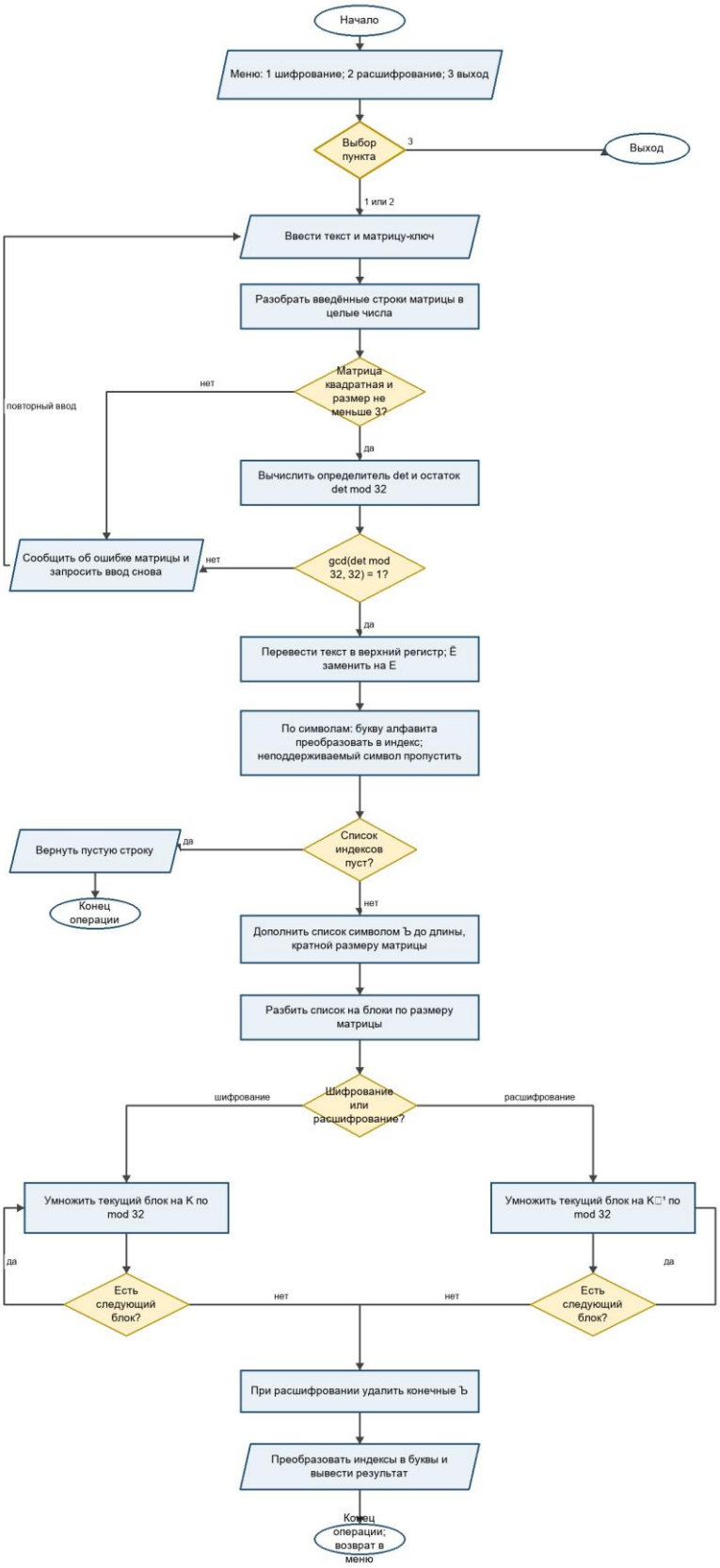


Рисунок 13. Блок-схема матричного шифра.

Код программы:

```
# МАТРИЧНЫЙ ШИФР (Шифр Хилла)
# Вариант 8: Матрица-ключ не меньше 3×3

import numpy as np
import math

# Русский алфавит (32 буквы, без Ё)
RUSSIAN_ALPHABET = 'АБВГДЕЖЗИЙКЛМНОПРСТУФХЦЧШЩЪЫЬЭЮЯ'
ALPHABET_SIZE = 32

class MatrixCipher:
    """Матричный шифр (шифр Хилла)"""

    def __init__(self, matrix):
        self.matrix = np.array(matrix, dtype=int)
        self.size = len(matrix)
        self.mod = ALPHABET_SIZE
        self.inverse_matrix = self._mod_inverse_matrix()

    def _mod_inverse(self, a, m):
        """Находит обратное число по модулю"""
        a = a % m
        for x in range(1, m):
            if (a * x) % m == 1:
                return x
        return None

    def _mod_inverse_matrix(self):
        """Находит обратную матрицу по модулю"""
        det = int(round(np.linalg.det(self.matrix)))
        det_mod = det % self.mod

        # Проверяем, что определитель обратим по модулю
        det_inv = self._mod_inverse(det_mod, self.mod)
        if det_inv is None:
            raise ValueError(f"Матрица необратима по модулю {self.mod}")

        # Для матрицы 3x3 вычисляем присоединенную матрицу
        if self.size == 3:
            adjugate = self._adjugate_3x3(self.matrix)
        else:
            adjugate = np.round(det * np.linalg.inv(self.matrix)).astype(int)

        # Умножаем на обратный определитель по модулю
        inverse = (det_inv * adjugate) % self.mod
        return inverse.astype(int)

    def _adjugate_3x3(self, matrix):
        """Вычисляет присоединенную матрицу для 3x3"""
        m = matrix
        adj = np.zeros((3, 3), dtype=int)

        # Формулы для присоединенной матрицы 3x3
        adj[0][0] = (m[1][1] * m[2][2] - m[1][2] * m[2][1])
        adj[1][0] = -(m[1][0] * m[2][2] - m[1][2] * m[2][0])
        adj[2][0] = (m[1][0] * m[2][1] - m[1][1] * m[2][0])

        adj[0][1] = -(m[0][1] * m[2][2] - m[0][2] * m[2][1])
        adj[1][1] = (m[0][0] * m[2][2] - m[0][2] * m[2][0])
        adj[2][1] = -(m[0][0] * m[2][1] - m[0][1] * m[2][0])

        adj[0][2] = (m[0][1] * m[1][2] - m[0][2] * m[1][1])
        adj[1][2] = -(m[0][0] * m[1][2] - m[0][2] * m[1][0])
        adj[2][2] = (m[0][0] * m[1][1] - m[0][1] * m[1][0])

        return adj

    def _text_to_numbers(self, text):
        """Преобразует текст в числа (А=0, Б=1, ...)"""
        text = text.upper()
        numbers = []
        for char in text:
            if char in RUSSIAN_ALPHABET:
                numbers.append(RUSSIAN_ALPHABET.index(char))
            elif char == 'Ё':
                numbers.append(RUSSIAN_ALPHABET.index('Е'))
            elif char.isalpha():
                # Для английских букв
                numbers.append(ord(char) % self.mod)
        return numbers

    def _numbers_to_text(self, numbers):
        """Преобразует числа обратно в текст"""
        text = ''
        for num in numbers:
```

```
            text += RUSSIAN_ALPHABET[num % self.mod]
        return text

    def encrypt(self, text):
        """Шифрование текста"""
        # Преобразуем текст в числа
        numbers = self._text_to_numbers(text)

        if not numbers:
            return ""

        # Дополняем до длины, кратной размеру матрицы
        while len(numbers) % self.size != 0:
            numbers.append(RUSSIAN_ALPHABET.index('ъ'))

        # Шифруем по блокам
        encrypted = []
        for i in range(0, len(numbers), self.size):
            block = numbers[i:i + self.size]
            encrypted_block = np.dot(self.matrix, block) % self.mod
            encrypted.extend(encrypted_block)

        return self._numbers_to_text(encrypted)

    def decrypt(self, text, remove_padding=True):
        """Расшифрование текста"""
        # Преобразуем текст в числа
        numbers = self._text_to_numbers(text)

        if not numbers:
            return ""

        # Проверяем длину
        if len(numbers) % self.size != 0:
            while len(numbers) % self.size != 0:
                numbers.append(RUSSIAN_ALPHABET.index('ъ'))

        # Расшифровываем по блокам
        decrypted = []
        for i in range(0, len(numbers), self.size):
            block = numbers[i:i + self.size]
            decrypted_block = np.dot(self.inverse_matrix, block) % self.mod
            decrypted.extend(decrypted_block)

        result = self._numbers_to_text(decrypted)

        # Убираем дополнение
        if remove_padding:
            result = result.rstrip('ъ')

        return result

def validate_matrix(matrix):
    """Проверяет, что матрица подходит для шифрования"""
    try:
        matrix = np.array(matrix)
        det = int(round(np.linalg.det(matrix)))

        # Определитель не должен быть равен 0
        if det == 0:
            return False, f"Определитель матрицы = 0 (матрица необратима)"

        # Определитель должен быть взаимно прост с размером алфавита
        det_mod = det % ALPHABET_SIZE
        if math.gcd(det_mod, ALPHABET_SIZE) != 1:
            return False, f"Определитель матрицы ({det}) не взаимно прост с {ALPHABET_SIZE}"

        return True, f"Матрица валидна (определитель = {det})"
    except Exception as e:
        return False, f"Ошибка при проверке матрицы: {e}"

def print_matrix(matrix):
    """Красиво выводит матрицу"""
    print("\nМатрица-ключ:")
    for row in matrix:
        print("  " + " ".join(f"{x:3}" for x in row))

def main():
    while True:
        print("\n" + "=" * 50)
        print("МАТРИЧНЫЙ ШИФР (Шифр Хилла)")
        print("=" * 50)
        print("1. Зашифровать")
        print("2. Расшифровать")
        print("3. Выход")

        choice = input("\nВыберите действие: ").strip()

        if choice == "1":
            text = input("Введите текст для шифрования: ").strip()
            print("\nВведите матрицу-ключ")
            print("Пример: 3 2 1, 1 1 1, 1 2 3")
```

```
            try:
                matrix_str = input("Матрица: ").strip()
                rows = matrix_str.split(',')
                matrix = []
                for row in rows:
                    matrix.append(
                        list(map(int, row.strip().split()))
                    )
                is_valid, message = validate_matrix(matrix)
                if not is_valid:
                    print(f"Ошибка: {message}")
                    continue
                cipher = MatrixCipher(matrix)
                encrypted = cipher.encrypt(text)
                print("\nРезультат шифрования:")
                print(encrypted)
            except Exception as e:
                print(f"Ошибка: {e}")
        elif choice == "2":
            text = input("Введите текст для расшифрования: ").strip()
            print("\nВведите ту же матрицу-ключ")
            try:
                matrix_str = input("Матрица: ").strip()
                rows = matrix_str.split(',')
                matrix = []
                for row in rows:
                    matrix.append(
                        list(map(int, row.strip().split()))
                    )
                is_valid, message = validate_matrix(matrix)
                if not is_valid:
                    print(f"Ошибка: {message}")
                    continue
                cipher = MatrixCipher(matrix)
                decrypted = cipher.decrypt(text)
                print("\nРезультат расшифрования:")
                print(decrypted)
            except Exception as e:
                print(f"Ошибка: {e}")
        elif choice == "3":
            print("Выход из программы...")
            break
        else:
            print("Неверный выбор!")
if __name__ == "__main__":
    main()
```

Результат работы программы:

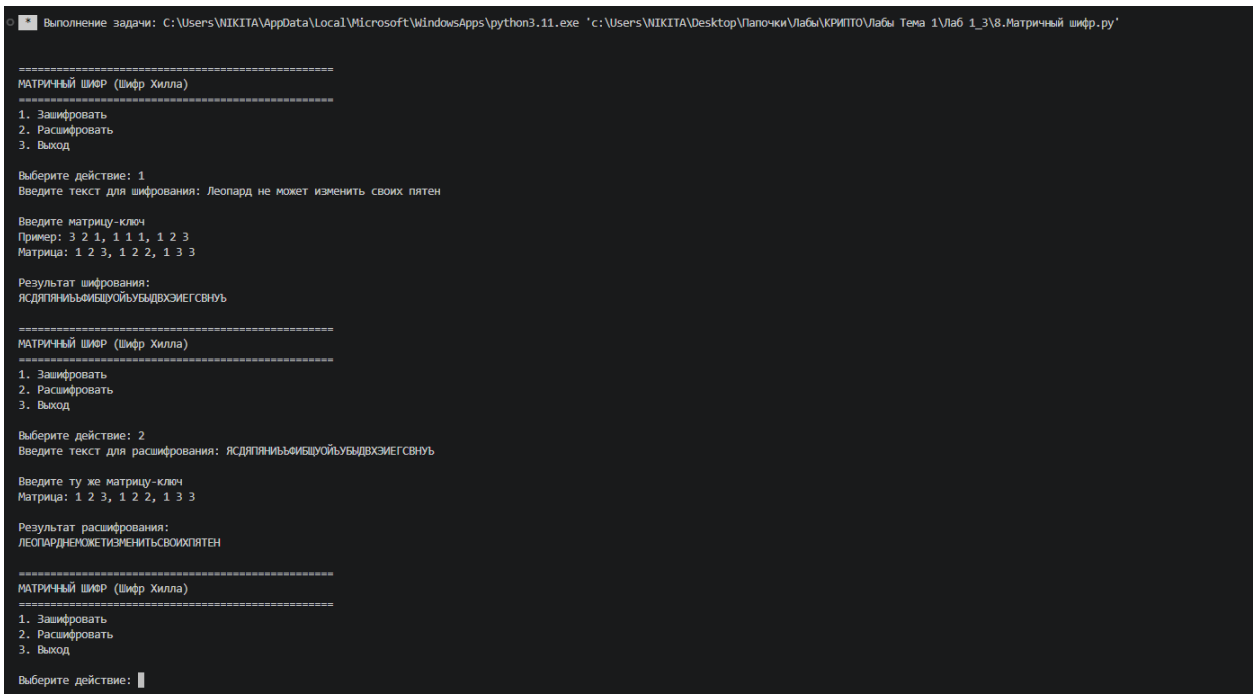


Рисунок 14. Результат работы матричного шифра.

Тест на 1000 знаков:

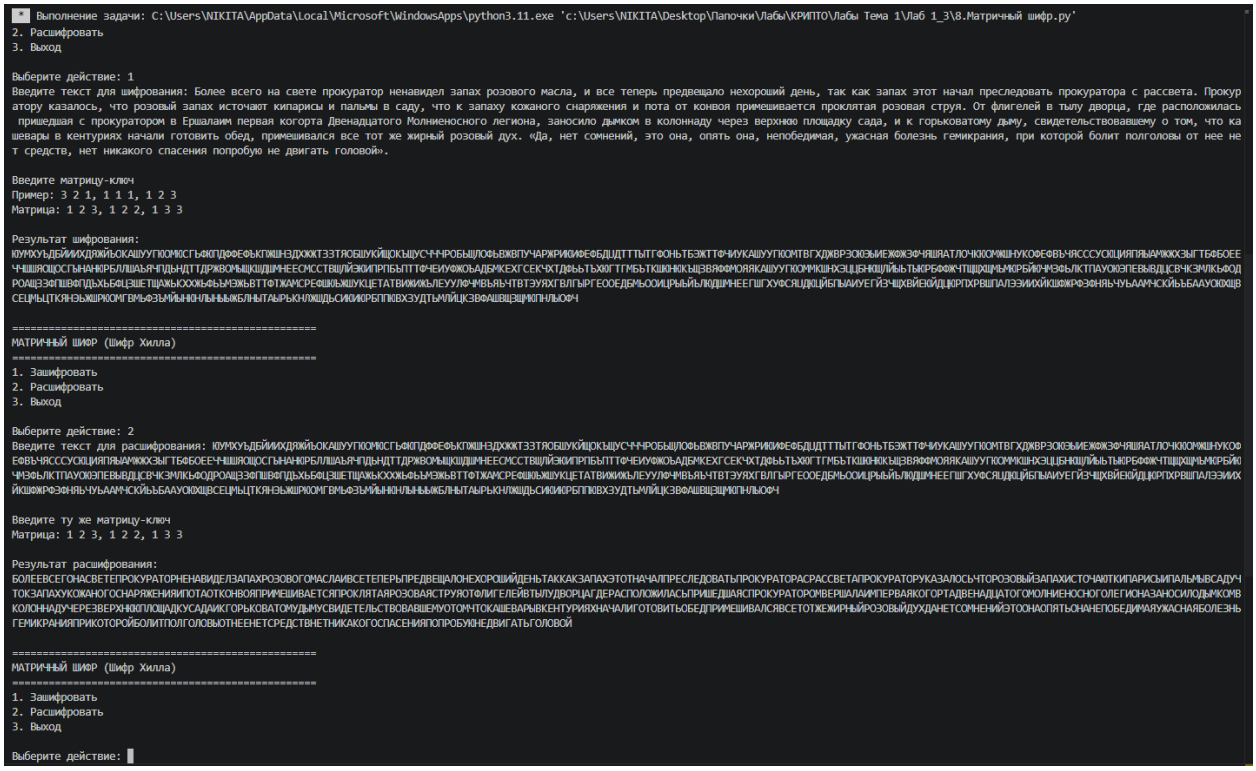


Рисунок 16. Тест на 1000 символов матричного шифра.

Блок D: Шифры перестановки

10. Вертикальная перестановка

Шифр вертикальной перестановки — это метод шифрования, при котором символы исходного сообщения записываются в таблицу по строкам, а затем считываются по столбцам в порядке, определяемом ключевым словом. Это классический транспозиционный шифр, который не заменяет символы, а только меняет их порядок.

Блок-схема:

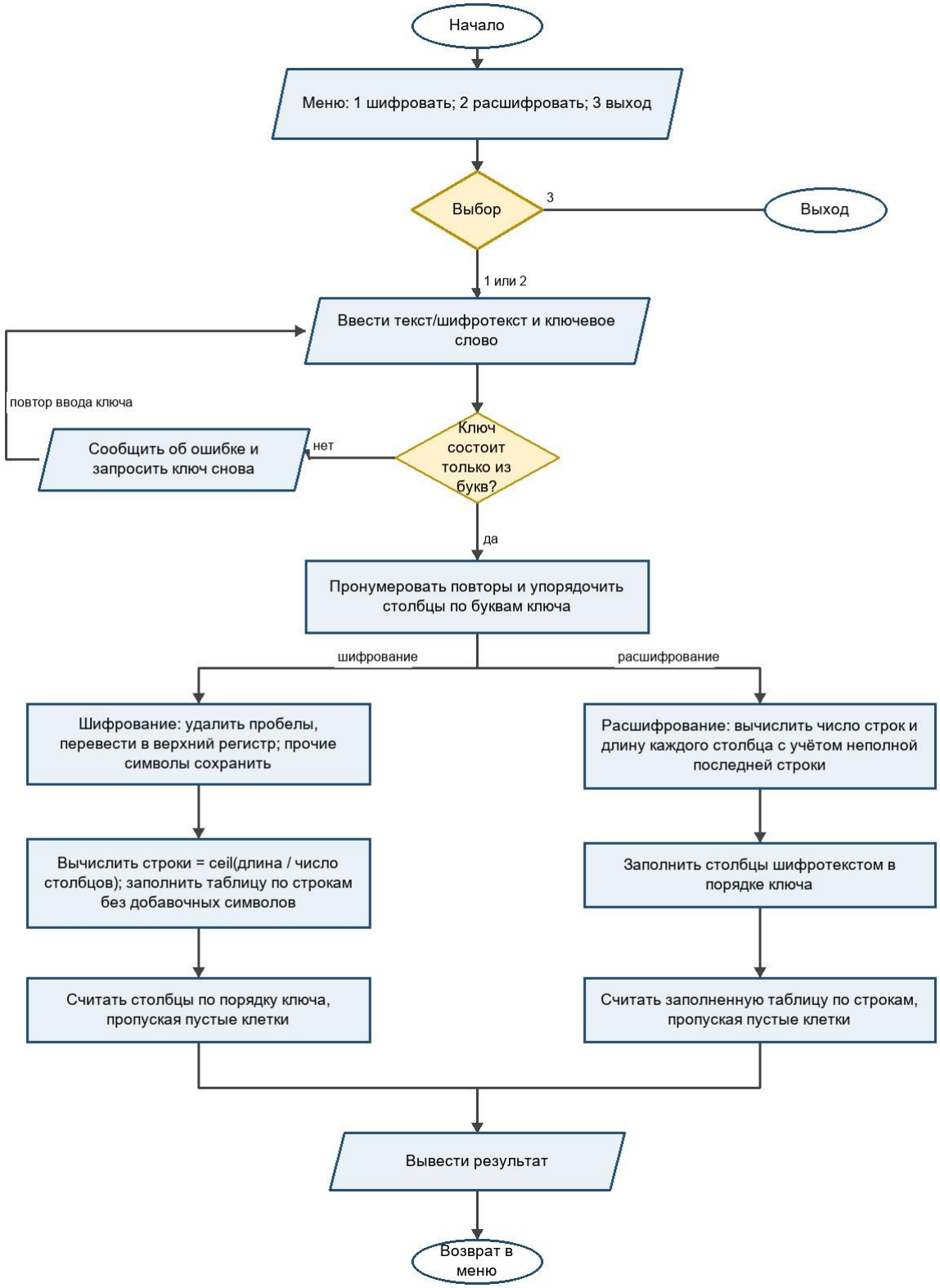


Рисунок 17. Блок-схема вертикальной перестановки.

Код программы:

```
# Шифр вертикальной перестановки (Вариант 10)
# Ключ - слово "МАГМА" (буквы могут повторяться)
# Пустые клетки НЕ ЗАПОЛНЯЮТСЯ
def proverka_klucha(key):
    """Проверка ключа-слова"""
    if not key.isalpha():
        print("❌ Ошибка: Ключ должен состоять только из букв!")
        return False
    print(f"☑ Ключ '{key}' принят!")
    return True
def poluchit_poryadok_stolbcov(key):
    """
    Получает порядок столбцов на основе алфавитного порядка букв ключа
    Для повторяющихся букв использует порядковый номер (1,2,3...)
    """
    print(f"\n🔑 Анализ ключа '{key}':")
    print("-" * 50)
    print("Исходная позиция | Буква | Номер повтора | Новый порядок")
    print("-" * 50)
    # Считаем сколько раз встретилась каждая буква
    letter_count = {}
    letters_with_pos = []
    for i, letter in enumerate(key):
        # Увеличиваем счетчик для буквы
        if letter in letter_count:
            letter_count[letter] += 1
        else:
            letter_count[letter] = 1
        # Запоминаем букву, позицию и номер повтора
        letters_with_pos.append((letter, i, letter_count[letter]))
    # Сортируем: сначала по букве, потом по номеру повтора
    # Для одинаковых букв меньший номер повтора идет первым
    sorted_letters = sorted(letters_with_pos, key=lambda x: (x[0], x[2]))
    # Создаем словарь: исходная позиция -> новый порядок
    order = {}
    for new_pos, (letter, old_pos, repeat_num) in enumerate(sorted_letters):
        order[old_pos] = new_pos + 1  # +1 для человеко-читаемого формата
    # Создаем ключ-цифру
    numeric_key = ''.join(str(order[i]) for i in range(len(key)))
    # Выводим таблицу анализа
    for i in range(len(key)):
        letter = key[i]
        # Находим номер повтора для этой позиции
        repeat = 1
        for j in range(i):
            if key[j] == letter:
                repeat += 1
        print(f"       {i+1}         |   {letter}   |       {repeat}       |       {order[i]}")
    print("-" * 50)
    print(f"👉 Ключ для шифрования: {numeric_key}")
    return numeric_key, sorted_letters
def shifrovat(text, word_key):
    """Шифрование вертикальной перестановкой (без заполнения пустых клеток)"""
    print("\n" + "="*70)
    print("🔐 НАЧИНАЕМ ШИФРОВАНИЕ")
    print("="*70)
    # Получаем числовой ключ из слова
    numeric_key, sorted_letters = poluchit_poryadok_stolbcov(word_key)
    # Шаг 1: Подготавливаем текст
    original_text = text
    text = text.replace(" ", "").upper()
    print(f"\n📝 Исходный текст: '{original_text}'")
    print(f"1. Текст без пробелов (upper): {text}")
    print(f"   Длина текста: {len(text)} символов")
    # Шаг 2: Определяем размер таблицы
    cols = len(word_key)  # количество столбцов = длина ключа
    rows = (len(text) + cols - 1) // cols  # количество строк
    print(f"2. Таблица будет размером {rows} строк × {cols} столбцов")
    print(f"   ⚠️ Пустые клетки останутся пустыми (не заполняются)")
```

```
    # Шаг 3: Записываем текст в таблицу по строкам
    print("\n3. Заполняем таблицу по строкам (пустые клетки - '.'):")
    print("-" * 70)

    # Печатаем заголовок с ключом
    print("   ", end="")
    for i, letter in enumerate(word_key):
        print(f"  {letter}({i+1}) ", end="")
    print("\n   " + "-" * (cols * 7))

    matrix = []
    index = 0
    total_chars = len(text)

    for i in range(rows):
        row = []
        print(f"   |", end="")
        for j in range(cols):
            if index < total_chars:
                row.append(text[index])
                print(f"  {text[index]}  |", end="")
                index += 1
            else:
                row.append('.')  # Пустая клетка
                print(f"  .  |", end="")
        matrix.append(row)
        print(f"  строка {i+1}")
        print("   " + "-" * (cols * 7))

    # Шаг 4: Читаем по столбцам в порядке, определенном ключом
    print("\n4. Читаем столбцы в порядке сортировки букв ключа:")
    print("   (пропуская пустые клетки)")

    result = ""
    print("\n   Порядок чтения столбцов:")
    for new_pos, (letter, col, repeat_num) in enumerate(sorted_letters):
        column_text = ""
        print(f"   {new_pos+1}. Столбец {col+1} (буква '{letter}' №{repeat_num}): ", end="")
        for row in range(rows):
            if matrix[row][col] != '.':  # Пропускаем пустые клетки
                result += matrix[row][col]
                column_text += matrix[row][col]
        print(column_text)

    print("\n" + "="*70)
    print(f"🔓 РЕЗУЛЬТАТ ШИФРОВАНИЯ: {result}")
    print("="*70)

    return result, numeric_key, sorted_letters, rows, cols

def rasshifrovat(cipher, word_key, numeric_key, sorted_letters, rows, cols):
    """Расшифрование вертикальной перестановки (с пустыми клетками)"""
    print("\n" + "="*70)
    print("🔓 НАЧИНАЕМ РАСШИФРОВАНИЕ")
    print("="*70)

    print(f"1. Таблица {rows}×{cols} (с пустыми клетками)")
    print(f"2. Шифротекст: {cipher}")
    print(f"3. Ключ-слово: {word_key}")
    print(f"4. Числовой ключ: {numeric_key}")

    # Создаем пустую таблицу
    matrix = [['.' for _ in range(cols)] for _ in range(rows)]

    # Сначала нужно определить, в каких столбцах есть пустые клетки
    # Вычисляем сколько символов в каждом столбце
    total_chars = len(cipher)
    chars_per_column = [rows] * cols  # по умолчанию во всех столбцах rows символов

    # В последних столбцах может быть меньше символов
    empty_in_last_row = cols * rows - total_chars
    if empty_in_last_row > 0:
        # Пустые клетки в последней строке, начиная с последних столбцов
        for i in range(empty_in_last_row):
            chars_per_column[cols - 1 - i] -= 1

    print(f"\n   Количество символов в каждом столбце: {chars_per_column}")

    # Заполняем по столбцам в порядке, определенном ключом
    index = 0
    print("\n5. Заполняем таблицу по столбцам в порядке сортировки букв:")

    for new_pos, (letter, col, repeat_num) in enumerate(sorted_letters):
        print(f"\n   {new_pos+1}. Столбец {col+1} (буква '{letter}' №{repeat_num}): ", end="")
        # Сколько символов нужно записать в этот столбец
        chars_in_this_col = chars_per_column[col]
        for row in range(chars_in_this_col):
            matrix[row][col] = cipher[index]
            print(f"{cipher[index]}", end="")
            index += 1
```

```
        # Остальные клетки в столбце остаются '.'
    # Показываем заполненную таблицу
    print("\n\n6. Заполненная таблица:")
    print("-" * 70)
    print("   ", end="")
    for i, letter in enumerate(word_key):
        print(f"  {letter}({i+1}) ", end="")
    print("\n   " + "-" * (cols * 7))
    for i in range(rows):
        print(f"   |", end="")
        for j in range(cols):
            print(f"  {matrix[i][j]}  |", end="")
        print(f"  строка {i+1}")
        print("   " + "-" * (cols * 7))
    # Читаем по строкам (пропуская пустые клетки)
    print("\n7. Читаем результат по строкам (пропуская пустые клетки):")
    result = ""
    for i in range(rows):
        row_text = ""
        for j in range(cols):
            if matrix[i][j] != '.':
                result += matrix[i][j]
                row_text += matrix[i][j]
        print(f"   Строка {i+1}: {row_text}")
    print(f"\n8. Результат: {result}")
    return result
# ============ ГЛАВНАЯ ПРОГРАММА ============
def main():
    while True:
        print("\n" + "=" * 50)
        print("ШИФР ВЕРТИКАЛЬНОЙ ПЕРЕСТАНОВКИ")
        print("=" * 50)
        print("1. Зашифровать")
        print("2. Расшифровать")
        print("3. Выход")
        choice = input("\nВыберите действие: ").strip()
        if choice == "1":
            text = input("Введите текст для шифрования: ")
            while True:
                key = input("Введите ключевое слово: ").upper()
                if proverka_klucha(key):
                    break
            cipher, numeric_key, sorted_letters, rows, cols = shifrovat(text, key)
            print("\nРезультат шифрования:")
            print(cipher)
        elif choice == "2":
            cipher = input("Введите шифротекст: ")
            while True:
                key = input("Введите ключевое слово: ").upper()
                if proverka_klucha(key):
                    break
            numeric_key, sorted_letters = poluchit_poryadok_stolbcov(key)
            cols = len(key)
            rows = (len(cipher) + cols - 1) // cols
            decrypted = rasshifrovat(
                cipher,
                key,
                numeric_key,
                sorted_letters,
                rows,
                cols
            )
            print("\nРезультат расшифрования:")
            print(decrypted)
        elif choice == "3":
            print("Выход из программы...")
            break
        else:
            print("Неверный выбор!")
if __name__ == "__main__":
    main()
```

Результат работы программы:

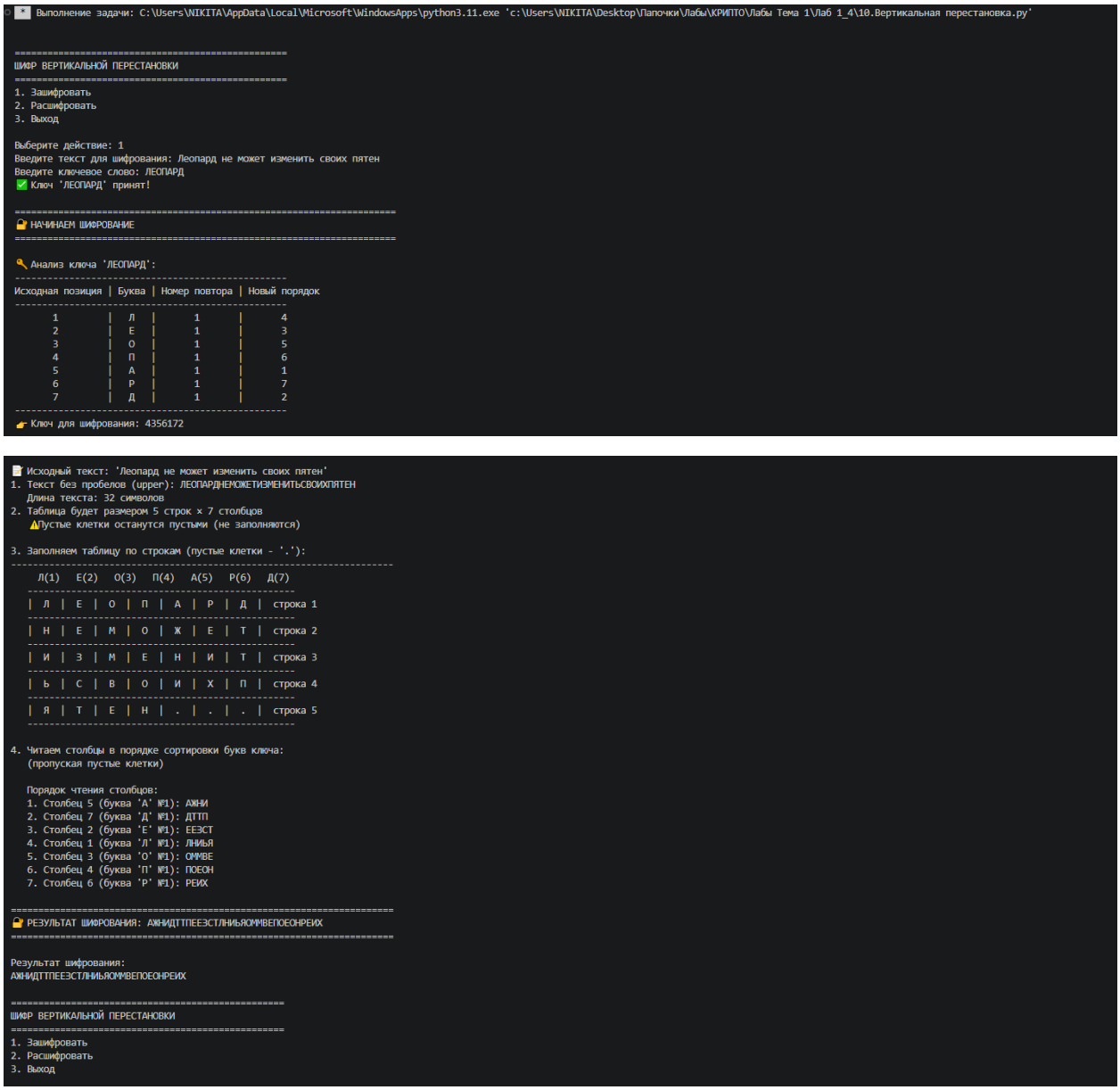


Рисунок 18. Работа шифра вертикальной перестановки.

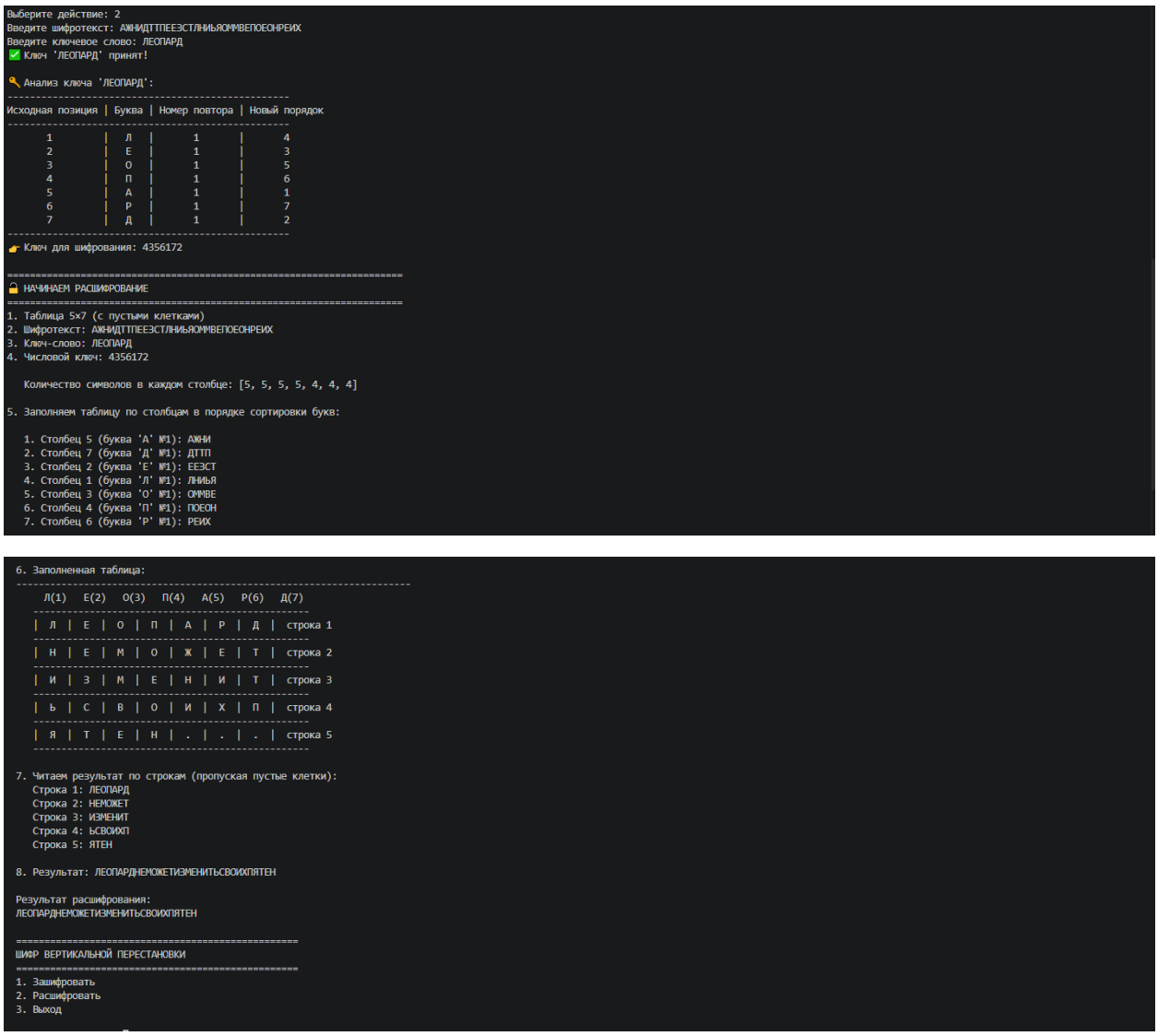
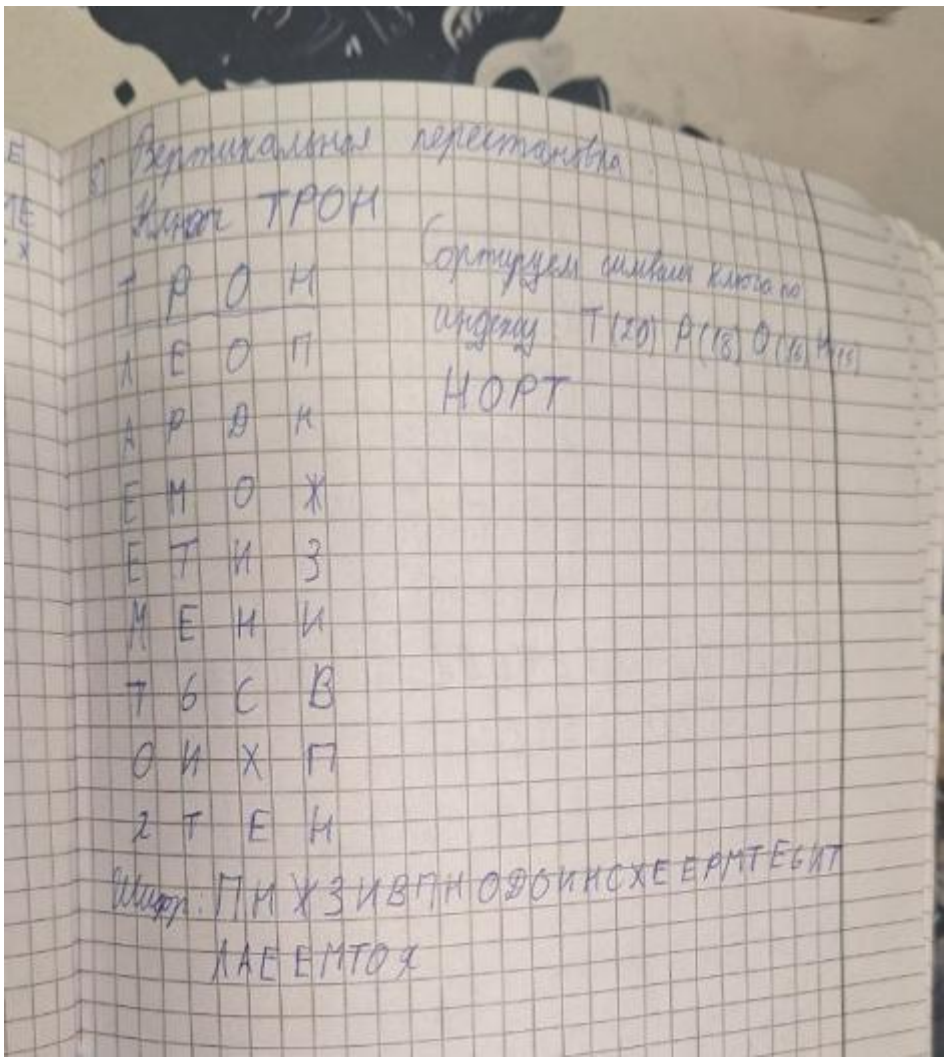


Рисунок 19. Работа шифра вертикальной перестановки (2).


Вертикальная перестановка
Ключ ТРОН
Сортируем имена ключа по индексу Т[20] Р[18] О[16] Н[15]
НОРТ

Блок E: Шифры гаммирования

13. Одноразовый блокнот Шеннона

Открытый текст сообщения m записывают, как последовательность бит или символов m = m0m1…mn-1, а двоичную или символьную шифрующую последовательность k той же самой длины — как k = k0k1…kn-1. Шифртекст c = c0c1…cn-1 определяется соотношением ci = mi ki при 0≤i≤n-1, где обозначает операцию «исключающее ИЛИ» (ассемблерная операция XOR) по модулю два или сложение по любому другому модулю в случае символьной гаммы.

Для генерации ПСЧ могут использоваться конгруэнтные генераторы, например, линейный генератор: T(i+1) = (aT(i) + c) mod m, где a и c – константы, T(0) – исходная величина, выбранная в качестве порождающего числа. Период ограничен модулем и не может быть больше m. Максимальный период равен m.

Период генерируемых чисел зависит от величины a и c и является максимальным тогда и только тогда, когда c – нечетное число, a (mod 4) = 1. В случае символьной гаммы величина m определяется размером алфавита, а период гаммы также параметрами a и c:

a – нечетное число

c – взаимно прост с модулем m

b = a – 1 кратно p для каждого простого p, делителя m b кратно 4, если m кратно 4.

Блок-схема:

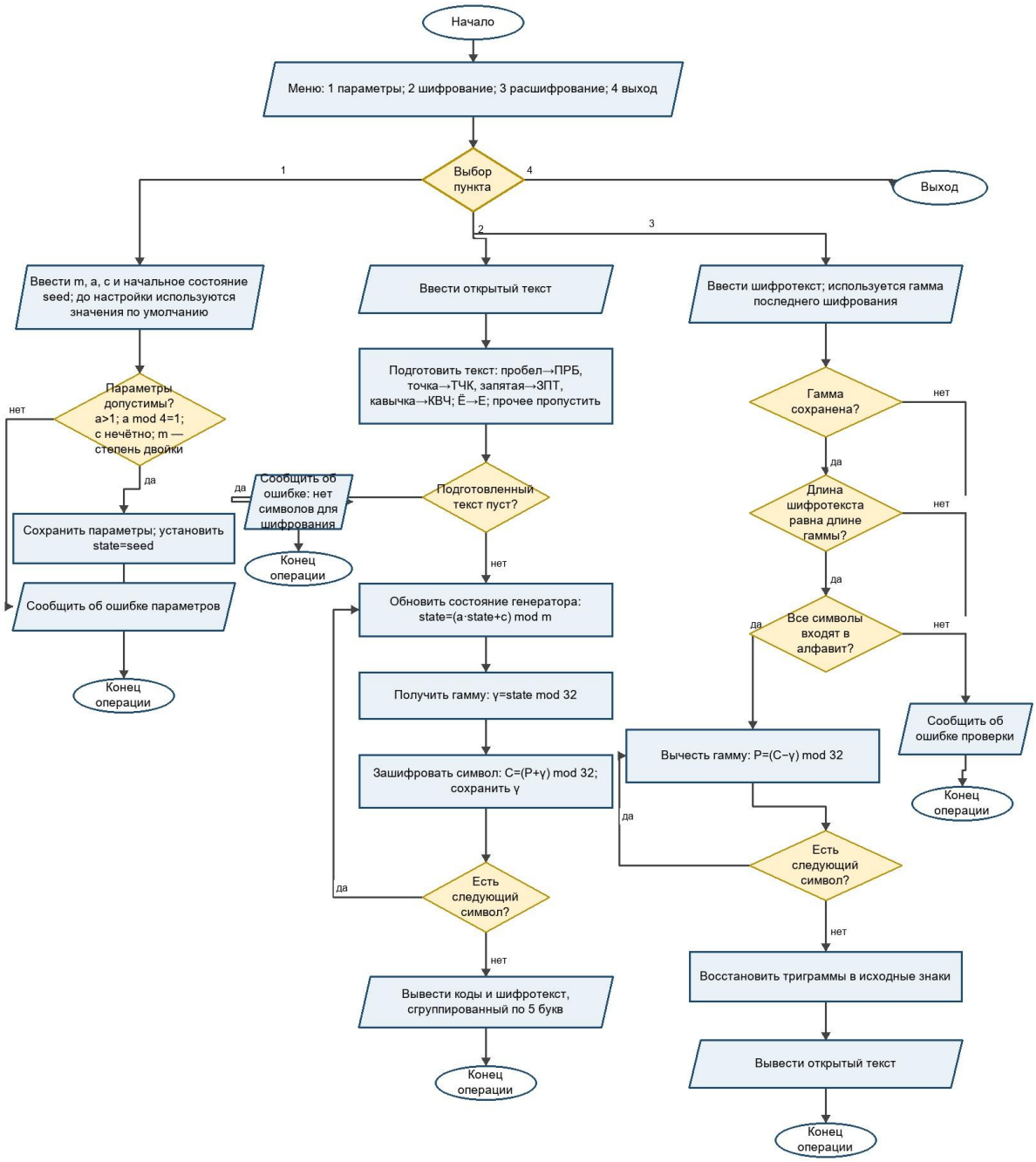


Рисунок 22. Блок-схема блокнота Шеннона.

Код программы:

```
       import sys
class ShannonCipher:
    def __init__(self):
        self.a = 1664525
        self.c = 1013904223
        self.m = 256
        self.state = 12345
        self.alphabet = "АБВГДЕЖЗИЙКЛМНОПРСТУФХЦЧШЩЬЫЬЭЮЯ"
        self.last_gamma = []
    def get_char_index(self, ch):
        try:
            return self.alphabet.index(ch)
        except ValueError:
            return -1
    def is_russian_char(self, ch):
        return ch.upper() in self.alphabet or ch.upper() == 'Ё'
    def prepare_text(self, text):
        result = []
        for ch in text:
            if ch == ' ':
                result.append("ПРБ")
            elif ch == '.':
                result.append("ТЧК")
            elif ch == ',':
                result.append("ЗПТ")
            elif ch == '"':
                result.append("КВЧ")
            else:
                upper_ch = ch.upper()
                if upper_ch == 'Ё':
                    upper_ch = 'Е'
                if upper_ch in self.alphabet:
                    result.append(upper_ch)
        return ''.join(result)
    def restore_text(self, text):
        result = []
        i = 0
        length = len(text)
        while i < length:
            if i + 2 < length:
                triple = text[i:i+3]
                if triple == "ПРБ":
                    result.append(' ')
                    i += 3
                    continue
                elif triple == "ТЧК":
                    result.append('.')
                    i += 3
                    continue
                elif triple == "ЗПТ":
                    result.append(',')
                    i += 3
                    continue
                elif triple == "КВЧ":
                    result.append('"')
                    i += 3
                    continue
            result.append(text[i])
            i += 1
        return ''.join(result)
    def lcg(self):
        self.state = (self.a * self.state + self.c) % self.m
        return self.state
    def set_default_parameters(self):
        self.a = 1664525
        self.c = 1013904223
        self.m = 256
        self.state = 12345
        print("Установлены параметры по умолчанию:")
        print(f"a = {self.a}, c = {self.c}, m = {self.m}, seed = {self.state}")
    def set_parameters(self, a, c, m, seed):
        valid = True
        errors = []
        if a <= 1:
```

```
                valid = False
                errors.append("- a должно быть больше 1")
            if a % 4 != 1:
                valid = False
                errors.append("- a mod 4 должно быть равно 1")
            if c % 2 == 0:
                valid = False
                errors.append("- c должно быть нечетным")
            if (m & (m - 1)) != 0:
                valid = False
                errors.append("- m должно быть степенью двойки")

            if not valid:
                print("\nОШИБКА: Параметры не соответствуют требованиям:")
                for err in errors:
                    print(err)
                return False

            self.a = a
            self.c = c
            self.m = m
            self.state = seed
            print("\nПараметры приняты и обеспечивают максимальный период")
            print(f"a = {self.a}, c = {self.c}, m = {self.m}, T(0) = {self.state}")
            return True

        def encrypt(self, text):
            prepared = self.prepare_text(text)
            if not prepared:
                print("Ошибка: текст пуст!")
                return

            self.last_gamma.clear()
            numbers = []
            letters = []

            for ch in prepared:
                idx = self.get_char_index(ch)
                if idx == -1:
                    print(f"Ошибка: символ {ch} не найден в алфавите!")
                    return

                gamma = self.lcg() % 32
                self.last_gamma.append(gamma)

                enc_idx = (idx + gamma) % 32
                enc_ch = self.alphabet[enc_idx]

                numbers.append(enc_idx)
                letters.append(enc_ch)

            raw_cipher = ''.join(letters)

            # Группировка по 5 символов
            grouped = ' '.join(raw_cipher[i:i+5] for i in range(0, len(raw_cipher), 5))

            print("\nЗашифрованный текст: ")
            print("В виде цифр:", ' '.join(str(x) for x in numbers))
            print(f"В виде букв: {grouped}")

        def decrypt(self, encrypted_text, gamma):
            cleaned_text = ''.join(encrypted_text.split())

            if len(cleaned_text) != len(gamma):
                print(f"Ошибка: длина текста ({len(cleaned_text)}) не совпадает с длиной гаммы
({len(gamma)})!")
                return

            temp_chars = []
            for i, ch in enumerate(cleaned_text):
                idx = self.get_char_index(ch)
                if idx == -1:
                    print(f"Ошибка: символ {ch} не найден в алфавите!")
                    return
                dec_idx = (idx - gamma[i] + 32) % 32
                temp_chars.append(self.alphabet[dec_idx])

            temp = ''.join(temp_chars)
            result = self.restore_text(temp)
            print("\nРасшифрованный текст:")
            print(result)

        def get_last_gamma(self):
            return self.last_gamma

        def print_alphabet(self):
            print("\nАлфавит (32 буквы, индексы 0-31):")
            for i, ch in enumerate(self.alphabet):
                print(f"{i}:{ch}", end=' ')
                if (i + 1) % 8 == 0:
                    print()
            print()
```

```
def process_user_input(cipher):
    print("\n1. Задать параметры генератора")
    print("2. Зашифровать текст")
    print("3. Расшифровать текст")
    print("4. Выход")
    choice = input("Ваш выбор: ").strip()
    if choice == '1':
        try:
            m = int(input("Введите модуль m (степень двойки): "))
            a = int(input("Введите множитель a (a > 1, a mod 4 = 1): "))
            c = int(input("Введите приращение c (нечетное): "))
            seed = int(input("Введите начальное значение T(0): "))
            cipher.set_parameters(a, c, m, seed)
        except ValueError:
            print("Ошибка: введите целые числа.")
    elif choice == '2':
        text = input("Введите текст для шифрования: ")
        cipher.encrypt(text)
    elif choice == '3':
        encrypted = input("Введите зашифрованный текст: ")
        gamma = cipher.get_last_gamma()
        if not gamma:
            print("Ошибка: сначала нужно зашифровать текст!")
        else:
            cipher.decrypt(encrypted, gamma)
    elif choice == '4':
        print("Программа завершена.")
        sys.exit(0)
    else:
        print("Неверный выбор!")

def main():
    print("Одноразовый блокнот Шеннона")
    cipher = ShannonCipher()
    while True:
        process_user_input(cipher)

if __name__ == "__main__":
    main()
```

Результат работы программы:

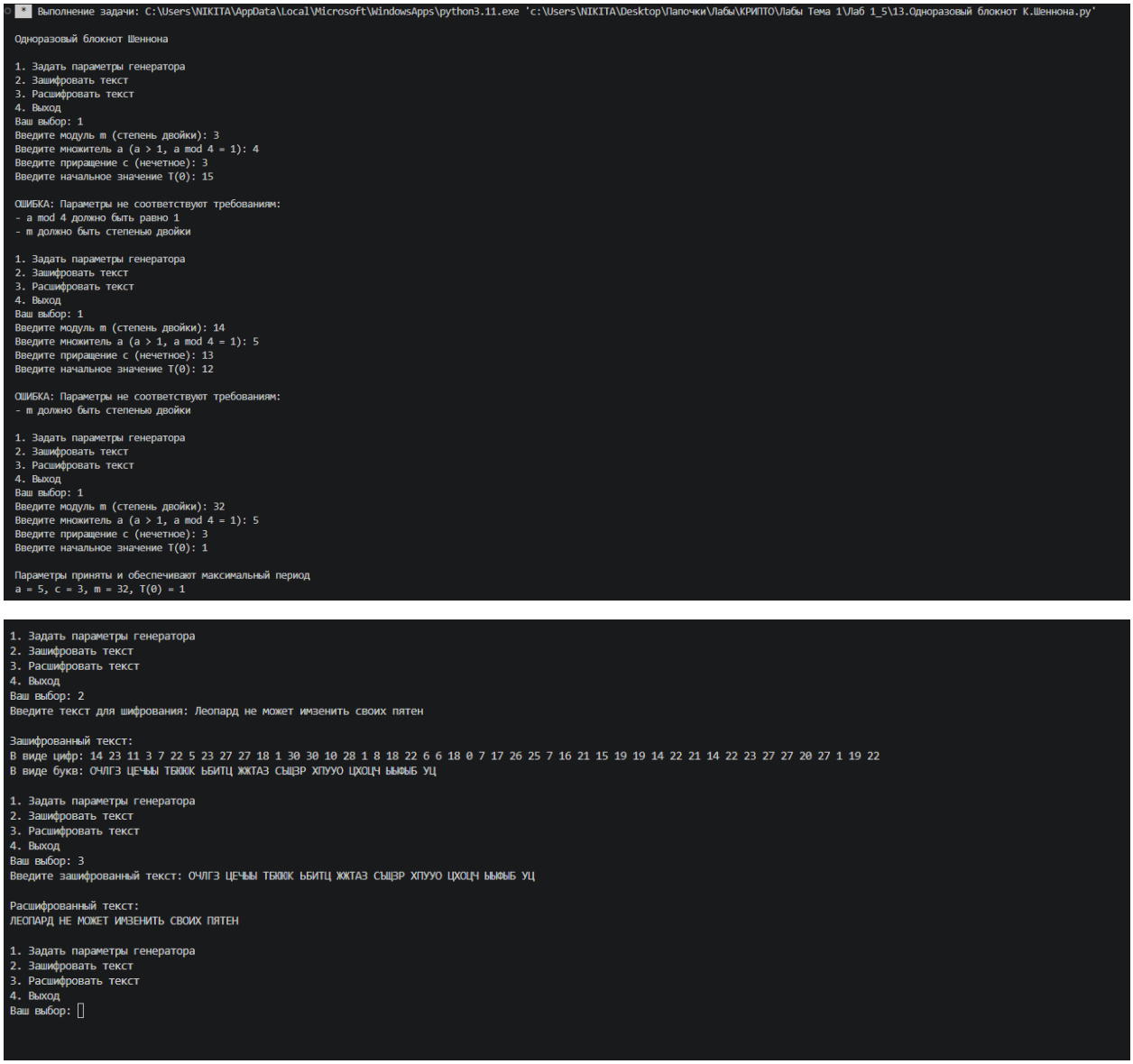
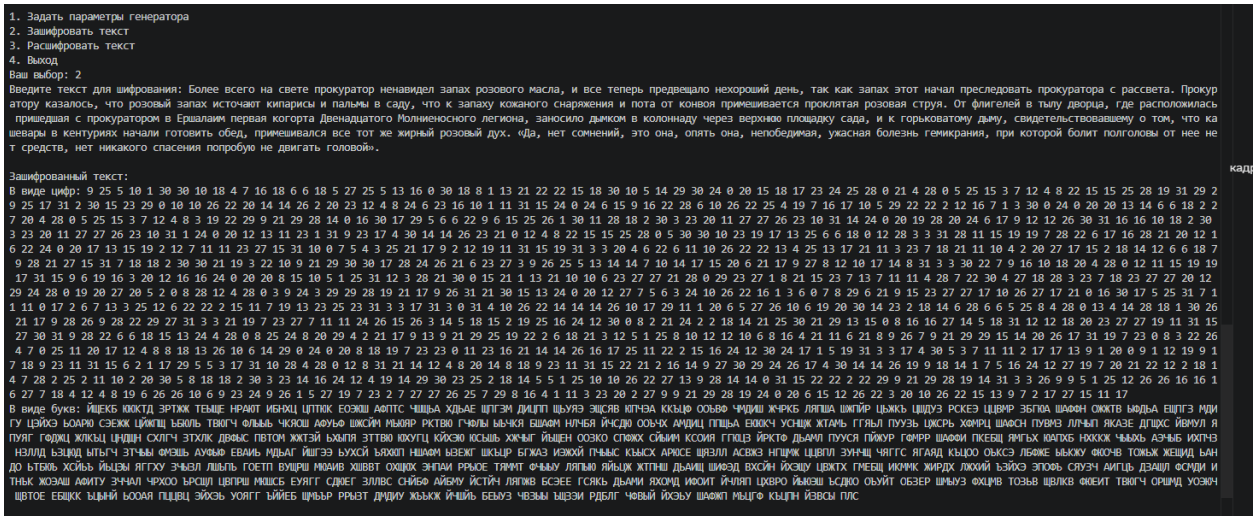


Рисунок 23. Результат выполнения блокнота Шеннона.

Результат теста на 1000 знаков:



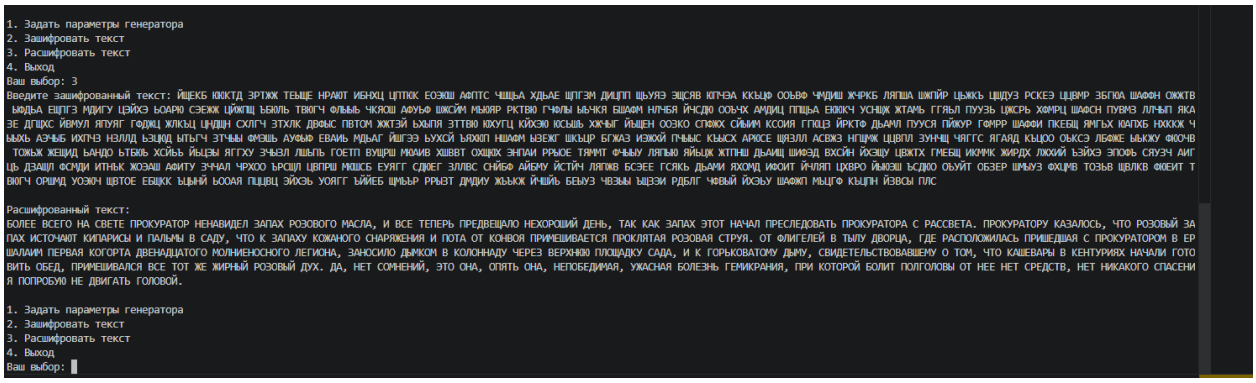


Рисунок 25. Тест на 1000 знаков блокнота Шеннона.

Блок F: Поточные шифры

15. Поточный шифр A5/1

A5/1 — это поточный шифр, используемый в стандарте GSM (Global System for Mobile Communications) для шифрования голосовых вызовов и передачи данных между мобильным телефоном и базовой станцией. Он генерирует гамму (псевдослучайную последовательность битов) с помощью трёх линейных регистров сдвига с обратной связью (LFSR - Linear Feedback Shift Register), которые тактируются неравномерно на основе мажоритарной функции.

Ключевая особенность: шифртекстом является результат сложения гаммы с исходным текстом по модулю 2 (XOR). Генератор гаммы (рисунок 72) в алгоритме A5/1 обеспечивает криптостойкость за счёт неравномерного тактирования регистров.

Блок-схема:

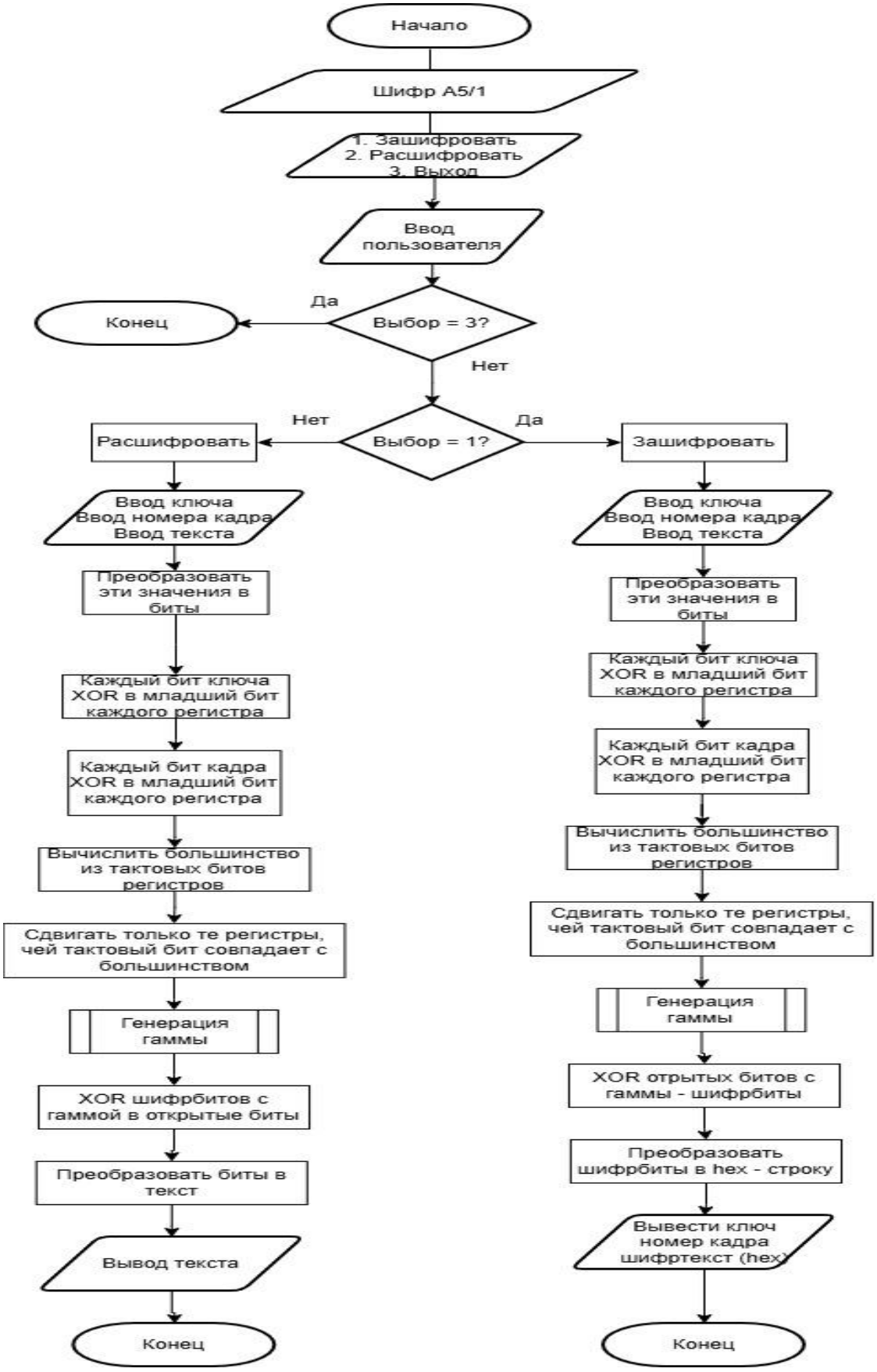

Начало
Шифр A5/1
1. Зашифровать
2. Расшифровать
3. Выход
Ввод пользователя
Да
Конец
Выбор = 3?
Нет
Нет
Расшифровать
Выбор = 1?
Да
Зашифровать
Ввод ключа
Ввод номера кадра
Ввод текста
Преобразовать эти значения в биты
Каждый бит ключа XOR в младший бит каждого регистра
Каждый бит кадра XOR в младший бит каждого регистра
Вычислить большинство из тактовых битов регистров
Сдвигать только те регистры, чей тактовый бит совпадает с большинством
Генерация гаммы
XOR шифрбитов с гаммой в открытые биты
Преобразовать биты в текст
Вывод текста
Конец
Ввод ключа
Ввод номера кадра
Ввод текста
Преобразовать эти значения в биты
Каждый бит ключа XOR в младший бит каждого регистра
Каждый бит кадра XOR в младший бит каждого регистра
Вычислить большинство из тактовых битов регистров
Сдвигать только те регистры, чей тактовый бит совпадает с большинством
Генерация гаммы
XOR отрытых битов с гаммы - шифрбиты
Преобразовать шифрбиты в hex - строку
Вывести ключ номер кадра шифртекст (hex)
Конец

Рисунок 26. Блок-схема шифра A5/1.



Рисунок 27. Генерация гаммы для A5/1.

Код программы:

```
import random

R1_LEN = 19
R2_LEN = 22
R3_LEN = 23

R1_FEEDBACK = [18, 17, 16, 13]
R2_FEEDBACK = [21, 20]
R3_FEEDBACK = [22, 21, 20, 7]

R1_CLOCK = 8
R2_CLOCK = 10
R3_CLOCK = 10

def majority(b1, b2, b3):
    return (b1 & b2) | (b1 & b3) | (b2 & b3)

def clock_register(reg, length, feedback_bits):
    out_bit = reg[length - 1]
    feedback = 0
    for idx in feedback_bits:
        feedback ^= reg[idx]
    for i in range(length - 1, 0, -1):
        reg[i] = reg[i - 1]
    reg[0] = feedback
    return out_bit

def string_to_64bit_key(text):
    """Преобразует текст в 64 бита (первые 8 байт UTF-8, дополняя нулями)."""
    data = text.encode('utf-8')[:8]
    if len(data) < 8:
        data += b'\x00' * (8 - len(data))
    bits = []
    for byte in data:
        for i in range(7, -1, -1):
            bits.append((byte >> i) & 1)
    return bits

def int_to_frame(num):
    """
    Преобразует целое число от 0 до 255 в список из 22 битов:
    """
    if not (0 <= num <= 255):
        raise ValueError("Номер кадра должен быть в диапазоне 0 … 255")
    # 8 бит числа
    low_bits = []
    for i in range(7, -1, -1):
        low_bits.append((num >> i) & 1)
    # 14 нулевых старших битов + 8 бит числа = 22 бита
    frame_bits = [0] * 14 + low_bits
    return frame_bits

def a5_1_keystream(key_bits, frame_bits, keystream_length):
    """Генерация гаммы A5/1."""
    r1 = [0] * R1_LEN
    r2 = [0] * R2_LEN
    r3 = [0] * R3_LEN

    # Загрузка 64-битного ключа
    for i in range(64):
        bit = key_bits[i]
        r1[0] ^= bit
        r2[0] ^= bit
        r3[0] ^= bit
        clock_register(r1, R1_LEN, R1_FEEDBACK)
        clock_register(r2, R2_LEN, R2_FEEDBACK)
        clock_register(r3, R3_LEN, R3_FEEDBACK)

    # Загрузка 22-битного номера кадра
    for i in range(22):
        bit = frame_bits[i]
        r1[0] ^= bit
        r2[0] ^= bit
        r3[0] ^= bit
        clock_register(r1, R1_LEN, R1_FEEDBACK)
        clock_register(r2, R2_LEN, R2_FEEDBACK)
        clock_register(r3, R3_LEN, R3_FEEDBACK)

    # Холостые такты
    for _ in range(100):
        maj = majority(r1[R1_CLOCK], r2[R2_CLOCK], r3[R3_CLOCK])
        if r1[R1_CLOCK] == maj:
            clock_register(r1, R1_LEN, R1_FEEDBACK)
        if r2[R2_CLOCK] == maj:
```

```
            clock_register(r2, R2_LEN, R2_FEEDBACK)
        if r3[R3_CLOCK] == maj:
            clock_register(r3, R3_LEN, R3_FEEDBACK)
    # Генерация гаммы
    keystream = []
    for _ in range(keystream_length):
        maj = majority(r1[R1_CLOCK], r2[R2_CLOCK], r3[R3_CLOCK])
        out_bit = r1[R1_LEN - 1] ^ r2[R2_LEN - 1] ^ r3[R3_LEN - 1]
        keystream.append(out_bit)
        if r1[R1_CLOCK] == maj:
            clock_register(r1, R1_LEN, R1_FEEDBACK)
        if r2[R2_CLOCK] == maj:
            clock_register(r2, R2_LEN, R2_FEEDBACK)
        if r3[R3_CLOCK] == maj:
            clock_register(r3, R3_LEN, R3_FEEDBACK)
    return keystream
def text_to_bits(text):
    bits = []
    for byte in text.encode('utf-8'):
        for i in range(7, -1, -1):
            bits.append((byte >> i) & 1)
    return bits
def bits_to_text(bits):
    bytes_list = []
    for i in range(0, len(bits), 8):
        if i + 7 < len(bits):
            byte = 0
            for j in range(8):
                byte |= (bits[i + j] << (7 - j))
            bytes_list.append(byte)
    return bytes(bytes_list).decode('utf-8', errors='ignore')
def bits_to_hex(bits):
    while len(bits) % 4 != 0:
        bits.append(0)
    hex_str = []
    for i in range(0, len(bits), 4):
        val = (bits[i] << 3) | (bits[i+1] << 2) | (bits[i+2] << 1) | bits[i+3]
        hex_str.append(f"{val:X}")
    return ''.join(hex_str)
def hex_to_bits(hex_str):
    bits = []
    for ch in hex_str:
        val = int(ch, 16)
        bits.append((val >> 3) & 1)
        bits.append((val >> 2) & 1)
        bits.append((val >> 1) & 1)
        bits.append(val & 1)
    return bits
def encrypt():
    print("\nШИФРОВАНИЕ")
    key_text = input("Введите ключ (текст): ")
    try:
        frame_num = int(input("Введите номер кадра (целое число от 0 до 255): "))
    except ValueError:
        print("Ошибка: необходимо ввести целое число")
        return
    plaintext = input("Введите текст для шифрования: ")
    if not plaintext:
        print("Текст не введён")
        return
    key_bits = string_to_64bit_key(key_text)
    try:
        frame_bits = int_to_frame(frame_num)
    except ValueError as e:
        print(e)
        return
    plain_bits = text_to_bits(plaintext)
    keystream = a5_1_keystream(key_bits, frame_bits, len(plain_bits))
    cipher_bits = [plain_bits[i] ^ keystream[i] for i in range(len(plain_bits))]
    print("\nРЕЗУЛЬТАТ")
    print(f"Ключ (hex): {bits_to_hex(key_bits)}")
    print(f"Номер кадра: {frame_num}")
    print(f"Зашифрованный текст (hex): {bits_to_hex(cipher_bits)}")
def decrypt():
    print("\nРАСШИФРОВАНИЕ")
    key_text = input("Введите ключ (текст): ")
```

```
            try:
                frame_num = int(input("Введите номер кадра (целое число от 0 до 255): "))
            except ValueError:
                print("Ошибка: необходимо ввести целое число")
                return
            cipher_hex = input("Введите зашифрованный текст в hex: ").strip()
            if not cipher_hex:
                print("Текст не введён")
                return
            key_bits = string_to_64bit_key(key_text)
            try:
                frame_bits = int_to_frame(frame_num)
            except ValueError as e:
                print(e)
                return
            cipher_bits = hex_to_bits(cipher_hex)
            keystream = a5_1_keystream(key_bits, frame_bits, len(cipher_bits))
            plain_bits = [cipher_bits[i] ^ keystream[i] for i in range(len(cipher_bits))]
            plaintext = bits_to_text(plain_bits)
            print("\nРЕЗУЛЬТАТ")
            print(f"Ключ (hex): {bits_to_hex(key_bits)}")
            print(f"Номер кадра: {frame_num}")
            print(f"Расшифрованный текст: Более всего на свете прокуратор ненавидел запах розового масла,
и все теперь предвещало нехороший день, так как запах этот начал преследовать прокуратора с рассвета.
Прокуратору казалось, что розовый запах источают кипарисы и пальмы в саду, что к запаху кожаного снаряжения
и пота от конвоя примешивается проклятая розовая струя. От флигелей в тылу дворца, где расположилась
пришедшая с прокуратором в Ершалаим первая когорта Двенадцатого Молниеносного легиона, заносило дымком в
колоннаду через верхнюю площадку сада, и к горьковатому дыму, свидетельствовавшему о том, что кашевары в
кентуриях начали готовить обед, примешивался все тот же жирный розовый дух. «Да, нет сомнений, это она,
опять она, непобедимая, ужасная болезнь гемикрания, при которой болит полголовы от нее нет средств, нет
никакого спасения попробую не двигать головой».")
        def main():
            while True:
                print("\nПОТОЧНЫЙ ШИФР A5/1")
                print("1. Зашифровать текст")
                print("2. Расшифровать текст")
                print("3. Выход")
                choice = input("Выберите действие: ")
                if choice == '1':
                    encrypt()
                elif choice == '2':
                    decrypt()
                elif choice == '3':
                    print("Программа завершена.")
                    break
                else:
                    print("Неверный выбор, попробуйте снова.")
        if __name__ == "__main__":
            main()
```

Результат работы программы:

```
Выполнение задачи: C:\Users\NIKITA\AppData\Local\Microsoft\WindowsApps\python3.11.exe 'c:\Users\NIKITA\Desktop\Папочки\Лабы\КРИПТО\Лабы Тема 1\Лаб 1_6\15.A5_1.py'
ШИФРОВАНИЕ
Введите ключ (текст): АВГУСТ
Введите номер кадра (целое число от 0 до 255): 155
Введите текст для шифрования: Леопард не может зименить своих пятен

РЕЗУЛЬТАТ
Ключ (hex): D090D092D093D0A3
Номер кадра: 155
Зашифрованный текст (hex): 011FF38D15D7BDCCF3C2E90BAD973F37FD945E9DDA3E76763232A577CFF10D61E9400B5D16EC1184077EEFE8C606D44EE1365A43F43C9D384B15F173C741ABFD57ED0B8FE7

ПОТОЧНЫЙ ШИФР A5/1
1. Зашифровать текст
2. Расшифровать текст
3. Выход
Выберите действие: 2

РАСШИФРОВАНИЕ
Введите ключ (текст): АВГУСТ
Введите номер кадра (целое число от 0 до 255): 155
Введите зашифрованный текст в hex: 011FF38D15D7BDCCF3C2E90BAD973F37FD945E9DDA3E76763232A577CFF10D61E9400B5D16EC1184077EEFE8C606D44EE1365A43F43C9D384B15F173C741ABFD57ED0B8FE7

РЕЗУЛЬТАТ
Ключ (hex): D090D092D093D0A3
Номер кадра: 155
Расшифрованный текст: Леопард не может изменить своих пятен

ПОТОЧНЫЙ ШИФР A5/1
1. Зашифровать текст
2. Расшифровать текст
3. Выход
Выберите действие:
```

Рисунок 28. Результат выполнения шифра A5/1.

Контрольный пример:

Таблица 1. Контрольный пример A5/1.

Ключ (hex):	1101011100001101	сасбабсс												
Ключ (bin):	1111000111100000111100011110000011110000111100001111100001111													
Номер кадра:	0000000000000000000													
Текст: кра	1110101011110000111100000													
Такт	Этап	R1 (19 бит)	R2 (22 бит)	R1 (23 бит)	Бит входа	S1 (бит 8 R	S2 (бит 10	S3 (бит 10	F	Сдвиг R1	Сдвиг R2	Сдвиг R3	Выходной	Гамма
0	Начало	0000000000000000000	0000000000000000000000	00000000000000000000000										
1	загрузка ключа	1000000000000000000	1000000000000000000000	10000000000000000000000	1	0	0	0	0	1	1	1	0	
2	загрузка ключа	1100000000000000000	1100000000000000000000	11000000000000000000000	1	0	0	0	0	1	1	1	0	
3	загрузка ключа	1110000000000000000	1110000000000000000000	11100000000000000000000	1	0	0	0	0	1	1	1	0	
4	загрузка ключа	1111000000000000000	1111000000000000000000	11110000000000000000000	1	0	0	0	0	1	1	1	0	
5	загрузка ключа	0111100000000000000	0111100000000000000000	01111000000000000000000	0	0	0	0	0	1	1	1	0	
6	загрузка ключа	0011110000000000000	0011110000000000000000	00111100000000000000000	0	0	0	0	0	1	1	1	0	
7	загрузка ключа	0001111000000000000	0001111000000000000000	00011110000000000000000	0	0	0	0	0	1	1	1	0	
8	загрузка ключа	1000111100000000000	1000111100000000000000	10001111000000000000000	1	0	0	0	0	1	1	1	0	
9	загрузка ключа	1100011110000000000	1100011110000000000000	11000111100000000000000	1	0	0	0	0	1	1	1	0	
10	загрузка ключа	1110001111000000000	1110001111000000000000	11100011110000000000000	1	1	0	0	0	0	1	1	0	
11	загрузка ключа	1111000111100000000	1111000111100000000000	11110001111000000000000	1	1	0	0	0	0	1	1	0	
12	загрузка ключа	0111100011110000000	0111100011110000000000	01111000111100000000000	0	1	1	1	1	1	1	1	0	
13	загрузка ключа	0011110001111000000	0011110001111000000000	00111100011110000000000	0	1	1	1	1	1	1	1	0	
14	загрузка ключа	0001111000111100000	0001111000111100000000	00011110001111000000000	0	0	1	1	1	0	1	1	0	
15	загрузка ключа	1000111100011110000	1000111100011110000000	10001111000111100000000	0	0	1	1	1	0	1	1	0	
16	загрузка ключа	1100011110001111000	1100011110001111000000	11000111100011110000000	0	0	0	0	0	1	1	1	0	
17	загрузка ключа	0110001111000111100	0110001111000111100000	01100011110001111000000	1	1	0	0	0	0	1	1	0	
18	загрузка ключа	1011000111100011110	1011000111100011110000	10110001111000111100000	1	1	0	0	0	0	1	1	0	
19	загрузка ключа	1101100011110001111	1101100011110001111000	11011000111100011110000	1	1	1	1	1	1	1	1	0	
20	загрузка ключа	0110110001111000111	0110110001111000111100	01101100011110001111000	1	1	1	0	1	1	1	0	1	
21	загрузка ключа	1011011000111100011	1011011000111100011110	10110110001111000111100	0	0	1	0	0	1	0	1	1	
22	загрузка ключа	1101101100011110001	1101101100011110001111	11011011000111100011110	0	0	1	0	0	1	0	1	1	
23	загрузка ключа	0110110110001111000	0110111100001111000111	01101111000011110001111	0	0	0	1	0	1	1	0	0	
24	загрузка ключа	0011011011000111100	0110111100000111100011	01101111000011110001111	1	1	0	0	0	0	1	1	0	
25	загрузка ключа	1001101101100011110	1101011110000011110001	11010011110001000100011	1	1	0	0	0	0	1	1	0	
26	загрузка ключа	1100110110110001111	0110101111000001111000	01101001111000100010001	1	0	0	0	0	1	1	1	0	
27	загрузка ключа	0110011011011000111	1011010111100000111100	10110100111100010001000	1	1	0	1	1	1	0	1	0	
28	загрузка ключа	1011001101101100011	0101101011110000011110	01011010011110001000100	0	1	1	1	1	1	1	1	1	
29	загрузка ключа	1101100110110110001	1010110101111000001111	10101101001111000100010	0	0	1	1	1	0	1	1	1	
30	загрузка ключа	0110110011011011000	0101011010111100000111	01010110100111100010001	0	1	1	1	1	1	1	1	0	
31	загрузка ключа	0011011001101101100	0010101101011110000011	10101011010011110001000	0	1	1	0	1	1	1	0	0	
32	загрузка ключа	1001101100110110110	1001010110101111000001	01010101101001111000100	1	0	0	0	0	1	1	1	1	
33	загрузка ключа	0100110110011011011	0100101011010111100000	10101010110100111100010	1	0	1	1	1	0	1	1	1	
34	загрузка ключа	1010011011001101101	1010010101101011110000	01010101011010011110001	1	1	0	0	0	0	1	1	1	
35	загрузка ключа	0101001101100110110	1101001010110101111000	10101010110100111100010	1	1	1	1	1	1	1	1	0	
36	загрузка ключа	1010100110110011011	0110100101011010111100	01010101010110100111100	0	0	1	1	1	0	1	1	0	
37	загрузка ключа	0101010011011001101	0011010010101101011110	00101010101011010011110	0	1	0	0	0	0	1	1	1	
38	загрузка ключа	0010101000110100110	1001101001010110101111	00010101010101101001111	0	1	1	1	1	1	1	1	1	
39	загрузка ключа	1001010100110110011	0100110100101011010111	00001010101010110100111	0	0	0	0	0	1	1	1	0	
40	загрузка ключа	1100101010011011001	0010011010010101101011	10000101010101011010011	0	0	1	1	1	0	1	1	1	
41	загрузка ключа	0110010101001101100	1001001101001010110101	01000010101010101101001	1	1	0	0	0	0	1	1	1	
42	загрузка ключа	1011001010100110110	0100100110100101011010	00100001010101010110100	1	0	0	1	0	1	1	0	0	
43	загрузка ключа	0101100101010011011	0010010011010010101101	10010000101010101011010	1	1	1	0	1	1	1	0	0	
44	загрузка ключа	0010110010101001101	1001001001101001010110	11001000010101010101101	0	0	0	1	0	1	1	0	0	
45	загрузка ключа	0001011001010100110	1100100100110100101011	01100100001010101010110	0	1	1	0	1	1	1	0	0	
46	загрузка ключа	1000101100101010011	0110010010011010010101	00110010000101010101011	0	0	1	1	1	0	1	1	1	
47	загрузка ключа	0100010110010101001	0011001001001101001010	00011001000010101010101	0	0	0	0	0	1	1	1	1	
48	загрузка ключа	1010001011001010100	0011100100100110100101	00001100100001010101010	1	1	0	0	0	0	1	1	0	
49	загрузка ключа	0101000101100101010	0010110010010011010010	00000110010000101010101	1	1	1	0	1	1	1	0	1	
50	загрузка ключа	1010100010110010101	0001011001001001101001	10000011001000010101010	1	0	0	0	0	1	1	1	1	
51	загрузка ключа	1101010001011001010	0000101100100100110100	11000001100100001010101	1	1	0	1	1	1	0	1	0	
52	загрузка ключа	0110101000101100101	1000010110010010011010	01100000110010000101010	1	0	1	0	0	1	0	1	1	
53	загрузка ключа	1011010100001011001	1100001011001001001101	10110000011001000010101	0	0	0	0	0	1	1	1	1	
54	загрузка ключа	0101101010001011001	1110000101100100100110	01011000001100100001010	0	0	0	1	0	1	1	0	0	
55	загрузка ключа	1010110100001011100	1111000010110010010011	10101100000110010000101	0	1	1	1	1	1	1	1	1	
56	загрузка ключа	1101011010100010110	1111100001011001001001	11010110000011001000010	1	0	1	0	0	1	0	1	0	
57	загрузка ключа	1110101101010001011	0111110000101100100100	01101011000001100100001	1	1	0	0	0	0	1	1	1	
58	загрузка ключа	1111010110101000101	1011111000010110010010	10110101100000110010000	1	0	1	0	0	1	0	1	0	
59	загрузка ключа	1111101011010100010	0101111100001011001001	01011010110000011001000	1	1	0	0	0	0	1	1	1	
60	загрузка ключа	1111110101101010001	0010111110000101100100	10101101011000001100100	1	1	0	0	0	0	1	1	1	

61	загрузка ключа	1111111010110101000	0001011111000010110010	0101011010110000011010	0	0	0	1	0	1	1	0	1
62	загрузка ключа	1111111101011010100	1000101111100001011001	1010101101011000001001	0	1	0	1	1	1	0	1	0
63	загрузка ключа	1111111110101101010	1100001011110000101100	0101010111010110000001100	0	0	1	0	0	1	0	1	0
64	загрузка ключа	1111111111010110101	1110000101111000010110	1010101011010110000010110	1	1	1	1	1	1	1	1	0
1	Загрузка кадра	1111111111101011010	1111000101111100001011	0101010101101010000011	0	1	1	0	1	1	1	0	1
2	Загрузка кадра	1111111111110101101	0111100010111110000101	1010101010110101000001	0	1	1	1	1	1	1	1	0
3	Загрузка кадра	1111111111111010110	1011110001011111000010	1101010101011010100000	0	1	1	1	1	1	1	1	1
4	Загрузка кадра	0111111111111101011	1101111000101111100001	1110101010101101010000	0	1	0	0	0	0	1	1	0
5	Загрузка кадра	1011111111111110101	1110111100010111110000	0111010101010110101000	0	1	1	1	1	1	1	1	0
6	Загрузка кадра	1101111111111111010	0111011110001011111000	1011101010101011010100	0	1	0	0	0	0	1	1	1
7	Загрузка кадра	0110111111111111101	0011101111000101111100	1101110101010110101010	0	1	0	1	1	1	0	1	0
8	Загрузка кадра	1011011111111111110	0001110111100010111110	1110111010101010110101	0	1	0	0	0	0	1	1	1
9	Загрузка кадра	1101101111111111111	1000111011110001011111	0111011101010101010101	0	1	1	1	1	1	1	1	1
10	Загрузка кадра	0110110111111111111	0100011101111000101111	1011101110101010110101	0	1	1	0	1	1	1	0	1
11	Загрузка кадра	0011011011111111111	0010001110111100010111	0101110111010101010101	0	1	1	1	1	1	1	1	0
12	Загрузка кадра	0001101101111111111	0001000111011110001011	1010111011101010101010	0	1	1	0	1	1	1	0	1
13	Загрузка кадра	0000110110111111111	0000100011101111000101	0101011101110101010111	0	0	0	1	0	1	1	0	0
14	Загрузка кадра	0000011011011111111	1000010001110111100010	1010101110111010101010	0	1	1	1	1	1	1	1	1
15	Загрузка кадра	0000001101101111111	1100001000111011110001	1101010111011101010101	0	1	1	1	1	1	1	1	0
16	Загрузка кадра	0000000110110111111	1110000100011101111000	0110101011101110101010	0	0	1	0	0	1	0	1	0
17	Загрузка кадра	0000000011011011111	0111000010001110111100	0011010101110111010101	0	1	0	1	1	1	0	1	0
18	Загрузка кадра	1000000001101101111	0011100001000111011110	0001101010111011101010	0	1	0	1	1	1	0	1	1
19	Загрузка кадра	0100000000110110111	1001110000100011101111	0000110101011101101010	0	0	0	1	0	1	1	0	0
20	Загрузка кадра	0010000000011011011	0100111000010001110111	0000011010101110111010	0	0	1	0	0	1	0	1	0
21	Загрузка кадра	0001000000001101101	0010011100001000111011	0000001101010111011110	0	0	0	1	0	1	1	0	1
22	Загрузка кадра	1000100000000110110	0001001110000100011101	0000000110101011101101	0	0	0	0	0	1	1	1	0
1	Холостой ход	1100010000000011011	1000100111000010001110	0000000110101011011101	0	0	0	1	0	1	1	0	0
2	Холостой ход	0110001000000001101	1100010011100001000111	0000000110101011011101	0	0	0	1	0	1	1	0	0
3	Холостой ход	0110001000000001101	0110001001110000100011	1000000011010110110110	0	0	1	1	1	0	1	1	1
4	Холостой ход	0011000100000000110	0110001001110000100011	0100000001101010111011	0	0	1	0	0	1	0	1	0
5	Холостой ход	0011000100000000110	0011000100111000010001	1010000000110101011011	0	0	1	1	1	0	1	1	0
6	Холостой ход	0011000100000000110	1001100010011100001000	0101000000011010111101	0	0	1	1	1	0	1	1	0
7	Холостой ход	0001100010000000011	0100110001001110000100	0010100000001101010110	0	0	0	0	0	1	1	1	1

7	Холостой ход	0001100010000000011	0100110001001110000100	0010100000001101010110	0	0	0	0	0	1	1	1	1
8	Холостой ход	0001100010000000011	0010011000100111000010	0001010000000110101011	0	1	0	0	0	0	1	1	1
9	Холостой ход	0000110001000000001	1001001100010011100001	0001010000000110101011	0	1	1	0	1	1	1	0	0
10	Холостой ход	1000011000100000000	1100100110001001110000	1000101000000011010101	0	0	0	0	0	1	1	1	1
11	Холостой ход	0100001100010000000	0110010011000100111000	0100001010000000110101	0	0	0	0	0	1	1	1	1
12	Холостой ход	0010000110001000000	0011001001100010011100	1010000101000000011010	0	0	0	0	0	1	1	1	1
13	Холостой ход	0001000011000100000	0001100100110001001110	1010000101000000011010	0	1	1	0	1	1	1	0	0
14	Холостой ход	1000100001100010000	1000110010011000100111	1010000101000000011010	0	1	1	0	1	1	1	0	0
15	Холостой ход	0100010000110001000	0100011001100100010011	1101000101000000011010	0	0	0	0	0	1	1	1	1
16	Холостой ход	0010001000011000100	0010001100100110001001	1110100010100000011010	0	0	0	0	0	1	1	1	0
17	Холостой ход	0010001000011000100	1001000110010011000100	1111010000101000000011	0	0	1	1	1	0	1	1	1
18	Холостой ход	1001000100001100010	0100100011001001100010	0111101000101000000110	0	0	0	0	0	1	1	1	1
19	Холостой ход	0100100010000110001	1010010000110010110001	0111101000101000000110	0	0	0	1	0	1	1	0	0
20	Холостой ход	0010010000100011000	1101000100011001011000	0011110100010100000011	0	1	1	1	1	1	1	1	0
21	Холостой ход	0001001000100001100	1101001000110010011000	1001111010001010000001	0	0	1	0	0	1	0	1	1
22	Холостой ход	1000100010001000110	1101001000110010011000	1100111101000101000000	0	0	1	0	0	1	0	1	1
23	Холостой ход	0100010010000100011	1101001000110010011000	1110011101000101000000	0	0	1	0	0	1	0	1	0
24	Холостой ход	0010001001000100001	0110100100011001001100	1111001111010001010000	0	1	1	1	1	1	1	1	1
25	Холостой ход	0001000100100010000	0011010010001100100110	1111100111101000101000	0	0	0	0	0	1	1	1	1
26	Холостой ход	0000100010010001000	1001101001000110010011	1111110011110100010100	0	0	0	1	0	1	1	0	0
27	Холостой ход	0000010001001000100	1001101001000110010011	1111111001111010001010	0	1	0	1	1	1	0	1	1
28	Холостой ход	1000001000100100010	0100110100100011001001	1111110011110100010100	0	0	0	1	0	1	1	0	1
29	Холостой ход	1000001000100100010	1010011010010001100100	0111111001111010010100	0	0	1	1	1	0	1	1	1
30	Холостой ход	0100000100010010001	0101001101001000110010	0111111001111010010100	0	0	0	1	0	1	1	0	0
31	Холостой ход	1010000010001001000	1010100110100100011001	0111111001111010010100	0	0	0	1	0	1	1	0	1
32	Холостой ход	0101000001000100100	1101010011010010001100	1011111100111101001010	0	1	1	1	1	1	1	1	1
33	Холостой ход	0010100000100010010	0110101001101001000110	1011111100111101001010	0	0	0	1	0	1	1	0	0
34	Холостой ход	0010100000100010010	1011010100110100100011	0101111110011101000101	0	0	1	1	1	0	1	1	0
35	Холостой ход	1001010000010001001	1011010100110100100011	1010111111001111010010	0	0	1	0	0	1	0	1	0
36	Холостой ход	1100101000001000100	1011010100110100100011	0101011111100111010001	0	0	1	0	0	1	0	1	0
37	Холостой ход	1100101000001000100	0101101010011010010001	0010101111110011101000	0	0	1	1	1	0	1	1	0
38	Холостой ход	1110010100000100010	1010110101001101001000	0010101111110011101000	0	0	0	1	0	1	1	0	1
39	Холостой ход	0111001010000010001	0101011010100110100100	0010101111110011101000	0	0	0	1	0	1	1	0	0

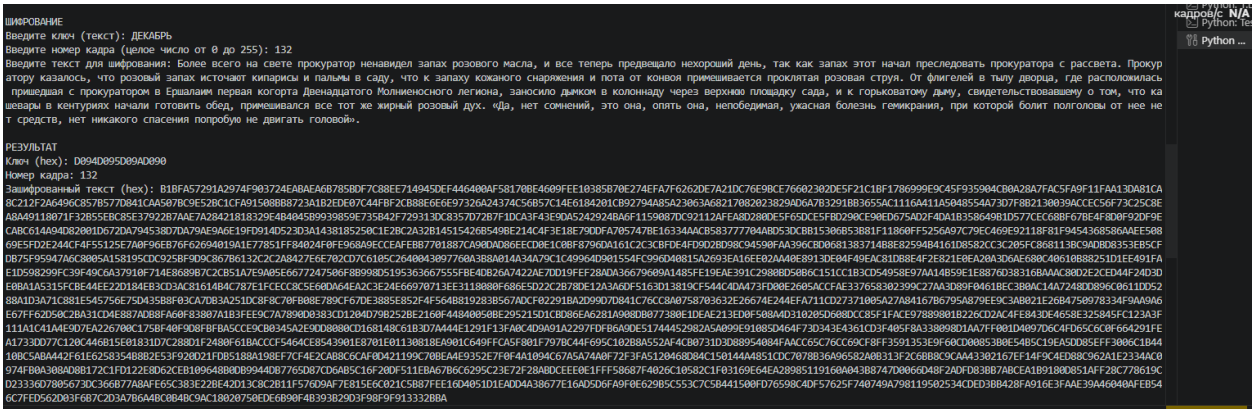
40	Холостой ход	1011100101000001000	0010101101010011010010	1001010111111100111101000	0	1	1	1	1	1	1	1	1
41	Холостой ход	0101110010100000100	1001010110101001101001	1001010111111100111101000	0	0	0	1	0	1	1	0	0
42	Холостой ход	1010111001010000010	1100101011010100110100	0100101011111100111110100	0	1	1	1	1	1	1	1	1
43	Холостой ход	1101011100101000001	0110010101101010011010	0100101011111100111110100	0	0	0	1	0	1	1	0	0
44	Холостой ход	1101011100101000001	1011001010110101001101	1010010101111110011110100	0	0	1	1	1	0	1	1	1
45	Холостой ход	1101011100101000001	1101100101011010100110	1101001010111111100111100	0	0	1	1	1	0	1	1	1
46	Холостой ход	1110101110010100000	1110110010110101010011	1101001010111111100111100	0	0	0	1	0	1	1	0	1
47	Холостой ход	1110101110010100000	0111011001010110101001	0110100101011111100111100	0	1	1	1	1	1	1	1	1
48	Холостой ход	1111010111001010000	1011101100101011010100	0011010010101111110011100	0	1	0	0	0	0	1	1	0
49	Холостой ход	0111110101110010000	0101110110010101101010	1001101001010111110011100	0	1	1	1	1	1	1	1	1
50	Холостой ход	0111101011100101000	1010111011001010110101	0100110100101011111100100	0	1	0	0	0	0	1	1	1
51	Холостой ход	1011110101110010100	1010111011001010110101	0010011010010101111110000	0	1	0	0	1	1	1	0	1
52	Холостой ход	1101111010111001010	1101011101100101011010	1001001101001010111110000	0	0	0	0	0	1	1	1	1
53	Холостой ход	1110111101011100101	1110101110110010101101	1001001101001010111110000	0	1	1	0	1	1	1	0	0
54	Холостой ход	1111011101101110010	1110101110110010101101	1100100110100101011111000	0	0	1	0	0	1	0	1	0
55	Холостой ход	0111101111010111001	1111010111011001010110	0110010011010010111111000	0	1	1	1	1	1	1	1	0
56	Холостой ход	0111101111010111001	1111101011101100101011	1011001001101001010111100	0	1	0	0	0	0	1	1	0
57	Холостой ход	0011110111101011100	0111110101110110010101	1101100100110100101011100	0	1	1	1	1	1	1	1	1
58	Холостой ход	1001111011110101110	1011111010111011001010	0110110010011010010101100	0	1	1	1	1	1	1	1	0
59	Холостой ход	1100111101111010111	1101111101011101100101	0110110010011010010101100	0	1	1	0	1	1	1	0	1
60	Холостой ход	1110011110111101011	1110111110101110110010	0011011001001010010101100	0	0	0	0	0	1	1	1	1
61	Холостой ход	1111001111101110101	1111011110101110011001	0011011001001010010101100	0	1	1	0	1	1	1	0	0
62	Холостой ход	1111001111011110101	1111101111101011101100	0001101100100110100101000	0	1	0	0	0	0	1	1	1
63	Холостой ход	1111100111101111010	0111110111101011110110	0000110110010011010010100	0	1	1	1	1	1	1	1	1
64	Холостой ход	0111110011110111101	1011111011111010111011	0000110110010011010010100	0	1	1	0	1	1	1	0	1
65	Холостой ход	1011111001111011110	0101111101111101011101	0000110110010011010010100	0	1	1	0	1	1	1	0	1
66	Холостой ход	0101111100111101111	0101111101111101011101	1000011011001001101001000	0	0	1	0	0	1	0	1	0
67	Холостой ход	0010111110011110111	0101111101111101011101	1100001101100100110100100	0	0	1	0	0	1	0	1	0
68	Холостой ход	0001011111001111011	1010111110111101011110	0110000110110010011010000	0	1	1	1	1	1	1	1	1
69	Холостой ход	1000101111100111101	1101011111011111010111	0011000011011001001101000	0	1	1	1	1	1	1	1	1
70	Холостой ход	1000101111100111101	0110101111011111101011	1001100001101100100110100	0	1	0	0	0	0	1	1	0
71	Холостой ход	1100001011110011110	0011010111101111110101	0100110000110110010011010	0	1	1	1	1	1	1	1	1

71	Холостой ход	1100001011110011110	0011010111101111110101	0100110000110110010011010	0	1	1	1	1	1	1	1	1
72	Холостой ход	0110001011111001111	1001101011111011111010	0010011000011011001001100	0	1	1	1	1	1	1	1	1
73	Холостой ход	1011000101111100111	1100110101111101111101	0010011000011011001001100	0	1	1	0	1	1	1	0	0
74	Холостой ход	0101100101111100111	1100110101111101111101	0001001100001101100100100	0	0	1	0	0	1	0	1	1
75	Холостой ход	1010110001011111001	1110011010111101111110	0001001100001101100100100	0	1	1	0	1	1	1	0	1
76	Холостой ход	0101011000101111100	1110011010111101111110	0000100110000110110010000	0	0	1	0	0	1	0	1	0
77	Холостой ход	0010101100010111110	1110011010111101111110	0000010011000011011001000	0	0	1	0	0	1	0	1	0
78	Холостой ход	1001010110001011111	1110011010111101111110	1000001001100001101100100	0	0	1	0	0	1	0	1	0
79	Холостой ход	1100101011000101111	1111001101011111011111	1100000100110000110110000	0	1	1	1	1	1	1	1	0
80	Холостой ход	0110010101100010111	1111001101011111011111	0110000100110000110110000	0	1	0	1	1	1	0	1	0
81	Холостой ход	1011001010110001011	0111100110101111101111	0011000001001100001101100	0	0	0	0	0	1	1	1	0
82	Холостой ход	0101100101011000101	0011110011010111101111	0011000001001100001101100	0	1	1	0	1	1	1	0	1
83	Холостой ход	0010110010101100010	0001111001101011111011	0001100000100110000110100	0	0	0	0	0	1	1	1	1
84	Холостой ход	0001011001010110001	0000111001101011111101	0000110000010011000011000	0	1	1	1	1	1	1	1	0
85	Холостой ход	0000101100101011000	0000111001101011111101	0000011000001001100001100	0	0	1	0	0	1	0	1	0
86	Холостой ход	0000010110010101100	0000111001101011111101	0000001100000100110000100	0	0	1	0	0	1	0	1	0
87	Холостой ход	0000001011001010110	1000011110011010111110	0000001100000100110000100	0	1	1	0	1	1	1	0	0
88	Холостой ход	0000001011001010110	1100001111001010111111	0000000110000010011000000	0	1	0	0	0	0	1	1	1
89	Холостой ход	0000001011001010110	0110000111001010111111	1000000011000001001100000	0	1	0	0	0	0	1	1	1
90	Холостой ход	0000000101100101011	0011000011110011010111	1000000011000001001100000	0	1	1	0	1	1	1	0	1
91	Холостой ход	1000000010110010101	0011000011110011010111	0100000011000001001100000	0	0	1	0	0	1	0	1	0
92	Холостой ход	0100000001011001010	0001100001111001101011	1010000000110000010011000	0	1	1	1	1	1	1	1	0
93	Холостой ход	0100000001011001010	0000110000111100110101	0101000000110000010011000	0	0	1	1	1	0	1	1	1
94	Холостой ход	1010000000101100101	0000110000111100110101	0010100000011000001001000	0	0	1	0	0	1	0	1	0
95	Холостой ход	1101000000010110010	0000110000111100110101	1001010000000110000010000	0	0	1	0	0	1	0	1	1
96	Холостой ход	0110100000001011001	0000110000111100110101	1100101000000110000010000	0	0	1	0	0	1	0	1	1
97	Холостой ход	1011010000000101100	0000110000111100110101	1110010100000011000001000	0	0	1	0	0	1	0	1	0
98	Холостой ход	0101101000000010110	0000110000111100110101	0111001010000000110000000	0	0	1	0	0	1	0	1	0
99	Холостой ход	0010110100000001011	0000110000111100110101	0011100101000000110000000	0	0	1	0	0	1	0	1	1
100	Холостой ход	0001011010000000101	0000110000111100110101	1001110010100000011000000	0	0	1	0	0	1	0	1	0
	Генерация	0000101101000000010	1000011000011110011010	0100111001010000000011000		1	1	1	1	1	1	0	0
		1000010110100000001	1100001100001111001101	1010011100101000000011000		0	0	0	0	1	1	0	00
		1100001011010000000	1100001100001111001101	1101001110010100000011000		1	0	1	1	1	0	0	000

1100001011010000000	1100001100001111001101	1101001110010100000011		1	0	1	1	1	0	1	0	000
1100001011010000000	1110000110000111100110	1110100111001010000001		1	0	0	0	0	1	1	0	0000
1100001011010000000	1111000011000011110011	0111010011100101000000		1	0	0	0	0	1	1	1	00001
0110000101101000000	1111000011000011110011	0011101001110010100000		1	0	1	1	1	0	1	1	000011
0011000010110100000	0111100001100001111001	0011101001110010100000		0	0	1	0	1	1	0	1	0000111
1001100001011010000	1011110000110000111100	0001110100111001010000		1	1	1	1	1	1	1	1	00001111
1001100001011010000	0101111000011000011110	1000111010011100101000		0	1	1	1	0	1	1	0	000011110
0100110000101101000	1010111100001100001111	0100011101001110010100		0	0	0	0	1	1	1	0	0000111100
1010011000010110100	0101011111000011000111	1010001110100111001010		0	0	0	0	0	1	1	1	00001111001
0101001100001011010	0010101111000011000111	1010001110100111001010		0	0	1	0	1	1	0	1	000011110011
1010100110000101101	0001010111100001100001	1010001110100111001010		0	0	0	1	0	1	1	0	0000111100111
1101010011000010110	1000101011110000110000	0101000111010011001010		1	1	1	1	1	1	1	0	00001111001110
0110101001100001011	0100010101111000011000	0101000111010011001010		1	1	0	1	1	1	0	0	000011110011100
0011010100110000101	0100010101111000011000	0001010001110100111001		0	1	0	0	1	0	1	1	0000111100111001
0011010100110000101	0010000101011110000110	0001010001110100111001		0	1	1	1	0	1	1	0	00001111001110010
0011010100110000101	0001000101011110000110	1000101000111010011001		0	1	1	1	0	1	1	1	000011110011001101

текст: 11101010111100001110000
результат: 0011010100110000101

Тест на 1000 знаков:


ШИФРОВАНИЕ
Введите ключ (текст): ДЕКАБРЬ
Введите номер кадра (целое число от 0 до 255): 132
Введите текст для шифрования: Более всего на свете прокуратор ненавидел запах розового масла, и все теперь предвещало нехороший день, так как запах этот начал преследовать прокуратора с рассвета. Прокур атору казалось, что розовый запах источают кипарисы и пальмы в саду, что к запаху кожаного снаряжения и пота от конвоя примешивается проклятая розовая струя. От флигелей в тылу дворца, где расположилась пришедшая с прокуратором в Ершалаим первая когорта Двенадцатого Молниеносного легиона, заносило дымком в колоннаду через верхнюю площадку сада, и к горьковатому дыму, свидетельствовавшему о том, что ка шевары в кентуриях начали готовить обед, примешивался все тот же жирный розовый дух. «Да, нет сомнений, это она, опять она, непобедимая, ужасная болезнь гемикрания, при которой болит полголовы от нее не т средств, нет никакого спасения попробую не двигать головой».
РЕЗУЛЬТАТ
Ключ (hex): D094D095D09AD090
Номер кадра: 132
Зашифрованный текст (hex): B1BFA57291A2974F903724EABAEA6B785BDF7C88EE714945DEF446400AF58170BE4609FEE10385B70E274EFA7F6262DE7A21DC76E9BCE76602302DE5F21C1BF1786999E9C45F935904CB0A28A7FAC5FA9F11FAA13DA81CA8C212F2A6496C857B577D841CAA507BC9E52BC1CFA915088B8723A1B2EDE07C44FBF2CB88E6E6E97326A24374C56B57C14E6184201CB92794A85A23063A68217082023829AD6A7B3291BB3655AC1116A411A5048554A73D7F8B213039ACCEC56F73C25C8EA8A49118071F32B55EBC85E37922B7AAE7A28421818329E4B4045B9939859E735B42F729313DC8357D72B7F1DCA3F43E9DA5242924BA6F1159087DC92112AFEA8D280DE5F65DCE5FBD290CE90ED675AD2F4DA1B358649B1D577CEC68BF67BE4F8D0F92DF9ECABC614A94D82001D672DA794538D7DA79AE9A6E19FD914D523D3A1438185250C1E2BC2A32B14515426B549BE214C4F3E18E79DDFA705747BE16334AACB583777704ABD53DCBB15306B53B81F11860FF5256A97C79EC469E92118F81F945436858C4B

Блок G: Комбинационные шифры

19. Шифр AES

AES (Advanced Encryption Standard) — это симметричный блочный шифр, принятый в качестве стандарта шифрования в США в 2001 году. Он пришёл на смену устаревшему DES (Data Encryption Standard). AES является одним из наиболее распространённых алгоритмов симметричного шифрования в мире, используется в протоколах SSL/TLS, Wi-Fi (WPA2), файловых архиваторах, банковских системах и во многих других приложениях.

Ключевая особенность: AES работает с блоками данных фиксированного размера (128 бит = 16 байт) и поддерживает ключи длиной 128, 192 или 256 бит.

Блок-схема:

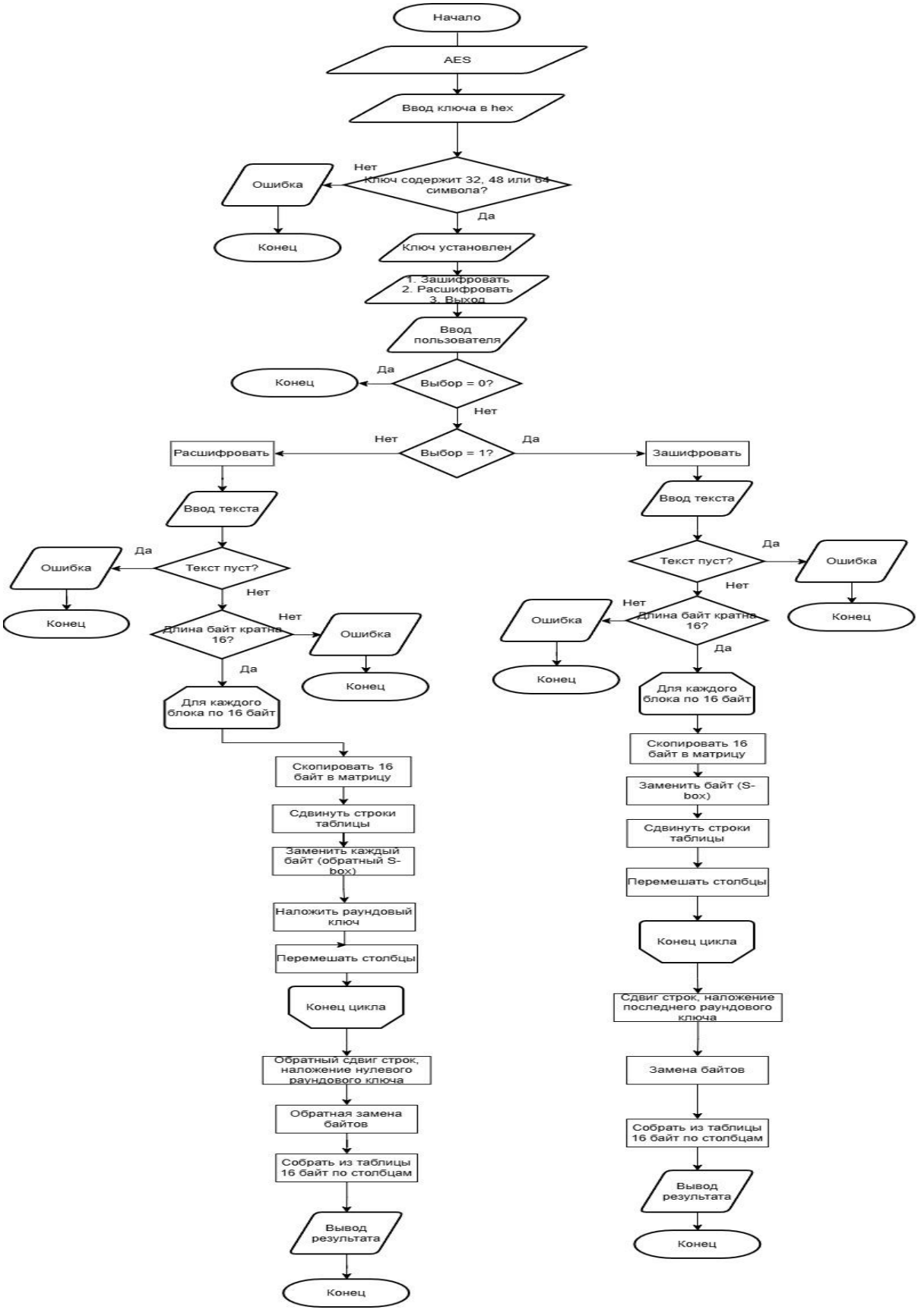


Рисунок 30. Блок-схема шифра AES.

Код программы:

```
import sys
# Таблицы S-box, обратный S-box и Rcon (как в оригинале)
SBOX = [
    0x63, 0x7c, 0x77, 0x7b, 0xf2, 0x6b, 0x6f, 0xc5, 0x30, 0x01, 0x67, 0x2b, 0xfe, 0xd7, 0xab,
0x76,
    0xca, 0x82, 0xc9, 0x7d, 0xfa, 0x59, 0x47, 0xf0, 0xad, 0xd4, 0xa2, 0xaf, 0x9c, 0xa4, 0x72,
0xc0,
    0xb7, 0xfd, 0x93, 0x26, 0x36, 0x3f, 0xf7, 0xcc, 0x34, 0xa5, 0xe5, 0xf1, 0x71, 0xd8, 0x31,
0x15,
    0x04, 0xc7, 0x23, 0xc3, 0x18, 0x96, 0x05, 0x9a, 0x07, 0x12, 0x80, 0xe2, 0xeb, 0x27, 0xb2,
0x75,
    0x09, 0x83, 0x2c, 0x1a, 0x1b, 0x6e, 0x5a, 0xa0, 0x52, 0x3b, 0xd6, 0xb3, 0x29, 0xe3, 0x2f,
0x84,
    0x53, 0xd1, 0x00, 0xed, 0x20, 0xfc, 0xb1, 0x5b, 0x6a, 0xcb, 0xbe, 0x39, 0x4a, 0x4c, 0x58,
0xcf,
    0xd0, 0xef, 0xaa, 0xfb, 0x43, 0x4d, 0x33, 0x85, 0x45, 0xf9, 0x02, 0x7f, 0x50, 0x3c, 0x9f,
0xa8,
    0x51, 0xa3, 0x40, 0x8f, 0x92, 0x9d, 0x38, 0xf5, 0xbc, 0xb6, 0xda, 0x21, 0x10, 0xff, 0xf3,
0xd2,
    0xcd, 0x0c, 0x13, 0xec, 0x5f, 0x97, 0x44, 0x17, 0xc4, 0xa7, 0x7e, 0x3d, 0x64, 0x5d, 0x19,
0x73,
    0x60, 0x81, 0x4f, 0xdc, 0x22, 0x2a, 0x90, 0x88, 0x46, 0xee, 0xb8, 0x14, 0xde, 0x5e, 0x0b,
0xdb,
    0xe0, 0x32, 0x3a, 0x0a, 0x49, 0x06, 0x24, 0x5c, 0xc2, 0xd3, 0xac, 0x62, 0x91, 0x95, 0xe4,
0x79,
    0xe7, 0xc8, 0x37, 0x6d, 0x8d, 0xd5, 0x4e, 0xa9, 0x6c, 0x56, 0xf4, 0xea, 0x65, 0x7a, 0xae,
0x08,
    0xba, 0x78, 0x25, 0x2e, 0x1c, 0xa6, 0xb4, 0xc6, 0xe8, 0xdd, 0x74, 0x1f, 0x4b, 0xbd, 0x8b,
0x8a,
    0x70, 0x3e, 0xb5, 0x66, 0x48, 0x03, 0xf6, 0x0e, 0x61, 0x35, 0x57, 0xb9, 0x86, 0xc1, 0x1d,
0x9e,
    0xe1, 0xf8, 0x98, 0x11, 0x69, 0xd9, 0x8e, 0x94, 0x9b, 0x1e, 0x87, 0xe9, 0xce, 0x55, 0x28,
0xdf,
    0x8c, 0xa1, 0x89, 0x0d, 0xbf, 0xe6, 0x42, 0x68, 0x41, 0x99, 0x2d, 0x0f, 0xb0, 0x54, 0xbb,
0x16
]
INV_SBOX = [
    0x52, 0x09, 0x6a, 0xd5, 0x30, 0x36, 0xa5, 0x38, 0xbf, 0x40, 0xa3, 0x9e, 0x81, 0xf3, 0xd7,
0xfb,
    0x7c, 0xe3, 0x39, 0x82, 0x9b, 0x2f, 0xff, 0x87, 0x34, 0x8e, 0x43, 0x44, 0xc4, 0xde, 0xe9,
0xcb,
    0x54, 0x7b, 0x94, 0x32, 0xa6, 0xc2, 0x23, 0x3d, 0xee, 0x4c, 0x95, 0x0b, 0x42, 0xfa, 0xc3,
0x4e,
    0x08, 0x2e, 0xa1, 0x66, 0x28, 0xd9, 0x24, 0xb2, 0x76, 0x5b, 0xa2, 0x49, 0x6d, 0x8b, 0xd1,
0x25,
    0x72, 0xf8, 0xf6, 0x64, 0x86, 0x68, 0x98, 0x16, 0xd4, 0xa4, 0x5c, 0xcc, 0x5d, 0x65, 0xb6,
0x92,
    0x6c, 0x70, 0x48, 0x50, 0xfd, 0xed, 0xb9, 0xda, 0x5e, 0x15, 0x46, 0x57, 0xa7, 0x8d, 0x9d,
0x84,
    0x90, 0xd8, 0xab, 0x00, 0x8c, 0xbc, 0xd3, 0x0a, 0xf7, 0xe4, 0x58, 0x05, 0xb8, 0xb3, 0x45,
0x06,
    0xd0, 0x2c, 0x1e, 0x8f, 0xca, 0x3f, 0x0f, 0x02, 0xc1, 0xaf, 0xbd, 0x03, 0x01, 0x13, 0x8a,
0x6b,
    0x3a, 0x91, 0x11, 0x41, 0x4f, 0x67, 0xdc, 0xea, 0x97, 0xf2, 0xcf, 0xce, 0xf0, 0xb4, 0xe6,
0x73,
    0x96, 0xac, 0x74, 0x22, 0xe7, 0xad, 0x35, 0x85, 0xe2, 0xf9, 0x37, 0xe8, 0x1c, 0x75, 0xdf,
0x6e,
    0x47, 0xf1, 0x1a, 0x71, 0x1d, 0x29, 0xc5, 0x89, 0x6f, 0xb7, 0x62, 0x0e, 0xaa, 0x18, 0xbe,
0x1b,
    0xfc, 0x56, 0x3e, 0x4b, 0xc6, 0xd2, 0x79, 0x20, 0x9a, 0xdb, 0xc0, 0xfe, 0x78, 0xcd, 0x5a,
0xf4,
    0x1f, 0xdd, 0xa8, 0x33, 0x88, 0x07, 0xc7, 0x31, 0xb1, 0x12, 0x10, 0x59, 0x27, 0x80, 0xec,
0x5f,
    0x60, 0x51, 0x7f, 0xa9, 0x19, 0xb5, 0x4a, 0x0d, 0x2d, 0xe5, 0x7a, 0x9f, 0x93, 0xc9, 0x9c,
0xef,
    0xa0, 0xe0, 0x3b, 0x4d, 0xae, 0x2a, 0xf5, 0xb0, 0xc8, 0xeb, 0xbb, 0x3c, 0x83, 0x53, 0x99,
0x61,
    0x17, 0x2b, 0x04, 0x7e, 0xba, 0x77, 0xd6, 0x26, 0xe1, 0x69, 0x14, 0x63, 0x55, 0x21, 0x0c,
0x7d
]
RCON = [0x00, 0x01, 0x02, 0x04, 0x08, 0x10, 0x20, 0x40, 0x80, 0x1b, 0x36]
def xtime(x):
    return ((x << 1) ^ (0x1b if (x & 0x80) else 0x00)) & 0xff
class AES:
    def __init__(self):
```

```
        self.Nk = 0
        self.Nr = 0
        self.w = [0] * (60 * 4)

    def set_key(self, key):
        key_len = len(key)
        self.Nk = key_len // 4
        self.Nr = {4: 10, 6: 12, 8: 14}[self.Nk]
        for i in range(self.Nk):
            self.w[4*i + 0] = key[4*i + 0]
            self.w[4*i + 1] = key[4*i + 1]
            self.w[4*i + 2] = key[4*i + 2]
            self.w[4*i + 3] = key[4*i + 3]
        for i in range(self.Nk, 4 * (self.Nr + 1)):
            temp = [self.w[4*(i-1)+0], self.w[4*(i-1)+1], self.w[4*(i-1)+2], self.w[4*(i-1)+3]]
            if i % self.Nk == 0:
                temp = [temp[1], temp[2], temp[3], temp[0]]
                temp = [SBOX[b] for b in temp]
                temp[0] ^= RCON[i // self.Nk]
            elif self.Nk > 6 and i % self.Nk == 4:
                temp = [SBOX[b] for b in temp]
            self.w[4*i + 0] = self.w[4*(i - self.Nk) + 0] ^ temp[0]
            self.w[4*i + 1] = self.w[4*(i - self.Nk) + 1] ^ temp[1]
            self.w[4*i + 2] = self.w[4*(i - self.Nk) + 2] ^ temp[2]
            self.w[4*i + 3] = self.w[4*(i - self.Nk) + 3] ^ temp[3]

    def add_round_key(self, state, rnd):
        for i in range(4):
            for j in range(4):
                state[j][i] ^= self.w[4 * (rnd * 4 + i) + j]

    def sub_bytes(self, state):
        for i in range(4):
            for j in range(4):
                state[i][j] = SBOX[state[i][j]]

    def inv_sub_bytes(self, state):
        for i in range(4):
            for j in range(4):
                state[i][j] = INV_SBOX[state[i][j]]

    def shift_rows(self, state):
        state[1] = state[1][1:] + state[1][:1]
        state[2] = state[2][2:] + state[2][:2]
        state[3] = state[3][3:] + state[3][:3]

    def inv_shift_rows(self, state):
        state[1] = state[1][-1:] + state[1][:-1]
        state[2] = state[2][-2:] + state[2][:-2]
        state[3] = state[3][-3:] + state[3][:-3]

    def mix_columns(self, state):
        for j in range(4):
            a = state[0][j]
            b = state[1][j]
            c = state[2][j]
            d = state[3][j]
            state[0][j] = xtime(a) ^ xtime(b) ^ b ^ c ^ d
            state[1][j] = a ^ xtime(b) ^ xtime(c) ^ c ^ d
            state[2][j] = a ^ b ^ xtime(c) ^ xtime(d) ^ d
            state[3][j] = xtime(a) ^ a ^ b ^ c ^ xtime(d)

    def inv_mix_columns(self, state):
        for j in range(4):
            a = state[0][j]
            b = state[1][j]
            c = state[2][j]
            d = state[3][j]
            state[0][j] = (xtime(xtime(xtime(a) ^ xtime(c)) ^ xtime(xtime(b) ^ xtime(d))) ^
                           xtime(xtime(a ^ c) ^ xtime(b ^ d)) ^ a ^ b ^ c ^ d) & 0xff
            state[1][j] = (xtime(xtime(xtime(b) ^ xtime(d)) ^ xtime(xtime(a) ^ xtime(c))) ^
                           xtime(xtime(b ^ d) ^ xtime(a ^ c)) ^ a ^ b ^ c ^ d) & 0xff
            state[2][j] = (xtime(xtime(xtime(c) ^ xtime(a)) ^ xtime(xtime(d) ^ xtime(b))) ^
                           xtime(xtime(c ^ a) ^ xtime(d ^ b)) ^ a ^ b ^ c ^ d) & 0xff
            state[3][j] = (xtime(xtime(xtime(d) ^ xtime(b)) ^ xtime(xtime(c) ^ xtime(a))) ^
                           xtime(xtime(d ^ b) ^ xtime(c ^ a)) ^ a ^ b ^ c ^ d) & 0xff

    def encrypt_block(self, plain):
        state = [[0]*4 for _ in range(4)]
        for i in range(4):
            for j in range(4):
                state[j][i] = plain[i*4 + j]
        self.add_round_key(state, 0)
        for rnd in range(1, self.Nr):
            self.sub_bytes(state)
            self.shift_rows(state)
```

```
                    self.mix_columns(state)
                    self.add_round_key(state, rnd)
                self.sub_bytes(state)
                self.shift_rows(state)
                self.add_round_key(state, self.Nr)
                out = bytearray(16)
                for i in range(4):
                    for j in range(4):
                        out[i*4 + j] = state[j][i]
                return bytes(out)
            def decrypt_block(self, cipher):
                state = [[0]*4 for _ in range(4)]
                for i in range(4):
                    for j in range(4):
                        state[j][i] = cipher[i*4 + j]
                self.add_round_key(state, self.Nr)
                for rnd in range(self.Nr - 1, 0, -1):
                    self.inv_shift_rows(state)
                    self.inv_sub_bytes(state)
                    self.add_round_key(state, rnd)
                    self.inv_mix_columns(state)
                self.inv_shift_rows(state)
                self.inv_sub_bytes(state)
                self.add_round_key(state, 0)
                out = bytearray(16)
                for i in range(4):
                    for j in range(4):
                        out[i*4 + j] = state[j][i]
                return bytes(out)
        class AESCipher:
            def __init__(self):
                self.aes = AES()
                self.key_set = False
            def set_key_from_hex(self, key_hex):
                key_hex = key_hex.strip().lower()
                if len(key_hex) not in (32, 48, 64):
                    print("Ошибка: длина ключа должна быть 32, 48 или 64 hex-символа (16, 24 или 32
байта).")
                    return False
                try:
                    key_bytes = bytes.fromhex(key_hex)
                except ValueError:
                    print("Ошибка: неверный hex-формат ключа.")
                    return False
                self.aes.set_key(key_bytes)
                self.key_set = True
                bits = len(key_bytes) * 8
                print(f"Ключ установлен ({bits} бит): {key_hex.upper()}")
                return True
            def encrypt_hex(self, plain_hex):
                if not self.key_set:
                    print("Ошибка: ключ не установлен!")
                    return ""
                try:
                    data = bytes.fromhex(plain_hex)
                except ValueError:
                    print("Ошибка: неверный hex-формат открытого текста.")
                    return ""
                # Требуем, чтобы длина данных была кратна 16 байтам
                if len(data) % 16 != 0:
                    print("Ошибка: длина открытого текста должна быть кратна 16 байтам (32 hex-
символа).")
                    return ""
                cipher = bytearray()
                for i in range(0, len(data), 16):
                    block = data[i:i+16]
                    enc_block = self.aes.encrypt_block(block)
                    cipher.extend(enc_block)
                return cipher.hex().upper()
            def decrypt_hex(self, cipher_hex):
                if not self.key_set:
                    print("Ошибка: ключ не установлен!")
                    return ""
                try:
                    cipher = bytes.fromhex(cipher_hex)
                except ValueError:
                    print("Ошибка: неверный hex-формат зашифрованного текста.")
                    return ""
```

```
        if len(cipher) % 16 != 0:
            print("Ошибка: длина зашифрованного текста должна быть кратна 16 байтам.")
            return ""
        plain = bytearray()
        for i in range(0, len(cipher), 16):
            block = cipher[i:i+16]
            dec_block = self.aes.decrypt_block(block)
            plain.extend(dec_block)
        return plain.hex().upper()
def main():
    cipher = AESCipher()
    while True:
        print("\nAES")
        print("1. Установить ключ (hex)")
        print("2. Зашифровать hex")
        print("3. Расшифровать hex")
        print("4. Выход")
        choice = input("Ваш выбор: ").strip()
        if choice == '1':
            key_hex = input("Введите ключ в hex: ").strip()
            cipher.set_key_from_hex(key_hex)
        elif choice == '2':
            plain_hex = input("Введите открытый текст в hex: ").strip()
            enc = cipher.encrypt_hex(plain_hex)
            if enc:
                print(f"Зашифрованный текст: {enc}")
        elif choice == '3':
            cipher_hex = input("Введите зашифрованный текст в hex: ").strip()
            dec = cipher.decrypt_hex(cipher_hex)
            if dec:
                print(f"Расшифрованный текст: {dec}")
        elif choice == '4':
            print("Программа завершена.")
            break
        else:
            print("Неверный выбор.")
if __name__ == "__main__":
    main()
```

Результат работы программы:

```
AES
1. Установить ключ (hex)
2. Зашифровать hex
3. Расшифровать hex
4. Выход
Ваш выбор: 1
Введите ключ в hex: 000102030405060708090a0b0c0d0e0f1011121314151617
Ключ установлен (192 бит): 000102030405060708090A0B0C0D0E0F1011121314151617

AES
1. Установить ключ (hex)
2. Зашифровать hex
3. Расшифровать hex
4. Выход
Ваш выбор: 2
Введите открытый текст в hex: 00112233445566778899aabbccddeeff
Зашифрованный текст: DDA97CA4864CDFE06EAF70A0EC0D7191

AES
1. Установить ключ (hex)
2. Зашифровать hex
3. Расшифровать hex
4. Выход
Ваш выбор: 3
Введите зашифрованный текст в hex: DDA97CA4864CDFE06EAF70A0EC0D7191
Расшифрованный текст: 52EAE9515657EDECE35B58E0E7E65C5D
```

Рисунок 31. Результат работы шифра AES.

Блок H: Ассиметричные шифры

21. Шифр RSA

RSA — это один из первых и наиболее известных асимметричных криптографических алгоритмов, названный в честь его создателей: Рона Ривеста, Ади Шамира и Леонарда Адлемана (1977 год). Алгоритм основан на вычислительной сложности факторизации больших целых чисел (разложения числа на простые множители).

Ключевая особенность: RSA использует два ключа: открытый (для шифрования) и закрытый (для расшифрования). Невозможно вычислить закрытый ключ из открытого за разумное время при достаточно больших размерах ключа.

Блок-схема:

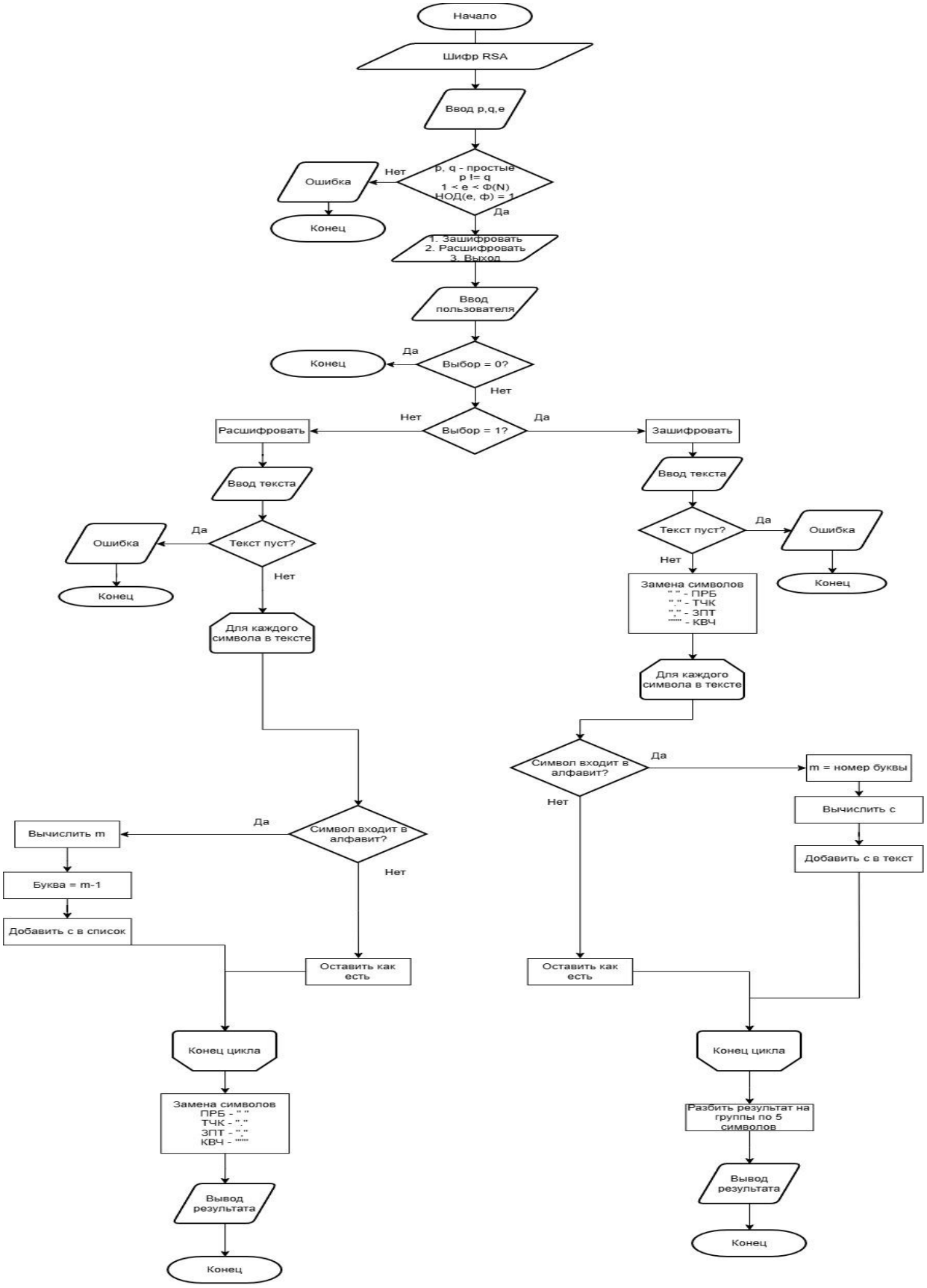


Рисунок 32. Блок-схема шифра RSA.

Код программы:

```
import sys
import math

# ---- Вспомогательные математические функции ----
def mod_pow(a, exp, mod):
    """Быстрое возведение в степень по модулю"""
    result = 1
    a %= mod
    while exp > 0:
        if exp & 1:
            result = (result * a) % mod
        a = (a * a) % mod
        exp >>= 1
    return result

def mod_inverse(a, m):
    """Расширенный алгоритм Евклида для нахождения обратного элемента"""
    m0, x0, x1 = m, 0, 1
    if m == 1:
        return 0
    while a > 1:
        q = a // m
        a, m = m, a % m
        x0, x1 = x1 - q * x0, x0
    return x1 + m0 if x1 < 0 else x1

def is_prime(n):
    """Проверка простоты (для небольших чисел)"""
    if n < 2:
        return False
    if n == 2:
        return True
    if n % 2 == 0:
        return False
    i = 3
    while i * i <= n:
        if n % i == 0:
            return False
        i += 2
    return True

def gcd(a, b):
    while b:
        a, b = b, a % b
    return a

class RSACipher:
    def __init__(self):
        self.p = self.q = self.N = self.phi = self.E = self.D = 0
        self.key_set = False
        self.alphabet = "АБВГДЕЖЗИЙКЛМНОПРСТУФХЦЧШЩЬЫЬЭЮЯ"   # 32 буквы

    def to_upper_russian(self, c):
        """Приведение русской буквы к верхнему регистру, Ё -> Е"""
        if 'а' <= c <= 'я' or 'А' <= c <= 'Я':
            return c.upper()
        if c == 'ё' or c == 'Ё':
            return 'Е'
        return c

    def is_russian_char(self, c):
        """Проверка, является ли символ русской буквой (включая Ё)"""
        return ('А' <= c <= 'Я') or ('а' <= c <= 'я') or c in ('Ё', 'ё')

    def prepare_text(self, text):
        """Подготовка текста: пробел -> ПРБ, точка -> ТЧК, запятая -> ЗПТ, кавычка -> КВЧ"""
        result = []
        for ch in text:
            if ch == ' ':
                result.append("ПРБ")
            elif ch == '.':
                result.append("ТЧК")
            elif ch == ',':
                result.append("ЗПТ")
            elif ch == '"':
                result.append("КВЧ")
            else:
                upper = self.to_upper_russian(ch)
                if self.is_russian_char(upper):
                    result.append(upper)
        return ''.join(result)

    def restore_text(self, text):
```

```
    """Восстановление пробелов, точек, запятых и кавычек после расшифровки"""
    result = []
    i = 0
    n = len(text)
    while i < n:
        if i + 2 < n:
            triple = text[i:i+3]
            if triple == "ПРБ":
                result.append(' ')
                i += 3
                continue
            elif triple == "ТЧК":
                result.append('.')
                i += 3
                continue
            elif triple == "ЗПТ":
                result.append(',')
                i += 3
                continue
            elif triple == "КВЧ":
                result.append('"')
                i += 3
                continue
        result.append(text[i])
        i += 1
    return ''.join(result)
def char_to_int(self, c):
    """Буква -> число 1..32 (нумерация с 1)"""
    try:
        return self.alphabet.index(c) + 1
    except ValueError:
        return -1
def int_to_char(self, n):
    """Число 1..32 -> буква"""
    if 1 <= n <= len(self.alphabet):
        return self.alphabet[n-1]
    return '?'
# ---- Установка ключа ----
def set_key(self, prime_p, prime_q, exp_e):
    if not is_prime(prime_p):
        raise ValueError("p не является простым числом.")
    if not is_prime(prime_q):
        raise ValueError("q не является простым числом.")
    if prime_p == prime_q:
        raise ValueError("p и q должны быть разными.")
    N = prime_p * prime_q
    phi = (prime_p - 1) * (prime_q - 1)
    if exp_e <= 1 or exp_e >= phi:
        raise ValueError("E должно быть в диапазоне 1 < E < ϕ(N).")
    if gcd(exp_e, phi) != 1:
        raise ValueError("E и ϕ(N) должны быть взаимно просты.")
    if N < 32:
        raise ValueError(f"Модуль N = {N} слишком мал для кодирования алфавита.")
    private_exponent = mod_inverse(exp_e, phi)
    if exp_e == private_exponent:
        raise ValueError("Открытый и секретный показатели совпадают; выберите другое E.")
    self.p = prime_p
    self.q = prime_q
    self.N = N
    self.phi = phi
    self.E = exp_e
    self.D = private_exponent
    self.key_set = True
    print("\nКЛЮЧИ УСТАНОВЛЕНЫ")
    print(f"p = {self.p}, q = {self.q}")
    print(f"N = {self.N}")
    print(f"ϕ(N) = {self.phi}")
    print(f"Открытый ключ (E, N) = ({self.E}, {self.N})")
    print(f"Секретный ключ D = {self.D}")
    return True
# ---- Шифрование ----
def encrypt(self, plain_text):
    if not self.key_set:
        print("Ошибка: ключ не установлен!")
        return ""
```

```
        prepared = self.prepare_text(plain_text)
        if not prepared:
            print("Ошибка: текст пуст!")
            return ""
        print(f"\nТекст после обработки: {prepared}")
        cipher_nums = []
        print("\n--- ШИФРОВАНИЕ ---")
        print("Символ -> Код -> Шифр")
        print("-------------------------------")
        for ch in prepared:
            m = self.char_to_int(ch)
            if m == -1:
                print(f"Ошибка: символ {ch} не найден в алфавите!")
                return ""
            ciph = mod_pow(m, self.E, self.N)
            cipher_nums.append(ciph)
            print(f"   {ch}    ->    {m:2d}     -> {ciph}")
        # Формируем строку с числами через пробел
        result = ' '.join(str(x) for x in cipher_nums)
        print("\nЗАШИФРОВАННЫЙ ТЕКСТ")
        print(result)
        return result
    # ---- Расшифровка ----
    def decrypt(self, cipher_text):
        if not self.key_set:
            print("Ошибка: ключ не установлен!")
            return ""
        # Разбираем строку с числами, разделёнными пробелами
        parts = cipher_text.strip().split()
        if not parts:
            print("Ошибка: не удалось разобрать зашифрованный текст!")
            return ""
        try:
            cipher_nums = [int(x) for x in parts]
        except ValueError:
            print("Ошибка: зашифрованный текст должен содержать только целые числа!")
            return ""
        print("\n--- РАСШИФРОВАНИЕ ---")
        print("Шифр -> Код -> Символ")
        print("-------------------------------")
        decrypted_chars = []
        for ciph in cipher_nums:
            m = mod_pow(ciph, self.D, self.N)
            if m < 1 or m > len(self.alphabet):
                print(f"Ошибка: расшифрованное число {m} вне диапазона 1..32!")
                return ""
            ch = self.int_to_char(m)
            decrypted_chars.append(ch)
            print(f"{ciph} ->    {m:2d}     -> {ch}")
        decrypted_str = ''.join(decrypted_chars)
        result = self.restore_text(decrypted_str)
        print("\nРАСШИФРОВАННЫЙ ТЕКСТ")
        print(result)
        return result

def process_user_input(cipher):
    print("\nRSA")
    print("1. Установить ключ (p, q, e)")
    print("2. Зашифровать текст")
    print("3. Расшифровать текст")
    print("4. Выход")
    choice = input("Ваш выбор: ").strip()
    if choice == '1':
        try:
            p = int(input("Введите простое число p (например, 23): "))
            q = int(input("Введите простое число q (например, 29): "))
            e = int(input("Введите открытый ключ e (например, 3 или 65537): "))
            cipher.set_key(p, q, e)
        except ValueError:
            print("Ошибка: введите целые числа.")
    elif choice == '2':
        text = input("Введите текст для шифрования: ")
        cipher.encrypt(text)
    elif choice == '3':
        text = input("Введите зашифрованный текст (числа через пробел): ")
        cipher.decrypt(text)
    elif choice == '4':
```

```
            print("Программа завершена.")
            sys.exit(0)
        else:
            print("Неверный выбор!")

    def main():
        cipher = RSACipher()
        while True:
            process_user_input(cipher)

    if __name__ == "__main__":
        main()
```

Результат работы программы:

```
RSA
1. Установить ключ (p, q, e)
2. Зашифровать текст
3. Расшифровать текст
4. Выход
Ваш выбор: 1
Введите простое число p (например, 23): 3
Введите простое число q (например, 29): 11
Введите открытый ключ e (например, 3 или 65537): 3

КЛЮЧИ УСТАНОВЛЕНЫ
p = 3, q = 11
N = 33
ф(N) = 20
Открытый ключ (E, N) = (3, 33)
Секретный ключ D = 7
```

```
Введите текст для шифрования: Леопард не может изменить своих пятен

Текст после обработки: ЛЕОПАРДПРБНЕПРБМОЖЕТПРБИЗМЕНИТЬПРБСВОИХПРБПЯТЕН

--- ШИФРОВАНИЕ ---
Символ -> Код -> Шифр
--------------------------------
   Л    ->   12    -> 12
   Е    ->    6    -> 18
   О    ->   15    -> 9
   П    ->   16    -> 4
   А    ->    1    -> 1
   Р    ->   17    -> 29
   Д    ->    5    -> 26
   П    ->   16    -> 4
   Р    ->   17    -> 29
   Б    ->    2    -> 8
   Н    ->   14    -> 5
   Е    ->    6    -> 18
   П    ->   16    -> 4
   Р    ->   17    -> 29
   Б    ->    2    -> 8
   М    ->   13    -> 19
   О    ->   15    -> 9
   Ж    ->    7    -> 13
   Е    ->    6    -> 18
   Т    ->   19    -> 28
   П    ->   16    -> 4
   Р    ->   17    -> 29
   Б    ->    2    -> 8
   И    ->    9    -> 3
   З    ->    8    -> 17
   М    ->   13    -> 19
   Е    ->    6    -> 18
   Н    ->   14    -> 5
   И    ->    9    -> 3
   Т    ->   19    -> 28
   Ь    ->   29    -> 2
   П    ->   16    -> 4
   Р    ->   17    -> 29
   Б    ->    2    -> 8
   С    ->   18    -> 24
   В    ->    3    -> 27
   О    ->   15    -> 9
   И    ->    9    -> 3
   Х    ->   22    -> 22
   П    ->   16    -> 4
   Р    ->   17    -> 29
   Б    ->    2    -> 8
   П    ->   16    -> 4
   Я    ->   32    -> 32
   Т    ->   19    -> 28
```

```
ЗАШИФРОВАННЫЙ ТЕКСТ
12 18 9 4 1 29 26 4 29 8 5 18 4 29 8 19 9 13 18 28 4 29 8 3 17 19 18 5 3 28 2 4 29 8 24 27 9 3 22 4 29 8 4 32 28 18 5

RSA
1. Установить ключ (p, q, e)
2. Зашифровать текст
3. Расшифровать текст
4. Выход
Ваш выбор: 3
Введите зашифрованный текст (числа через пробел): 12 18 9 4 1 29 26 4 29 8 5 18 4 29 8 19 9 13 18 28 4 29 8 3 17 19 18 5 3 28 2 4 29 8 24 27 9 3 22 4 29 8 4 32 28 18 5

--- РАСШИФРОВАНИЕ ---
Шифр -> Код -> Символ
```

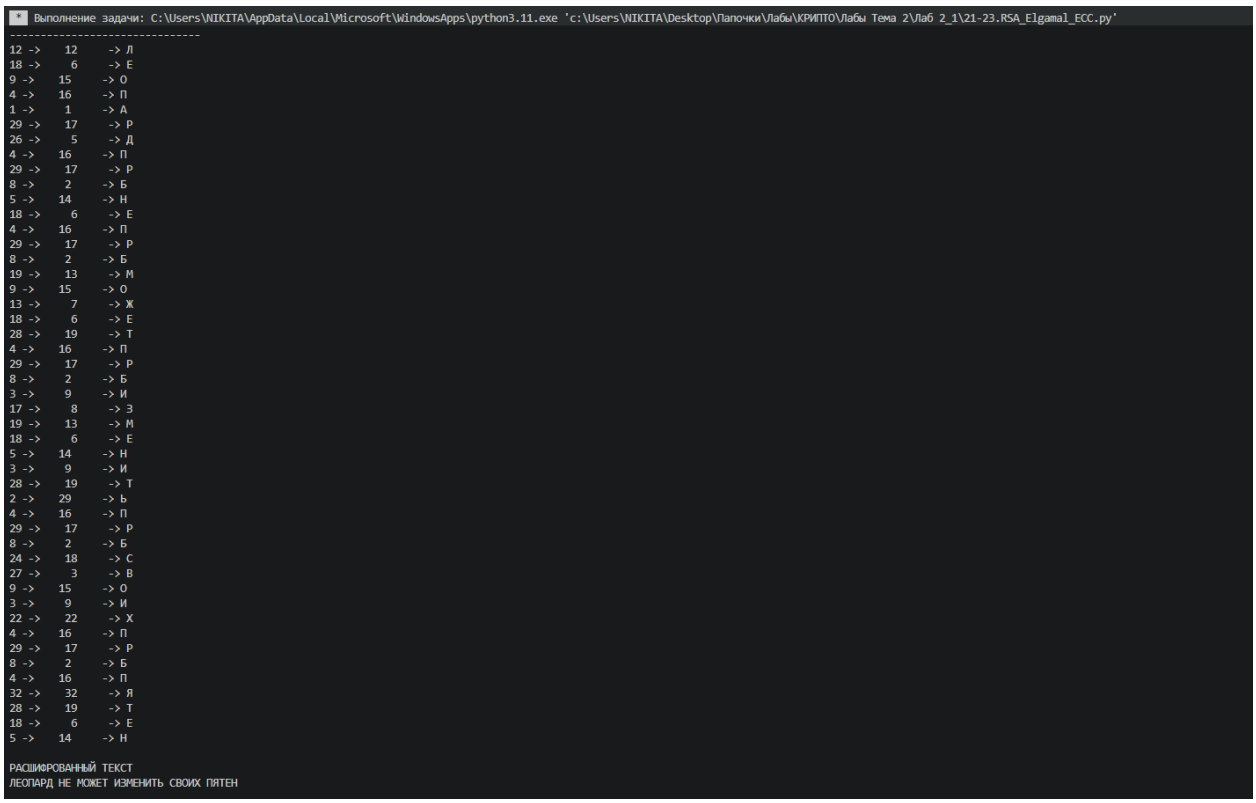


Рисунок 33. Результат шифра RSA.

Результат шифра в тетради:

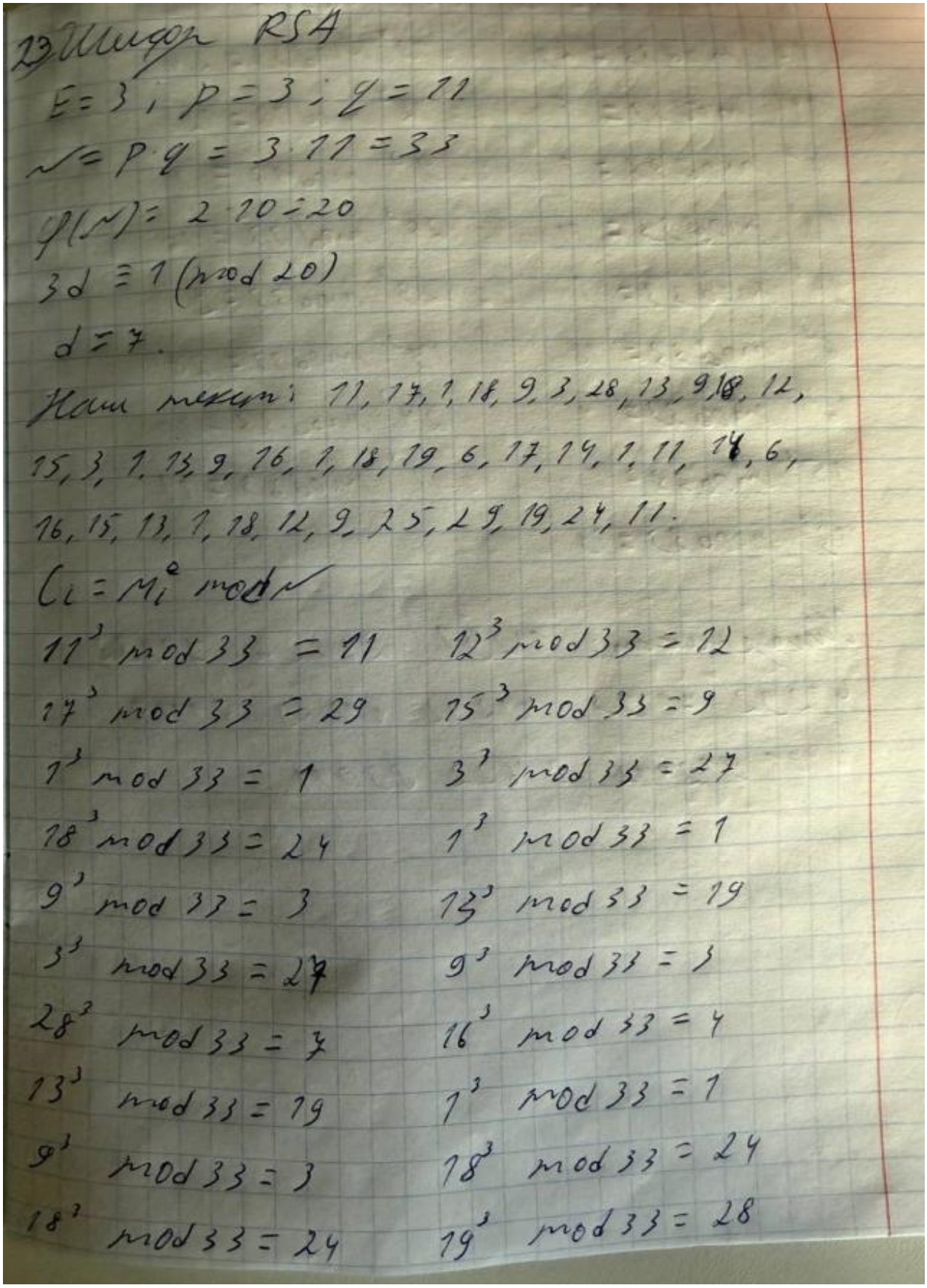


Рисунок 34. Шифр RSA в тетради.

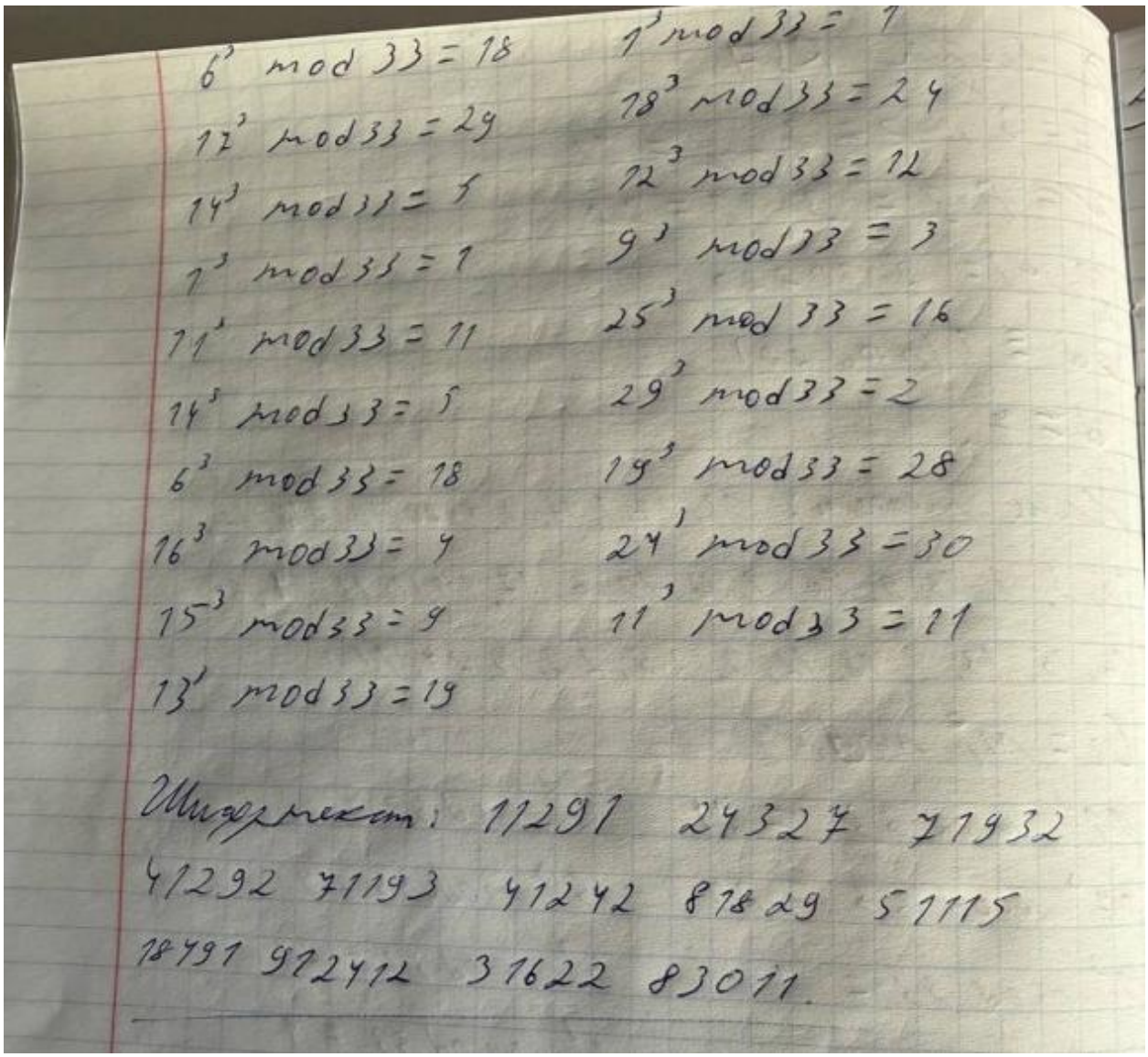


Рисунок 35. Шифр RSA в тетради (2).

Блок I: алгоритмы цифровых подписей

24. Цифровая подпись RSA

RSA - первый алгоритм цифровой подписи, который был разработан в 1977 году в Массачусетском технологическом институт и назван по первым буквам фамилий ее разработчиков (Ronald Rivest, Adi Shamir и Leonard Adleman). RSA основывается на сложности разложения большого числа n на простые множители.

1. Берутся два очень больших простых числа P и Q и находятся произведение простых чисел $N=P\times Q$ и функция Эйлера от этого произведения $\varphi(N)=(P-1)\times(Q-1)$.

2. Выбирается случайное целое число E: $1 < E < \varphi(N)$ и взаимно простое с $\varphi(N)$, и вычисляется D = 1 MOD $\varphi(N))/E$.

3. Потом E и N публикуются как открытый ключ, D сохраняется в тайне.

4. Если M — сообщение, а h(M) = m – хеш-код сообщения, длина которого должна быть в интервале (1,N), то электронная цифровая подпись S получается шифрованием хеш-кода сообщения m возведением в степень D по модулю N: S = mD MOD N.

5. Получателю отправляется сообщение M и подпись S.

6. Получатель сообщения хеширует M, получает хеш-код m’.

7. Проверяет подпись S: расшифровывает хеш-код, возведя S в степень E (число E ему известно) по модулю N: m = SE MOD N.

8. Сравнивает m и m’: если m = m’ - подпись верна.

Блок – схема алгоритма:

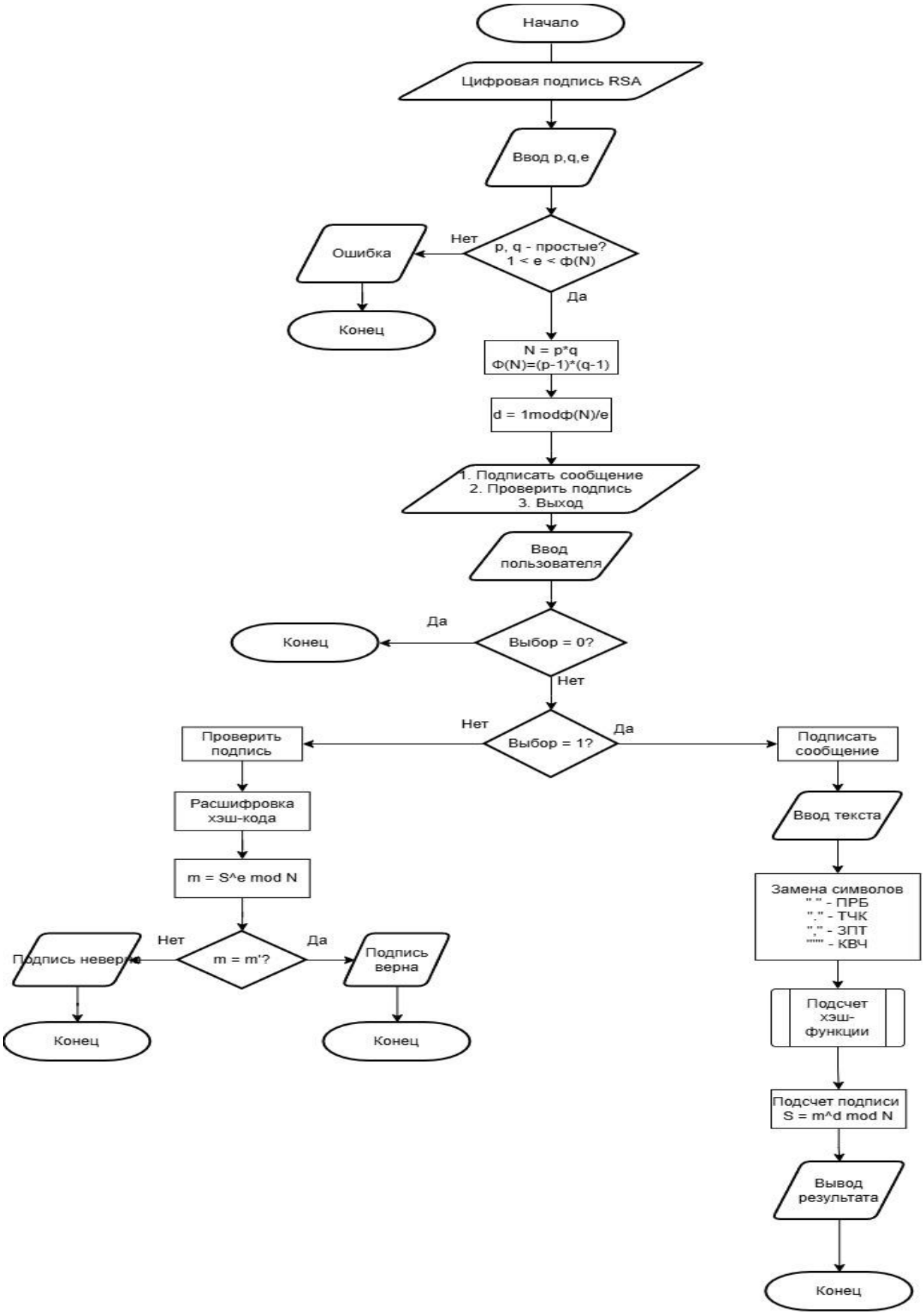


Рисунок 37. Блок-схема цифровой подписи RSA.

Код программы:

```
import random
import os
def is_prime(n: int, k: int = 10) -> bool:
    if n < 2:
        return False
    if n in (2, 3):
        return True
    if n % 2 == 0:
        return False
    r, d = 0, n - 1
    while d % 2 == 0:
        r += 1
        d //= 2
    for _ in range(k):
        a = random.randint(2, n - 2)
        x = pow(a, d, n)
        if x == 1 or x == n - 1:
            continue
        for _ in range(r - 1):
            x = pow(x, 2, n)
            if x == n - 1:
                break
        else:
            return False
    return True
def gcd(a, b):
    while b:
        a, b = b, a % b
    return a
def mod_inverse(a, m):
    m0, x0, x1 = m, 0, 1
    if m == 1:
        return 0
    while a > 1:
        q = a // m
        a, m = m, a % m
        x0, x1 = x1 - q * x0, x0
    return x1 + m0 if x1 < 0 else x1
def prepare_text(message: str) -> str:
    result = ""
    for ch in message.upper():
        if ch == ' ':
            result += "ПРБ"
        elif ch == '.':
            result += "ТЧК"
        else:
            result += ch
    return result
def square_hash(message: str, N: int) -> int:
    alphabet = "АБВГДЕЖЗИЙКЛМНОПРСТУФХЦЧШЩЪЫЬЭЮЯ"
    h = 0
    print(f"Начальное значение h0 = {h}")
    for ch in message:
        if ch in alphabet:
            code = alphabet.index(ch) + 1
        else:
            continue
        h = (h + code) % N
        h = (h * h) % N
    print(f"Итоговый хеш: {h}")
    return h
class RSASignature:
    def __init__(self):
        self.p = None
        self.q = None
        self.N = None
        self.phi = None
        self.e = None
        self.d = None
        self.keys_ready = False
        self.last_message = ""
        self.last_hash = 0
        self.last_signature = 0
    def set_keys(self, p, q, e):
        if not is_prime(p):
```

```
            raise ValueError("p не является простым числом")
        if not is_prime(q):
            raise ValueError("q не является простым числом")
        if p == q:
            raise ValueError("p и q должны быть разными")
        self.p = p
        self.q = q
        self.N = p * q
        self.phi = (p - 1) * (q - 1)
        if not (1 < e < self.phi):
            raise ValueError("e должен быть в диапазоне 1 < e < ϕ(N)")
        if gcd(e, self.phi) != 1:
            raise ValueError("e и ϕ(N) не взаимно просты")
        self.e = e
        self.d = mod_inverse(self.e, self.phi)
        if self.e == self.d:
            raise ValueError("Открытый и секретный ключи совпадают")
        if self.N <= 32:
            raise ValueError(f"Модуль N = {self.N} должен быть больше 32")
        self.keys_ready = True
        print("\nКлючи установлены")
        print(f"p = {p}, q = {q}")
        print(f"N = {self.N}")
        print(f"ϕ(N) = {self.phi}")
        print(f"Открытый ключ e = {e}")
        print(f"Секретный ключ d = {self.d}")
    def sign(self, message):
        if not self.keys_ready:
            raise RuntimeError("Ключи не установлены")
        prepared = prepare_text(message)
        h = square_hash(prepared, self.N)
        signature = pow(h, self.d, self.N)
        self.last_message = message
        self.last_hash = h
        self.last_signature = signature
        print(f"\nПодпись = {h}^{self.d} mod {self.N} = {signature}")
        return signature
    def verify(self):
        if not self.keys_ready:
            raise RuntimeError("Ключи не установлены")
        if not self.last_message:
            raise RuntimeError("Нет подписанного сообщения")
        prepared = prepare_text(self.last_message)
        h = square_hash(prepared, self.N)
        decrypted = pow(self.last_signature, self.e, self.N)
        print(f"\nПроверка подписи")
        print(f"Сообщение: {self.last_message}")
        print(f"Хэш сообщения (число): {h}")
        print(f"Подпись: {self.last_signature}")
        print(f"Вычисляем (подпись)^e mod N = {decrypted}")
        print(f"Сравниваем с хэшем: {h} == {decrypted} => {h == decrypted}")
        return h == decrypted
def clear_screen():
    os.system('cls' if os.name == 'nt' else 'clear')
def main():
    rsa = RSASignature()
    while True:
        clear_screen()
        print("\nЦИФРОВАЯ ПОДПИСЬ RSA")
        print("1. Ввести ключи (p, q, e)")
        print("2. Подписать сообщение")
        print("3. Проверить подпись")
        print("4. Выход")
        choice = input("Ваш выбор: ").strip()
        if choice == '1':
            try:
                p = int(input("Введите простое число p: "))
                q = int(input("Введите простое число q: "))
                e = int(input("Введите открытый ключ e: "))
                rsa.set_keys(p, q, e)
            except Exception as ex:
                print(f"Ошибка: {ex}")
        elif choice == '2':
            if not rsa.keys_ready:
                print("Сначала установите ключи (пункт 1).")
            else:
```

```
                msg = input("Введите сообщение для подписи: ")
                try:
                    rsa.sign(msg)
                except Exception as ex:
                    print(f"Ошибка: {ex}")
        elif choice == '3':
            if not rsa.keys_ready:
                print("Сначала установите ключи (пункт 1).")
            elif not rsa.last_message:
                print("Сначала подпишите сообщение (пункт 2).")
            else:
                try:
                    valid = rsa.verify()
                    if valid:
                        print("\nПОДПИСЬ ВЕРНА")
                    else:
                        print("\nПОДПИСЬ НЕВЕРНА")
                except Exception as ex:
                    print(f"Ошибка: {ex}")
        elif choice == '4':
            print("Программа завершена.")
            break
        else:
            print("Неверный выбор")
if __name__ == "__main__":
    main()
```

Результат работы программы:

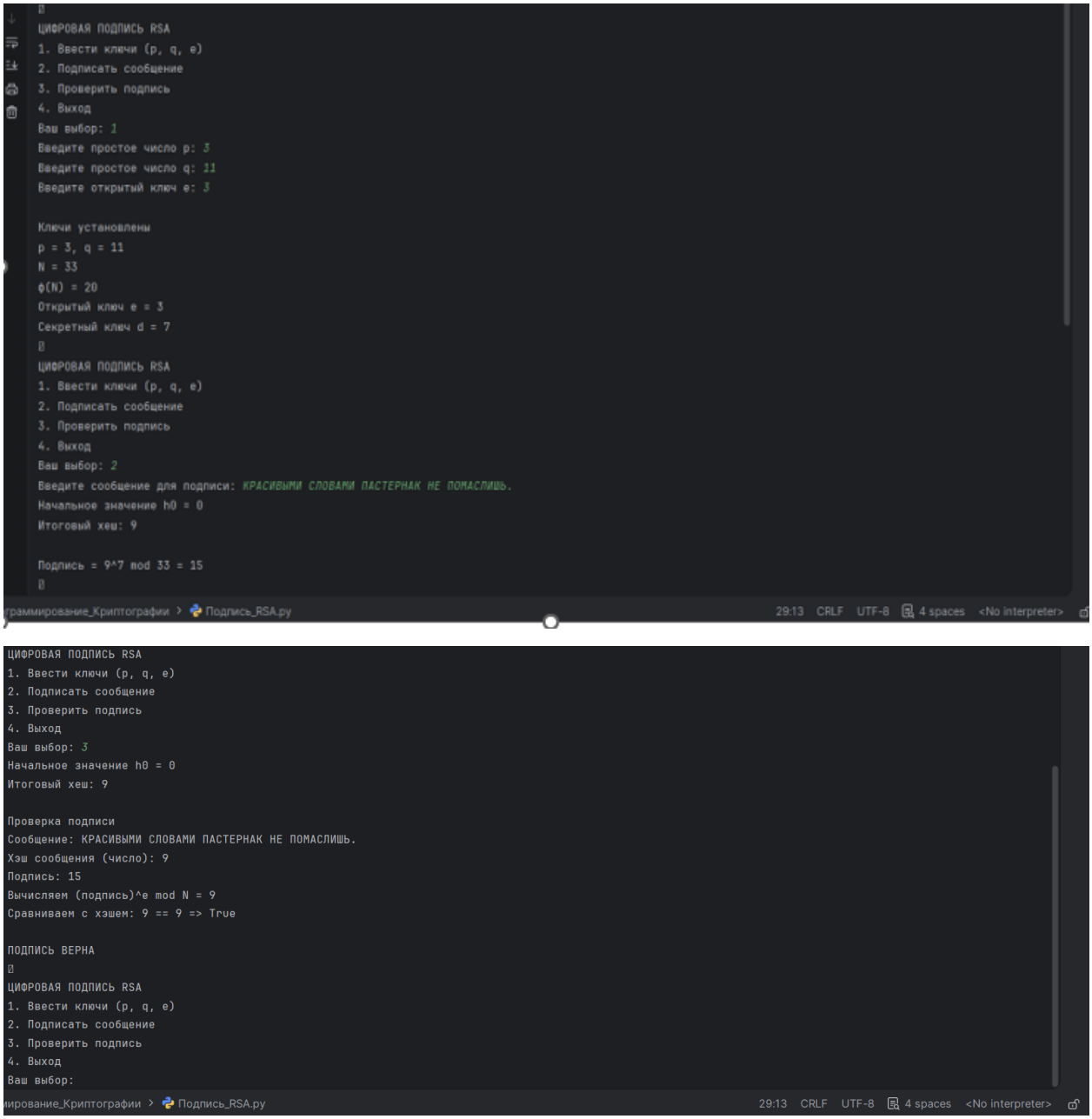


Рисунок 38. Результат подписи RSA.

Результат подписи в тетради:

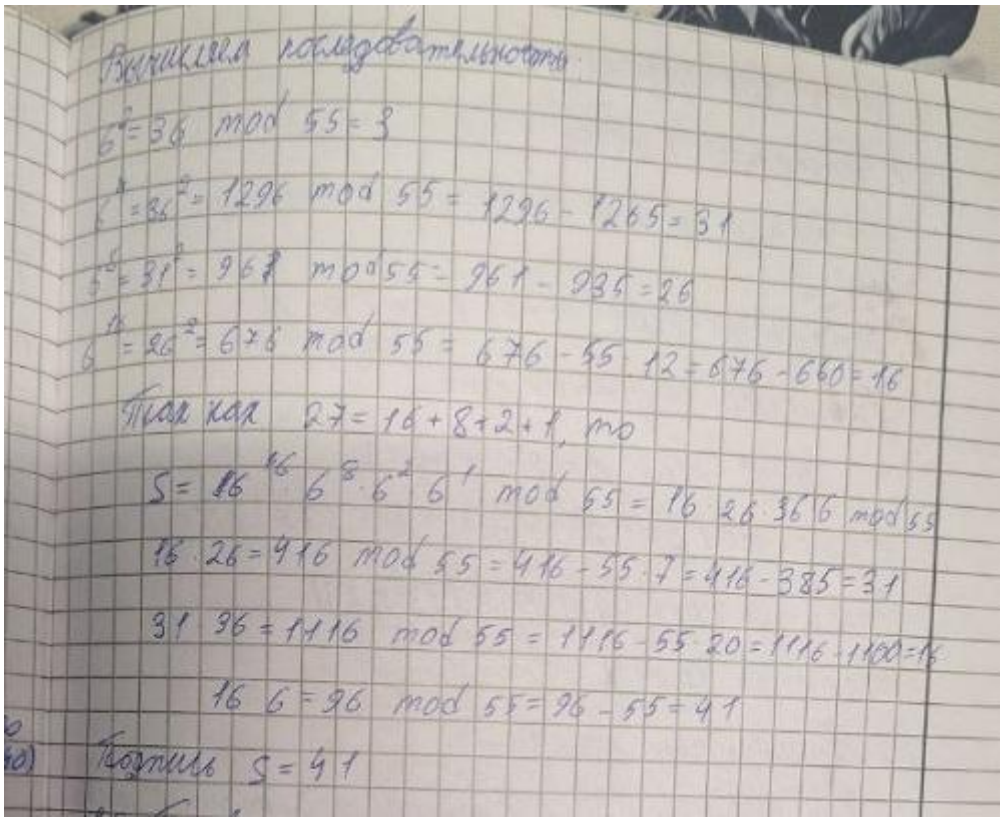


Рисунок 39. Работа в тетради подписи RSA.

Блок J: Стандарты цифровых подписей

26. Цифровая подпись ГОСТ 34.10-94

p - большое простое число длиной от 509 до 512 бит либо от 1020 до 1024 бит;

q - простой сомножитель числа (p -1), имеющий длину 254...256 бит;

a - любое число, большее 1 и меньшее (p-1), причем такое, что aq mod p=1; x - некоторое число, меньшее q;

y = ax mod p.

Кроме того, этот алгоритм использует однонаправленную хэш-функцию H(x). Стандарт ГОСТ Р 34.11-94 определяет хэш-функцию, основанную на использовании стандартного симметричного алгоритма ГОСТ 28147-89. Первые три параметра p, q и a являются открытыми и могут быть общими для всех пользователей сети. Число x является секретным ключом. Число y является открытым ключом. Чтобы подписать некоторое сообщение m, а затем проверить подпись, выполняются следующие шаги.

Блок-схема вычисления хеш-кода сообщения:

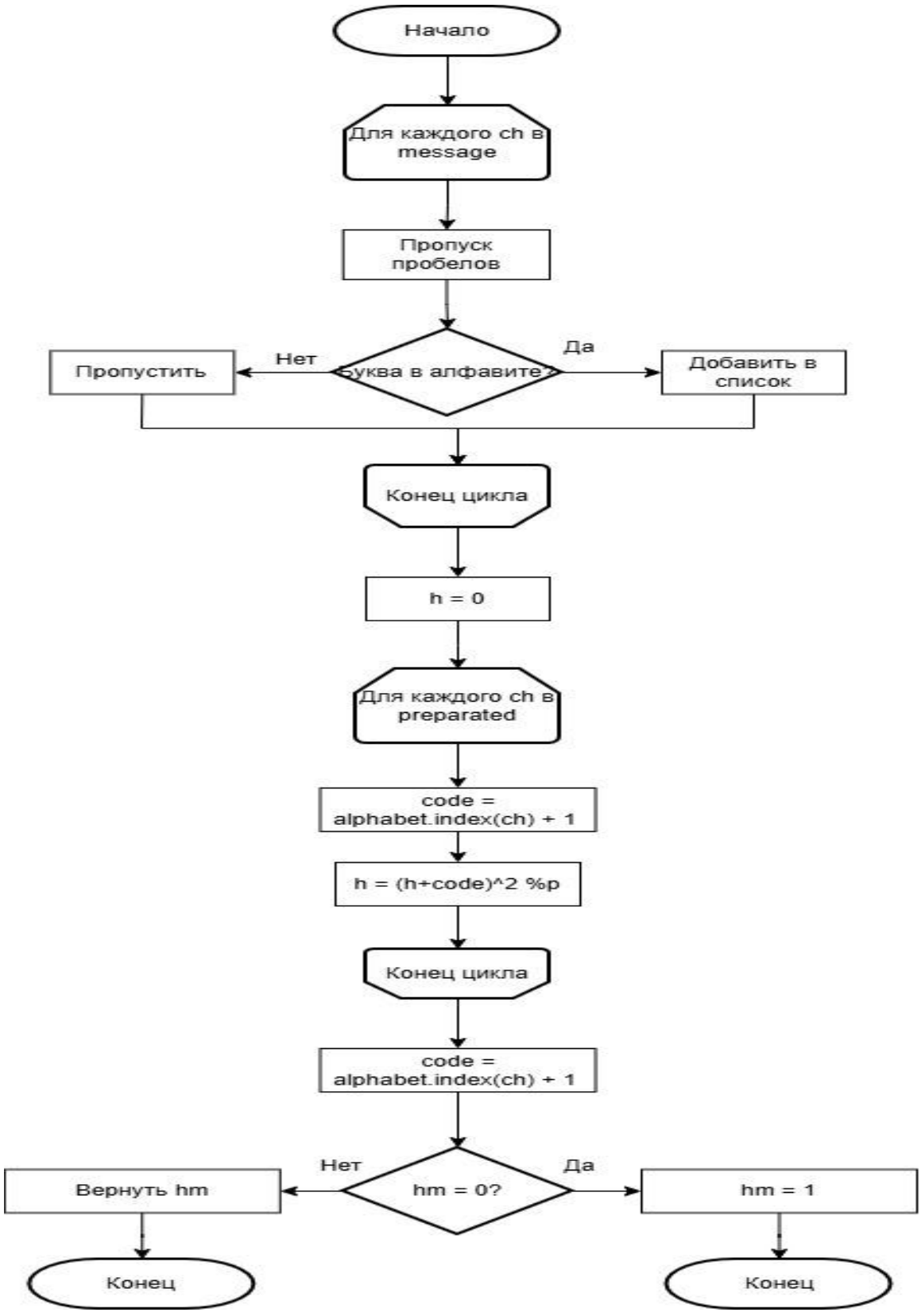


Рисунок 40. Блок-схема вычисления хеш-кода сообщения для ГОСТ Р 34.10-94.

Блок-схема:

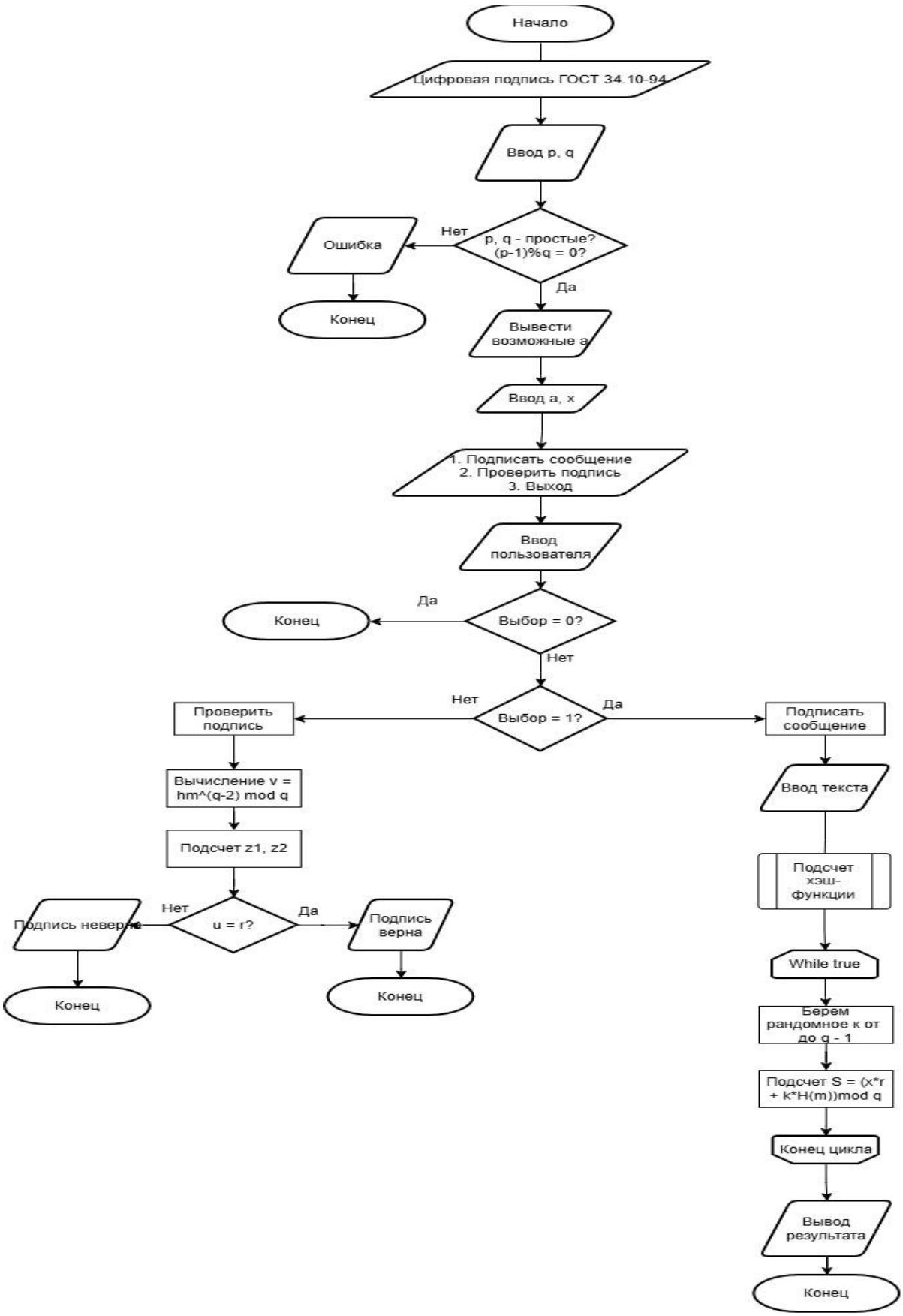


Рисунок 41. Блок-схема подписи ГОСТ 34.10-94.

Код программы:

```
import random
import sys
# ---------- Тест Миллера–Рабина ----------
def is_prime(n: int, k: int = 10) -> bool:
    if n < 2:
        return False
    if n in (2, 3):
        return True
    if n % 2 == 0:
        return False
    r, d = 0, n - 1
    while d % 2 == 0:
        r += 1
        d //= 2
    for _ in range(k):
        a = random.randint(2, n - 2)
        x = pow(a, d, n)
        if x == 1 or x == n - 1:
            continue
        for _ in range(r - 1):
            x = pow(x, 2, n)
            if x == n - 1:
                break
        else:
            return False
    return True

# ---------- Хэш-функция квадратичной свертки ----------
def square_hash(message: str, p: int, q: int) -> int:
    alphabet = "АБВГДЕЖЗИЙКЛМНОПРСТУФХЦЧШЩЪЫЬЭЮЯ"
    prepared = []
    for ch in message.upper():
        if ch == ' ':
            continue
        elif ch == '.':
            prepared.extend(['Т', 'Ч', 'К'])
        else:
            prepared.append(ch)
    h = 0
    for ch in prepared:
        if ch not in alphabet:
            continue
        code = alphabet.index(ch) + 1
        h = (h + code) ** 2 % p
    hm = h % q
    if hm == 0:
        hm = 1
    return hm

# ---------- Генерация всех возможных a для заданных p и q ----------
def generate_possible_a(p: int, q: int) -> list:
    result = []
    for a in range(2, p - 1):
        if pow(a, q, p) == 1:
            result.append(a)
    return result

# ---------- Класс ГОСТ ----------
class Gost341094:
    def __init__(self):
        self.p = None
        self.q = None
        self.a = None
        self.x = None
        self.y = None
        self.params_ready = False
        self.last_message = ""
        self.last_signature = (None, None)
        self.last_hash = None
    def set_params(self, p: int, q: int, a: int, x: int):
        """Устанавливает параметры после проверки."""
        if p <= 32:
            raise ValueError(f"p = {p} должно быть больше 32")
        if not is_prime(p):
            raise ValueError(f"p = {p} не является простым числом")
        if not is_prime(q):
            raise ValueError(f"q = {q} не является простым числом")
```

```
        if (p - 1) % q != 0:
            raise ValueError(f"q = {q} не делит (p-1) = {p-1}")
        if not (1 < a < p - 1):
            raise ValueError(f"a = {a} не в диапазоне 1 < a < p-1")
        if pow(a, q, p) != 1:
            raise ValueError(f"a^{q} mod {p} = {pow(a, q, p)} ≠ 1")
        if not (0 < x < q):
            raise ValueError(f"x = {x} не в диапазоне 0 < x < q")
        self.p = p
        self.q = q
        self.a = a
        self.x = x
        self.y = pow(a, x, p)
        self.params_ready = True
        print("\nПараметры приняты.")
        print(f"p = {p}")
        print(f"q = {q}")
        print(f"a = {a}")
        print(f"Секретный ключ x = {x}")
        print(f"Открытый ключ y = a^x mod p = {self.y}")
    def sign(self, message: str):
        if not self.params_ready:
            raise RuntimeError("Сначала установите параметры.")
        q = self.q
        p = self.p
        hm = square_hash(message, p, q)
        self.last_hash = hm
        print(f"\nХэш сообщения (H(m) mod q) = {hm}")
        while True:
            k = random.randint(1, q - 1)
            r = pow(self.a, k, self.p) % q
            if r == 0:
                continue
            s = (self.x * r + k * hm) % q
            break
        self.last_message = message
        self.last_signature = (r, s)
        print(f"\nПодпись для \"{message}\": r = {r}, s = {s}")
        return (r, s)
    def verify(self) -> bool:
        if not self.params_ready:
            raise RuntimeError("Параметры не установлены.")
        if self.last_signature == (None, None):
            raise RuntimeError("Нет подписанного сообщения.")
        r, s = self.last_signature
        message = self.last_message
        p, q, a, y = self.p, self.q, self.a, self.y
        hm = square_hash(message, p, q)
        print(f"\nХэш сообщения (H(m) mod q) = {hm}")
        v = pow(hm, q - 2, q)
        z1 = (s * v) % q
        z2 = ((q - r) % q) * v % q
        u = (pow(a, z1, p) * pow(y, z2, p)) % p % q
        print(f"v = {v}, z1 = {z1}, z2 = {z2}, u = {u}")
        if u == r:
            print("u == r => ПОДПИСЬ ВЕРНА")
            return True
        else:
            print("u != r => ПОДПИСЬ НЕВЕРНА")
            return False
def input_int(prompt: str) -> int:
    while True:
        try:
            return int(input(prompt))
        except ValueError:
            print("Ошибка: введите целое число.")
# ---------- Главное меню ----------
def main():
    gost = Gost341094()
    while True:
        print("ЦИФРОВАЯ ПОДПИСЬ ГОСТ Р 34.10-94")
        print("1. Установить параметры (p, q, a, x)")
        print("2. Подписать сообщение")
        print("3. Проверить подпись")
        print("4. Выход")
        choice = input("Ваш выбор: ").strip()
```

```
        if choice == '1':
            try:
                # Ввод p
                p = input_int("Введите простое p (p > 32): ")
                # Ввод q
                q = input_int("Введите простое q (делитель p-1): ")
                # Предварительные проверки (без a)
                if p <= 32:
                    raise ValueError("p должно быть > 32")
                if not is_prime(p):
                    raise ValueError("p не простое")
                if not is_prime(q):
                    raise ValueError("q не простое")
                if (p - 1) % q != 0:
                    raise ValueError("q не делит p-1")
                # Генерация возможных a
                possible_a = generate_possible_a(p, q)
                if not possible_a:
                    raise ValueError("Нет подходящих a для данных p и q.")
                print(f"\nНайдено {len(possible_a)} возможных значений a:")
                print(possible_a)
                print("\nВыберите одно из них (введите число).")
                a = input_int("a = ")
                if a not in possible_a:
                    raise ValueError(f"a = {a} не подходит: a^q mod p != 1.")
                # Ввод секретного ключа x
                x = input_int("Введите секретный ключ x (0 < x < q): ")
                # Установка параметров
                gost.set_params(p, q, a, x)
            except Exception as e:
                print(f"Ошибка: {e}")
        elif choice == '2':
            if not gost.params_ready:
                print("Сначала установите параметры (пункт 1).")
            else:
                msg = input("Введите сообщение для подписи: ")
                try:
                    gost.sign(msg)
                except Exception as e:
                    print(f"Ошибка: {e}")
        elif choice == '3':
            if not gost.params_ready:
                print("Сначала установите параметры (пункт 1).")
            elif gost.last_signature == (None, None):
                print("Сначала подпишите сообщение (пункт 2).")
            else:
                try:
                    gost.verify()
                except Exception as e:
                    print(f"Ошибка: {e}")
        elif choice == '4':
            print("Программа завершена.")
            break
        else:
            print("Неверный выбор.")
if __name__ == "__main__":
    main()
```

Результат работы программы:

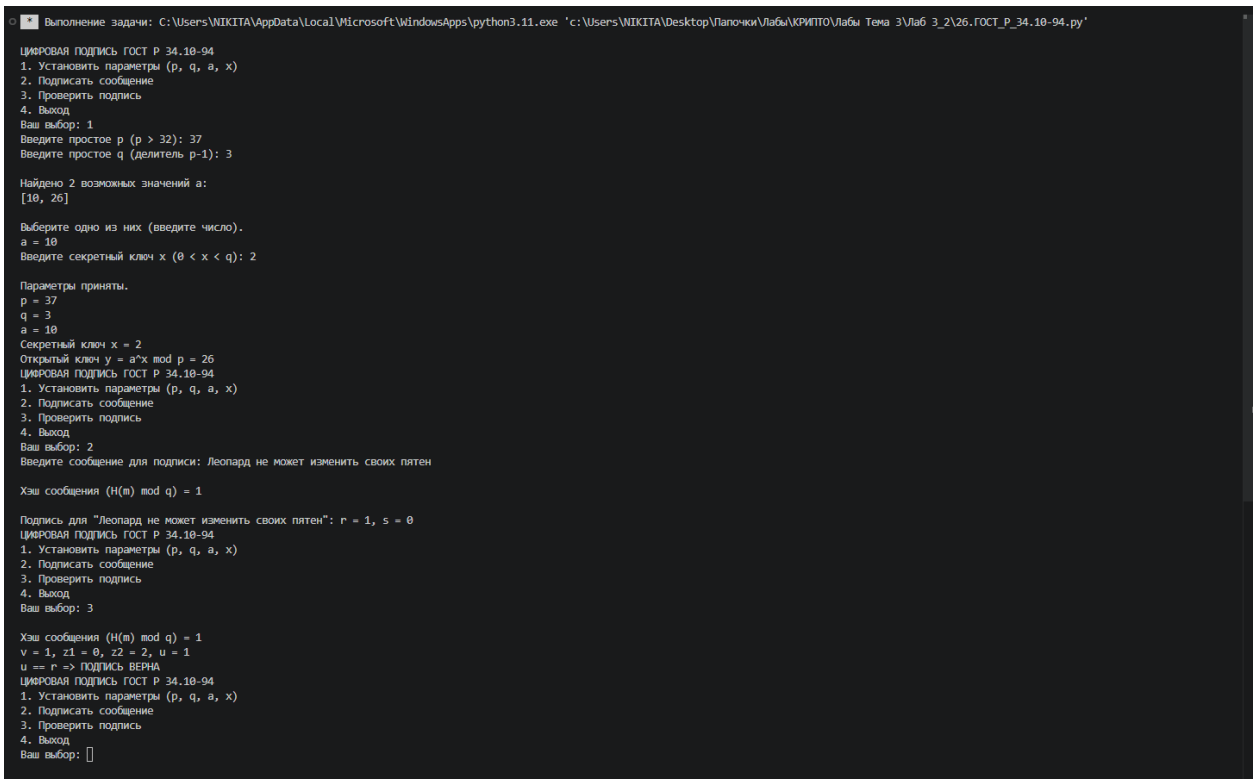
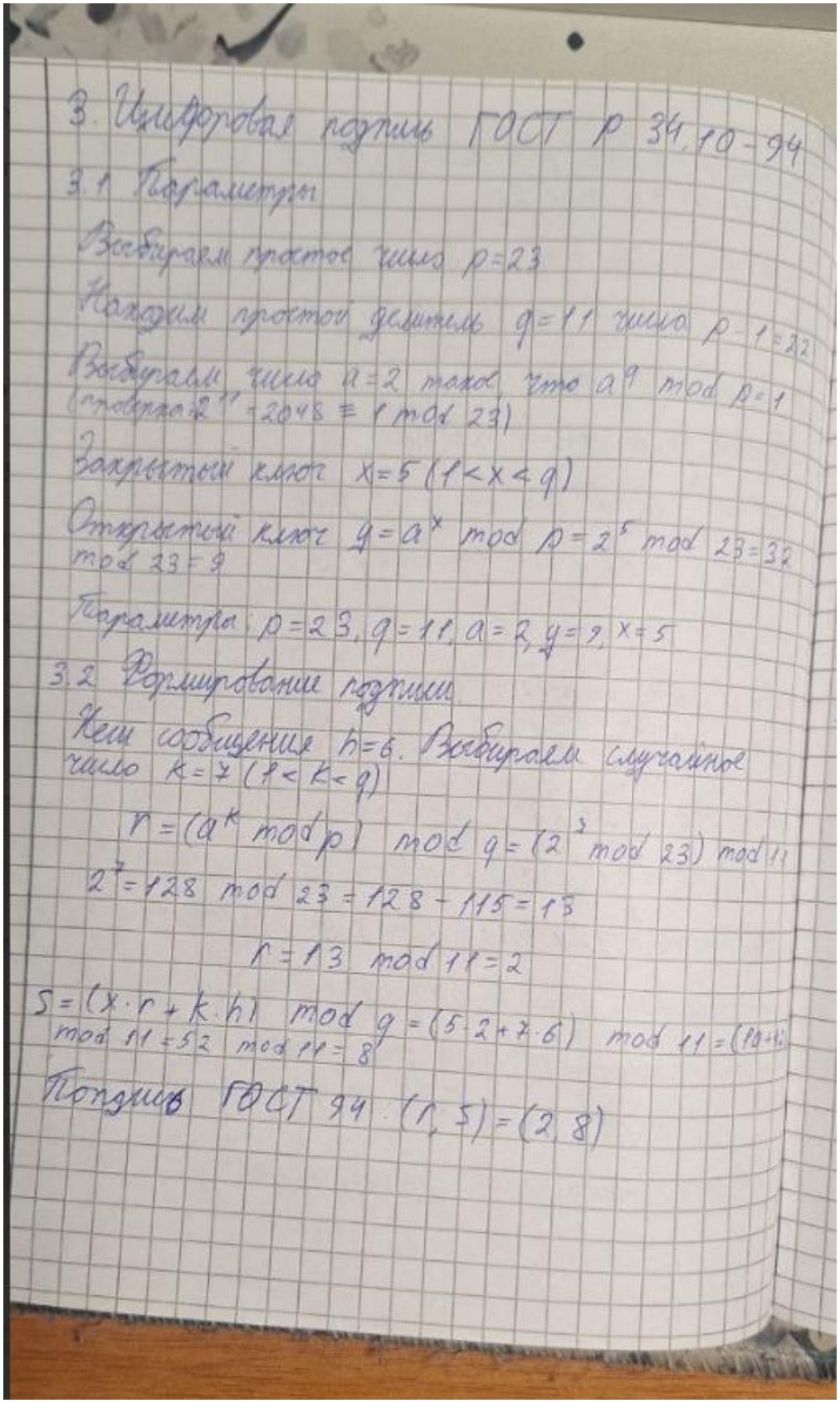


Рисунок 42. Результат цифровой подписи ГОСТ 34.10-94.

Результат подписи в тетради:



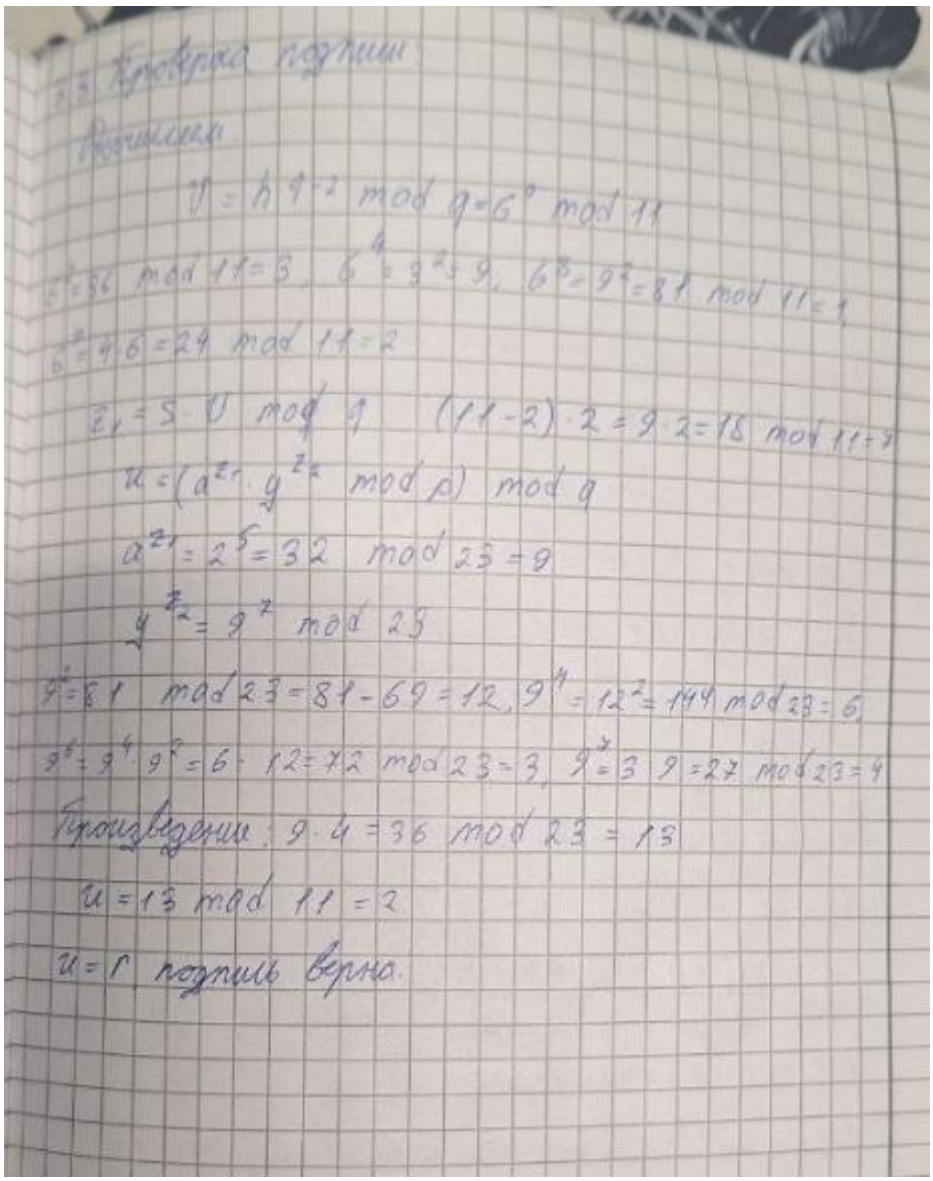


Рисунок 43. Подпись ГОСТ 34.10-94 в тетради.

Блок К: обмен ключами

28. Обмен ключами по алгоритму Диффи-Хеллмана

Протокол Диффи-Хеллмана — это криптографический протокол, позволяющий двум и более сторонам получить общий секретный ключ по незащищённому каналу связи. Протокол был предложен Уитфилдом Диффи и Мартином Хеллманом в 1976 году и стал первым практическим методом асимметричного распределения ключей.

Ключевая особенность: Протокол не выполняет шифрование напрямую, а позволяет сторонам договориться о секретном ключе, который затем можно использовать для симметричного шифрования. Алгоритм основан на вычислительной сложности дискретного логарифмирования.

Блок-схема алгоритма:

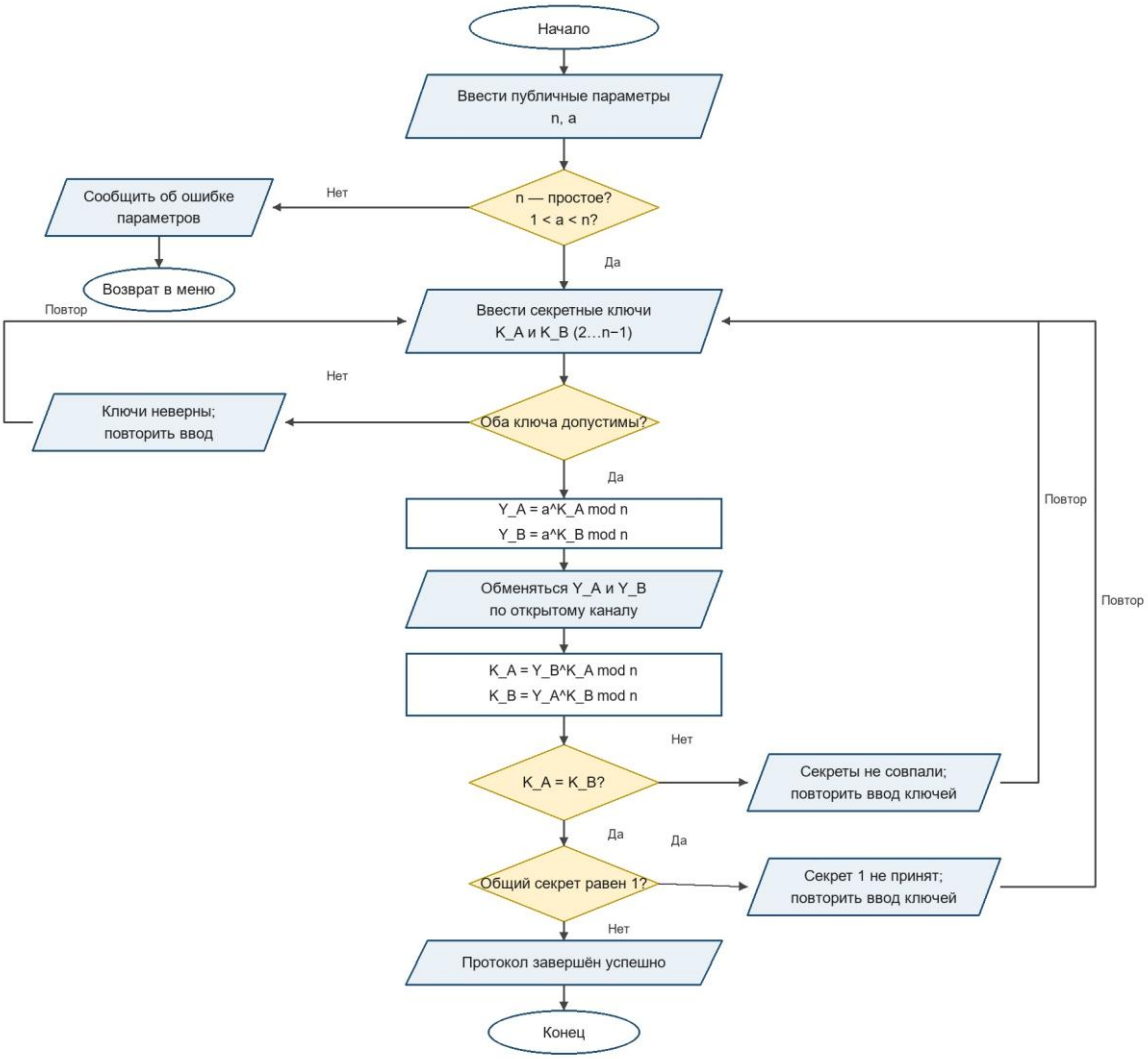


Рисунок 45. Блок-схема алгоритма Диффи-Хеллмана.

Код программы:

```
# ============================================================
# БЛОК K: ОБМЕН КЛЮЧАМИ
# Лаб 2_2 (задание 28) — Обмен ключами по Диффи-Хеллману
# Источник: ЛАБКРИПТ_2022, стр. 40
# ============================================================
#
# АЛГОРИТМ (по методичке):
#   Публичные параметры: n — большое простое, a (1 < a < n)
#   Секретные ключи:     K_A, K_B — случайные числа из [2, n-1]
#   Открытые ключи:      Y_A = a^K_A mod n,  Y_B = a^K_B mod n
#   Обмен:               Y_A и Y_B передаются по открытому каналу
#   Общий секрет:        K_A: K = Y_B^K_A mod n
#                        K_B: K = Y_A^K_B mod n
#   Проверка:            K_A == K_B (оба должны совпадать)
#
# ДОКАЗАТЕЛЬСТВО:
#   A: Y_B^K_A mod n = (a^K_B)^K_A mod n = a^(K_B*K_A) mod n = K
#   B: Y_A^K_B mod n = (a^K_A)^K_B mod n = a^(K_A*K_B) mod n = K
# ============================================================
import random
import sys

# ──────────────────────────────────────────────
#  Математические вспомогательные функции
# ──────────────────────────────────────────────
def mod_pow(base, exp, mod):
    """Быстрое возведение в степень по модулю (алгоритм «двоичного возведения»)."""
    result = 1
    base %= mod
    while exp > 0:
        if exp & 1:
            result = (result * base) % mod
        base = (base * base) % mod
        exp >>= 1
    return result

def is_prime(n):
    """Детерминированная проверка простоты методом пробных делений."""
    if n < 2:
        return False
    if n == 2:
        return True
    if n % 2 == 0:
        return False
    i = 3
    while i * i <= n:
        if n % i == 0:
            return False
        i += 2
    return True

def miller_rabin(n, k=10):
    """Тест Миллера–Рабина для больших чисел (вероятностный, k раундов)."""
    if n < 2:
        return False
    if n == 2 or n == 3:
        return True
    if n % 2 == 0:
        return False
    # Представляем n-1 = 2^r * d
    r, d = 0, n - 1
    while d % 2 == 0:
        r += 1
        d //= 2
    for _ in range(k):
        a = random.randrange(2, n - 1)
        x = mod_pow(a, d, n)
        if x == 1 or x == n - 1:
            continue
        for _ in range(r - 1):
            x = (x * x) % n
            if x == n - 1:
                break
        else:
            return False
    return True

def generate_prime(bits=512):
```

```
    """Генерация случайного простого числа заданной битности (тест Миллера–Рабина)."""
    while True:
        candidate = random.getrandbits(bits) | (1 << (bits - 1)) | 1  # нечётное, старший бит =
1
        if miller_rabin(candidate):
            return candidate

def find_primitive_root(p):
    """
    Находит первообразный корень (примитивный корень) по модулю простого p.
    Для простого p φ(p) = p-1. Перебираем кандидатов от 2.
    """
    phi = p - 1
    # Факторизация phi
    factors = set()
    temp = phi
    d = 2
    while d * d <= temp:
        if temp % d == 0:
            factors.add(d)
            while temp % d == 0:
                temp //= d
        d += 1
    if temp > 1:
        factors.add(temp)
    # Проверяем кандидатов
    for g in range(2, p):
        ok = True
        for factor in factors:
            if mod_pow(g, phi // factor, p) == 1:
                ok = False
                break
        if ok:
            return g
    return -1  # не найден (не должно случиться для простого p)

# ──────────────────────────────────────────────
#  Класс протокола Диффи-Хеллмана
# ──────────────────────────────────────────────
class DiffieHellman:
    """
    Протокол обмена ключами Диффи–Хеллмана.
    Публичные параметры:  n (простое), a (1 < a < n)
    Секретные ключи:      K_A, K_B ∈ [2, n-1]
    Открытые ключи:       Y_A = a^K_A mod n,  Y_B = a^K_B mod n
    Общий секрет:         K = Y_B^K_A mod n  =  Y_A^K_B mod n
    """
    def __init__(self):
        self.n = None      # публичный модуль (большое простое)
        self.a = None      # публичное основание a
        self.K_A = None    # секретный ключ пользователя A
        self.K_B = None    # секретный ключ виртуального пользователя B
        self.Y_A = None    # открытый ключ A
        self.Y_B = None    # открытый ключ B
        self.shared_K_A = None   # общий секрет, вычисленный стороной A
        self.shared_K_B = None   # общий секрет, вычисленный стороной B
    # ── Установка публичных параметров ──────────────────────
    def set_params(self, n, a):
        """Устанавливает публичные параметры n и a с проверками."""
        # Не используем пробное деление для больших n: проверяем простоту
        # тестом Миллера–Рабина с достаточным числом раундов.
        if not miller_rabin(n, k=32):
            print("  Ошибка: n должно быть простым числом!")
            return False
        if not (1 < a < n):
            print(f"  Ошибка: a должно быть в диапазоне (1 < a < n={n})!")
            return False
        self.n = n
        self.a = a
        # Сброс предыдущих ключей
        self.K_A = self.K_B = self.Y_A = self.Y_B = None
        self.shared_K_A = self.shared_K_B = None
        print(f"\n  Публичные параметры установлены:")
        print(f"    n = {n}")
        print(f"    a = {a}  (основание степени)")
        return True
    def auto_generate_params(self, bits=24):
        """Генерирует учебные параметры; подбор корня ограничен 32 битами."""
```

```
            if not 4 <= bits <= 32:
                print("  Ошибка: учебная автогенерация поддерживает битность от 4 до 32.")
                return False
            print(f"\n  Генерация простого числа ({bits} бит)...")
            n = generate_prime(bits)
            print(f"  Найдено простое n = {n}")
            print(f"  Поиск примитивного корня по модулю n...")
            a = find_primitive_root(n)
            if a == -1:
                print("  Ошибка: не удалось найти примитивный корень.")
                return False
            print(f"  Найден примитивный корень a = {a}")
            self.n = n
            self.a = a
            self.K_A = self.K_B = self.Y_A = self.Y_B = None
            self.shared_K_A = self.shared_K_B = None
            return True

        # ── Шаг 1: Выбор секретных ключей ──────────────────────────
        def set_secret_key_A(self, K_A):
            """Задаёт секретный ключ пользователя A."""
            if self.n is None:
                print("  Ошибка: сначала установите публичные параметры!")
                return False
            if not (2 <= K_A <= self.n - 1):
                print(f"  Ошибка: K_A должно быть в диапазоне [2, n-1 = {self.n - 1}]!")
                return False
            self.K_A = K_A
            self.Y_A = self.Y_B = None
            self.shared_K_A = self.shared_K_B = None
            print(f"  Секретный ключ A: K_A = {K_A}")
            return True

        def auto_secret_key_B(self):
            """Виртуальный пользователь B сам выбирает случайный секретный ключ."""
            if self.n is None:
                print("  Ошибка: сначала установите публичные параметры!")
                return False
            self.K_B = random.randint(2, self.n - 1)
            self.Y_A = self.Y_B = None
            self.shared_K_A = self.shared_K_B = None
            print(f"  Виртуальный пользователь B выбрал: K_B = {self.K_B}  (СЕКРЕТ — известен только
B)")
            return True

        def set_secret_key_B(self, K_B):
            """Задаёт секретный ключ пользователя B, введённый вручную."""
            if self.n is None:
                print("  Ошибка: сначала установите публичные параметры!")
                return False
            if not (2 <= K_B <= self.n - 1):
                print(f"  Ошибка: K_B должно быть в диапазоне [2, n-1 = {self.n - 1}]!")
                return False
            self.K_B = K_B
            self.Y_A = self.Y_B = None
            self.shared_K_A = self.shared_K_B = None
            print(f"  Секретный ключ B: K_B = {K_B}")
            return True

        # ── Шаг 2: Вычисление открытых ключей ──────────────────────
        def compute_public_keys(self):
            """
            Шаг 2 алгоритма:
              Y_A = a^K_A mod n
              Y_B = a^K_B mod n
            """
            if None in (self.n, self.a, self.K_A, self.K_B):
                print("  Ошибка: не все параметры установлены!")
                return False
            self.Y_A = mod_pow(self.a, self.K_A, self.n)
            self.Y_B = mod_pow(self.a, self.K_B, self.n)
            print(f"\n  Открытые ключи (передаются по открытому каналу):")
            print(f"    Y_A = a^K_A mod n = {self.a}^{self.K_A} mod {self.n} = {self.Y_A}")
            print(f"    Y_B = a^K_B mod n = {self.a}^{self.K_B} mod {self.n} = {self.Y_B}")
            return True

        # ── Шаг 3: Вычисление общего секрета ───────────────────────
        def compute_shared_secret(self):
            """
            Шаг 3 алгоритма:
              K_A: K = Y_B^K_A mod n
              K_B: K = Y_A^K_B mod n
```

```
          Проверка: вычисленные сторонами значения K совпадают и не равны 1
        """
        if None in (self.Y_A, self.Y_B, self.K_A, self.K_B):
            print("  Ошибка: сначала вычислите открытые ключи!")
            return False
        self.shared_K_A = mod_pow(self.Y_B, self.K_A, self.n)
        self.shared_K_B = mod_pow(self.Y_A, self.K_B, self.n)
        print(f"\n  Вычисление общего секрета:")
        print(f"   A: K = Y_B^K_A mod n = {self.Y_B}^{self.K_A} mod {self.n} = {self.shared_K_A}")
        print(f"   B: K = Y_A^K_B mod n = {self.Y_A}^{self.K_B} mod {self.n} = {self.shared_K_B}")
        if self.shared_K_A == self.shared_K_B == 1:
            print("\n  ✗ ОШИБКА: общий секрет равен 1 и считается недопустимым.")
            print("    Выберите другие секретные ключи A и B.")
            self.shared_K_A = self.shared_K_B = None
            return False
        if self.shared_K_A == self.shared_K_B:
            print(f"\n  ✓ ПРОВЕРКА ПРОЙДЕНА: K у A = K у B = {self.shared_K_A}")
            print(f"    Общий секретный ключ успешно выработан!")
        else:
            print(f"\n  ✗ ОШИБКА: K у A ≠ K у B  ({self.shared_K_A} ≠ {self.shared_K_B})")
        return self.shared_K_A == self.shared_K_B

    # ── Полная демонстрация протокола ──────────────────────
    def full_demo(self):
        """Полная пошаговая демонстрация протокола Диффи-Хеллмана."""
        sep = "─" * 60
        print(f"\n{sep}")
        print("  ПРОТОКОЛ ОБМЕНА КЛЮЧАМИ ДИФФИ-ХЕЛЛМАНА")
        print(f"  Источник: ЛАБКРИПТ_2022, Блок K, Задание 28")
        print(sep)
        print("\n  Шаг 1. Публичные параметры (известны всем):")
        print(f"    n = {self.n}  (простое число)")
        print(f"    a = {self.a}  (1 < a < n)")
        print("\n  Шаг 2. Выбор секретных ключей:")
        print(f"    Пользователь A выбирает K_A = {self.K_A}  (из интервала [2, n-1])")
        print(f"    Пользователь B выбирает K_B = {self.K_B}  (из интервала [2, n-1])")
        print("\n  Шаг 3. Вычисление открытых ключей:")
        self.compute_public_keys()
        print("\n  Шаг 4. Обмен открытыми ключами по открытому каналу:")
        print(f"    A → B: Y_A = {self.Y_A}")
        print(f"    B → A: Y_B = {self.Y_B}")
        print("\n  Шаг 5. Независимое вычисление общего секрета:")
        ok = self.compute_shared_secret()
        print(f"\n{sep}")
        if ok:
            print(f"  ИТОГ: Общий секретный ключ K = {self.shared_K_A}")
        else:
            print("  ИТОГ: Ошибка в протоколе!")
        print(sep)
        return ok

# ═══════════════════════════════════════════════
#  Меню
# ═══════════════════════════════════════════════
def print_menu(dh):
    """Выводит текущее меню с состоянием."""
    print("\n" + "=" * 60)
    print("  БЛОК K: ОБМЕН КЛЮЧАМИ (Задание 28 — Диффи-Хеллман)")
    print("=" * 60)
    # Текущее состояние параметров
    if dh.n:
        print(f"  n = {dh.n}  |  a = {dh.a}")
    else:
        print("  Параметры не установлены")
    if dh.K_A:
        print(f"  K_A = {dh.K_A}  |  K_B = {'установлен' if dh.K_B else 'не установлен'}")
    if dh.shared_K_A:
        print(f"  Общий ключ K = {dh.shared_K_A}")
    print()
    print("  1 — Ввести публичные параметры (n, a) вручную")
    print("  2 — Авто-генерация параметров (простое n и генератор a)")
    print("  3 — Ввести секретный ключ A (K_A) вручную")
    print("  4 — Ввести секретный ключ B (K_B) вручную")
    print("  5 — Вычислить открытые ключи Y_A и Y_B")
    print("  6 — Вычислить общий секрет и проверить протокол")
    print("  7 — Пошаговый обмен (параметры, ключи, проверка)")
    print("  8 — Пример из методички (малые числа)")
```

```
        print("  0 — Выход")
        print()

    def demo_textbook(dh):
        """
        Демонстрация с малыми числами по образцу методички.
        Используем простое n=23, генератор a=5 — классический учебный пример.
        """
        print("\n" + "—" * 60)
        print("  ПРИМЕР ПО МЕТОДИЧКЕ (малые числа)")
        print("—" * 60)
        print("  Публичные параметры: n = 23 (простое), a = 5")
        print("  Пользователь A выбирает K_A = 6")
        print("  Пользователь B выбирает K_B = 15")
        dh.n = 23
        dh.a = 5
        dh.K_A = 6
        dh.K_B = 15
        dh.Y_A = None
        dh.Y_B = None
        dh.shared_K_A = None
        dh.shared_K_B = None
        print("\n  Открытые ключи:")
        dh.compute_public_keys()
        print("\n  Общий секрет:")
        dh.compute_shared_secret()

    def manual_protocol(dh):
        """Последовательно запрашивает параметры и ключи, затем проверяет обмен."""
        print("\n" + "—" * 60)
        print("  ПОШАГОВЫЙ ОБМЕН ДИФФИ-ХЕЛЛМАНА")
        print("—" * 60)
        print("\n1. Введите публичные параметры")
        try:
            n = int(input("  Простое число n: "))
            a = int(input("  Основание a (1 < a < n): "))
        except ValueError:
            print("  Ошибка: введите целые числа.")
            return False
        if not dh.set_params(n, a):
            return False
        while True:
            print("\n2. Введите секретные ключи A и B")
            try:
                K_A = int(input(f"  Секретный ключ A (2..{n - 1}): "))
                K_B = int(input(f"  Секретный ключ B (2..{n - 1}): "))
            except ValueError:
                print("  Ошибка: введите целые числа; повторите ввод ключей.")
                continue
            if not dh.set_secret_key_A(K_A) or not dh.set_secret_key_B(K_B):
                print("  Повторите ввод секретных ключей.")
                continue
            print("\n3. Вычислите открытые ключи")
            if not dh.compute_public_keys():
                return False
            print("\n4. Вычислите общий секрет и проверьте протокол")
            if dh.compute_shared_secret():
                print("\n  Протокол успешно завершён.")
                return True
            print("\n  Обмен не принят. Введите другие секретные ключи A и B.")

    def main():
        dh = DiffieHellman()
        while True:
            print_menu(dh)
            choice = input("  Ваш выбор: ").strip()
            if choice == "1":
                try:
                    n = int(input("  Введите простое число n: "))
                    a = int(input("  Введите основание a (1 < a < n): "))
                    dh.set_params(n, a)
                except ValueError:
                    print("  Ошибка: введите целые числа.")
            elif choice == "2":
                try:
                    bits = int(input("  Битность n для учебной автогенерации (4..32) [24]: ").strip()
or "24")
                    dh.auto_generate_params(bits)
```

```
                except ValueError:
                    print("  Ошибка: введите целое число.")
            elif choice == "3":
                if dh.n is None:
                    print("  Ошибка: сначала установите параметры (п.1 или п.2).")
                else:
                    try:
                        K_A = int(input(f"  Введите секретный ключ K_A (из [2, {dh.n - 1}]): "))
                        dh.set_secret_key_A(K_A)
                    except ValueError:
                        print("  Ошибка: введите целое число.")
            elif choice == "4":
                if dh.n is None:
                    print("  Ошибка: сначала установите параметры (п.1 или п.2).")
                else:
                    try:
                        K_B = int(input(f"  Введите секретный ключ K_B (из [2, {dh.n - 1}]): "))
                        dh.set_secret_key_B(K_B)
                    except ValueError:
                        print("  Ошибка: введите целое число.")
            elif choice == "5":
                dh.compute_public_keys()
            elif choice == "6":
                dh.compute_shared_secret()
            elif choice == "7":
                manual_protocol(dh)
            elif choice == "8":
                demo_textbook(dh)
            elif choice == "0":
                print("  Выход.")
                break
            else:
                print("  Неверный выбор.")
            input("\n  Нажмите Enter для продолжения...")
    if __name__ == "__main__":
        main()
```

Результат работы программы:

```
ОБМЕН КЛЮЧАМИ ПО АЛГОРИТМУ ДИФФИ-ХЕЛЛМАНА
1. Ввести параметры и ключи (p, g, a, b)
2. Показать текущие данные
3. Выход
Ваш выбор: 1

Введите общие параметры и секретные ключи:
p (простое число, например 23): 23
g (основание, 1 < g < p): 5
Подсказка: первообразный корень по модулю 23 — 5
Секретный ключ пользователя A (0 < a < p-1): 7
Секретный ключ пользователя B (0 < b < p-1): 9

=== ВЫЧИСЛЕНИЯ ===
Открытый ключ A: A = g^a mod p = 5^7 mod 23 = 17
Открытый ключ B: B = g^b mod p = 5^9 mod 23 = 11
Общий секретный ключ (сторона A): K = B^a mod p = 11^7 mod 23 = 7
Общий секретный ключ (сторона B): K = A^b mod p = 17^9 mod 23 = 7
```

Рисунок 46. Результат обмена ключами.

Работа в тетради:

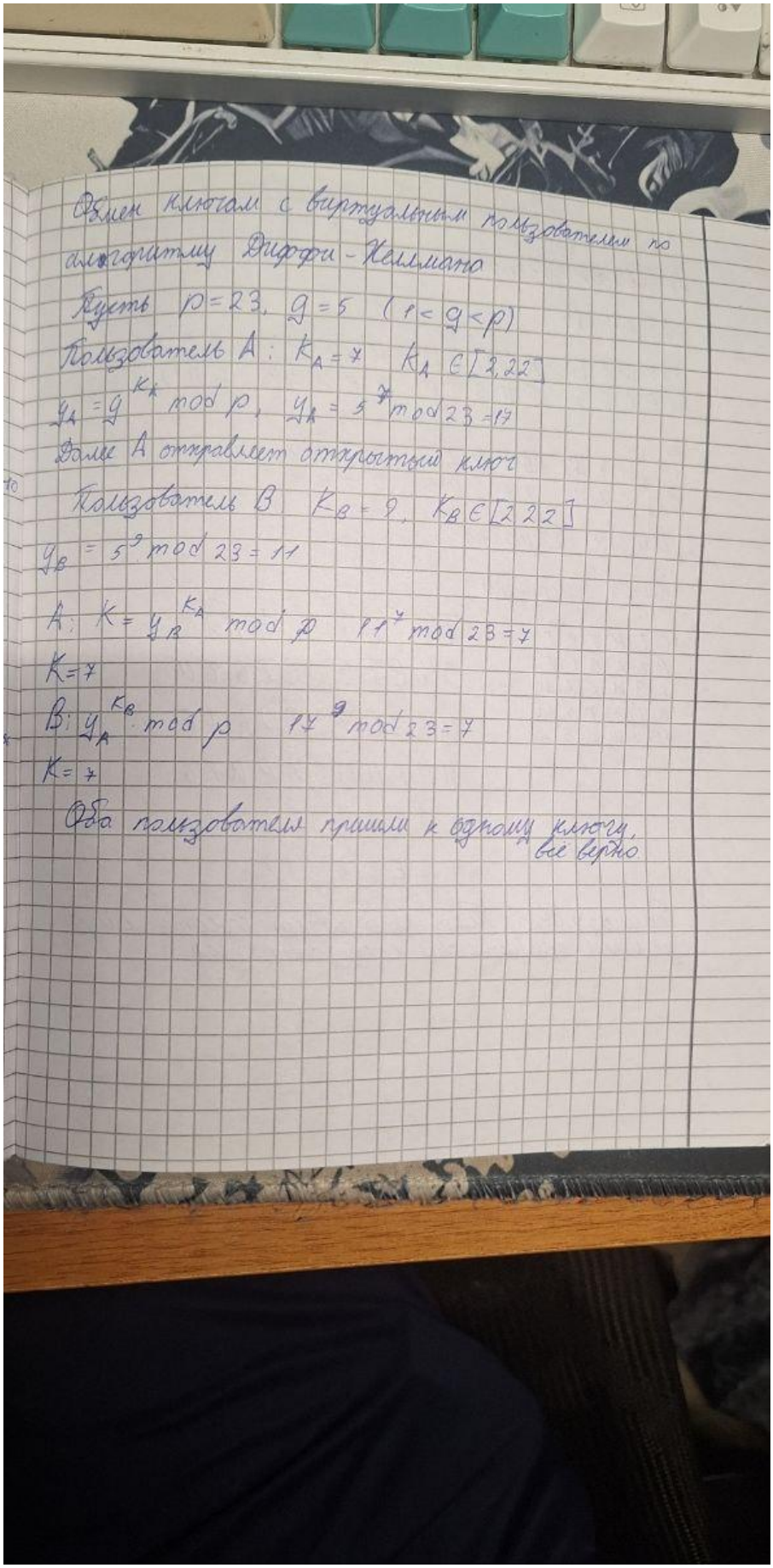


Рисунок 47. Обмен ключами в тетради.

Заключение

В работе рассмотрены криптографические алгоритмы из блоков А–К: от классических шифров до цифровых подписей и обмена ключами. Общий интерфейс позволяет запускать реализации из одного меню, а приведённые схемы, исходный код и результаты выполнения помогают проследить основные этапы каждого алгоритма.

Проверка обратных преобразований, контрольных примеров и результатов протоколов позволяет оценить корректность работы реализованных программ на приведённых входных данных. Актуальные версии алгоритмов МАГМА, Кузнечик и ГОСТ Р 34.10-2012 вынесены в приложение А.

Приложение А. Обновлённый код алгоритмов

В этом приложении приведены актуальные версии программ МАГМА, Кузнечик и ГОСТ Р 34.10-2012. Ввод текста и ключевых данных выполняется пользователем через консольное меню.

17. Шифр МАГМА

Файл программы: 17.MAGMA.py

```
# -*- coding: utf-8 -*-
"""
17. Шифр МАГМА (ГОСТ Р 34.12-2015)
Реализация блочного шифра «Магма» с режимом ECB.
Блок: 64 бита, ключ: 256 бит, 32 раунда (сеть Фейстеля).
Тестирование по ГОСТ Р 34.12-2018 (Приложение А).
Источник: ЛАБКРИПТ_2022, Блок G, задание 17.
ВВОД (как в методичке):
  - Открытый текст: произвольная строка (кодируется в UTF-8)
  - Ключ: 64 шестнадцатеричных символа (256 бит)
АЛГОРИТМ (по ГОСТ Р 34.12-2018 раздел 5):
  Блок a = a1|a0 (a1 — старшие 32 бит, a0 — младшие 32 бит).
  Раунд: G[k](a1,a0) = (a0, g[k](a0) XOR a1)
  Последний раунд: G*[k](a1,a0) = (g[k](a0) XOR a1, a0)
  Ключ K делится на K1..K8 (big-endian, 32 бит каждый).
  Расписание: K1..K8 × 3, затем K8..K1.
"""
# ──────────────────────────────────────────────
#  S-блоки ГОСТ Р 34.12-2015 (Приложение А)
# ──────────────────────────────────────────────
S_BOX = (
    (12, 4, 6, 2, 10, 5, 11, 9, 14, 8, 13, 7, 0, 3, 15, 1),
    (6, 8, 2, 3, 9, 10, 5, 12, 1, 14, 4, 7, 11, 13, 0, 15),
    (11, 3, 5, 8, 2, 15, 10, 13, 14, 1, 7, 4, 12, 9, 6, 0),
    (12, 8, 2, 1, 13, 4, 15, 6, 7, 0, 10, 5, 3, 14, 9, 11),
    (7, 15, 5, 10, 8, 1, 6, 13, 0, 9, 3, 14, 11, 4, 2, 12),
    (5, 13, 15, 6, 9, 2, 12, 10, 11, 7, 8, 1, 4, 3, 14, 0),
    (8, 14, 2, 5, 6, 9, 1, 12, 15, 4, 11, 0, 13, 10, 3, 7),
    (1, 7, 14, 13, 0, 5, 8, 3, 4, 15, 10, 6, 9, 12, 11, 2),
)
# ──────────────────────────────────────────────
#  Нелинейное преобразование T (8 × S-блоков)
# ──────────────────────────────────────────────
def t_direct(a: int) -> int:
    a &= 0xFFFFFFFF
    out = 0
    for i in range(8):
        nibble = (a >> (4 * i)) & 0xF
        out |= S_BOX[i][nibble] << (4 * i)
    return out & 0xFFFFFFFF
# ──────────────────────────────────────────────
#  Циклический сдвиг влево на 11 бит
# ──────────────────────────────────────────────
def rotl32(x: int, n: int) -> int:
    x &= 0xFFFFFFFF
    return ((x << n) | (x >> (32 - n))) & 0xFFFFFFFF
# ──────────────────────────────────────────────
#  Раундовая функция g[k](a) = rotl11(T(a + k mod 2^32))
# ──────────────────────────────────────────────
def g_round(k: int, a: int) -> int:
    x = (a + k) & 0xFFFFFFFF
    x = t_direct(x)
    return rotl32(x, 11)
# ──────────────────────────────────────────────
#  Расписание ключей
#  K = K1|K2|...|K8 (big-endian 32-бит слова)
#  Раунды 1-24: K1,K2,...,K8 повторить 3 раза
#  Раунды 25-32: K8,K7,...,K1
# ──────────────────────────────────────────────
def round_keys_from_key(key32: bytes) -> list:
    if len(key32) != 32:
        raise ValueError("Ключ должен быть 32 байта (256 бит)")
    K = [int.from_bytes(key32[4 * i: 4 * (i + 1)], "big") for i in range(8)]
    rk = []
    for i in range(32):
        if i < 24:
            rk.append(K[i % 8])
        else:
```

```
            rk.append(K[7 - (i - 24)])
    return rk

# ──────────────────────────────────────────────
#  Шифрование одного 64-битного блока (big-endian)
#  a = a1|a0  (a1 — старшие 32 бит, a0 — младшие 32 бит)
#  G[k](a1,a0)  = (a0,  g[k](a0) XOR a1)   — обычный раунд
#  G*[k](a1,a0) = (g[k](a0) XOR a1, a0)    — последний раунд
# ──────────────────────────────────────────────
def encrypt_block_magma(block: bytes, rk: list) -> bytes:
    a1 = int.from_bytes(block[0:4], "big")
    a0 = int.from_bytes(block[4:8], "big")
    for i in range(31):
        # G[rk[i]](a1, a0)
        a1, a0 = a0, (g_round(rk[i], a0) ^ a1) & 0xFFFFFFFF
    # G*[rk[31]](a1, a0) — без перестановки полублоков
    b1 = (g_round(rk[31], a0) ^ a1) & 0xFFFFFFFF
    b0 = a0
    return b1.to_bytes(4, "big") + b0.to_bytes(4, "big")

def decrypt_block_magma(block: bytes, rk: list) -> bytes:
    b1 = int.from_bytes(block[0:4], "big")
    b0 = int.from_bytes(block[4:8], "big")
    # Обратное G*[rk[31]]: a1 = g[k](b0) XOR b1, a0 = b0
    a1 = (g_round(rk[31], b0) ^ b1) & 0xFFFFFFFF
    a0 = b0
    # Обратные раунды 30..0
    for i in range(30, -1, -1):
        # Обратное G[rk[i]](a1, a0) = (old_a0, ...) = (a0 прежде свопа)
        # G дало: new_a1=a0_old, new_a0 = g[k](a0_old) XOR a1_old
        # Значит: a0_old = a1, a1_old = g[k](a1) XOR a0
        a1, a0 = (g_round(rk[i], a1) ^ a0) & 0xFFFFFFFF, a1
    return a1.to_bytes(4, "big") + a0.to_bytes(4, "big")

# ──────────────────────────────────────────────
#  ECB-режим с PKCS#7-дополнением (блок 8 байт)
# ──────────────────────────────────────────────
def pad_pkcs7(data: bytes, block_size: int) -> bytes:
    pad_len = block_size - (len(data) % block_size)
    return data + bytes([pad_len] * pad_len)

def unpad_pkcs7(data: bytes) -> bytes:
    if not data:
        raise ValueError("Нельзя снять дополнение с пустых данных.")
    pad_len = data[-1]
    if pad_len < 1 or pad_len > 8 or data[-pad_len:] != bytes([pad_len]) * pad_len:
        raise ValueError("Неверный ключ или повреждённый шифртекст: дополнение некорректно.")
    return data[:-pad_len]

def magma_ecb_encrypt(plaintext: bytes, key: bytes) -> bytes:
    rk = round_keys_from_key(key)
    padded = pad_pkcs7(plaintext, 8)
    ciphertext = b''
    for i in range(0, len(padded), 8):
        ciphertext += encrypt_block_magma(padded[i:i + 8], rk)
    return ciphertext

def magma_ecb_decrypt(ciphertext: bytes, key: bytes) -> bytes:
    rk = round_keys_from_key(key)
    plaintext = b''
    for i in range(0, len(ciphertext), 8):
        plaintext += decrypt_block_magma(ciphertext[i:i + 8], rk)
    return unpad_pkcs7(plaintext)

# ──────────────────────────────────────────────
#  Тест по ГОСТ Р 34.12-2018 (Приложение А)
# ──────────────────────────────────────────────
def test_gost_vectors():
    print("=" * 70)
    print("ТЕСТ МАГМА по ГОСТ Р 34.12-2018 (Приложение А)")
    print("=" * 70)
    key = bytes.fromhex(
        "ffeeddccbbaa99887766554433221100"
        "f0f1f2f3f4f5f6f7f8f9fafbfcfdfeff"
    )
    plaintext          = bytes.fromhex("fedcba9876543210")
    expected_ciphertext= bytes.fromhex("4ee901e5c2d8ca3d")
    rk         = round_keys_from_key(key)
    ciphertext = encrypt_block_magma(plaintext, rk)
    decrypted  = decrypt_block_magma(ciphertext, rk)
    print(f"Ключ (hex)        : {key.hex()}")
```

```
    print(f"Открытый текст    : {plaintext.hex()}")
    print(f"Ожидаемый шифр    : {expected_ciphertext.hex()}")
    print(f"Полученный шифр   : {ciphertext.hex()}")
    print(f"Тест шифрования   : {'ДА ✓' if ciphertext == expected_ciphertext else 'НЕТ ✗'}")
    print(f"Расшифрование     : {decrypted.hex()}")
    print(f"Тест расшифрования: {'ДА ✓' if decrypted == plaintext else 'НЕТ ✗'}")
    print("=" * 70)
    return ciphertext == expected_ciphertext and decrypted == plaintext

# ──────────────────────────────────────────────
#  Ввод ключа от пользователя
# ──────────────────────────────────────────────
def read_key_from_user() -> bytes:
    """Запрашивает 256-битный ключ в виде 64 hex-символов."""
    print("Ключ вводится как 64 шестнадцатеричных символа (256 бит).")
    print("Пример: ffeeddccbbaa99887766554433221100f0f1f2f3f4f5f6f7f8f9fafbfcfdfeff")
    key_hex = input("Введите ключ: ").strip().replace(" ", "")
    if len(key_hex) != 64:
        raise ValueError("Ключ должен содержать ровно 64 hex-символа (32 байта).")
    try:
        return bytes.fromhex(key_hex)
    except ValueError as exc:
        raise ValueError("Ключ должен состоять только из символов 0-9 и A-F.") from exc

# ──────────────────────────────────────────────
#  Интерактивные функции
# ──────────────────────────────────────────────
def interactive_encrypt_hex():
    """
    Зашифровать hex-блок (как в контрольном примере ГОСТ Р 34.12-2018).
    Ввод : hex-строка, кратная 16 символам (= 8 байт на блок).
    Пример: PT = fedcba9876543210  →  CT = 4ee901e5c2d8ca3d
    """
    pt_hex = input("Введите открытый текст в hex (кратно 16 символам): ").strip().replace(" ",
"")
    if not pt_hex:
        raise ValueError("Блок не должен быть пустым.")
    if len(pt_hex) % 16 != 0:
        raise ValueError("Длина hex должна быть кратна 16 (8 байт на блок).")
    try:
        pt_bytes = bytes.fromhex(pt_hex)
    except ValueError as exc:
        raise ValueError("Допустимы только символы 0-9 и A-F.") from exc
    key = read_key_from_user()
    rk = round_keys_from_key(key)
    ciphertext = b''
    for i in range(0, len(pt_bytes), 8):
        ciphertext += encrypt_block_magma(pt_bytes[i:i + 8], rk)
    print(f"Шифртекст (hex): {ciphertext.hex()}")


def interactive_decrypt():
    """Расшифровать hex-шифртекст."""
    ciphertext_hex = input("Введите шифртекст в hex: ").strip().replace(" ", "")
    if not ciphertext_hex:
        raise ValueError("Шифртекст не должен быть пустым.")
    if len(ciphertext_hex) % 16 != 0:
        raise ValueError("Длина hex-шифртекста должна быть кратна 16 символам (8 байт на блок).")
    try:
        ciphertext = bytes.fromhex(ciphertext_hex)
    except ValueError as exc:
        raise ValueError("Шифртекст должен состоять только из символов 0-9 и A-F.") from exc
    key = read_key_from_user()
    plaintext = magma_ecb_decrypt(ciphertext, key)
    try:
        print(f"Открытый текст: {plaintext.decode('utf-8')}")
    except UnicodeDecodeError:
        print(f"Результат (hex): {plaintext.hex()}")

# ──────────────────────────────────────────────
#  Главное меню
# ──────────────────────────────────────────────
def interactive_menu():
    print("\n" + "=" * 70)
    print("ШИФР МАГМА (ГОСТ Р 34.12-2015)  —  Задание 17")
    print("Сеть Фейстеля, 32 раунда, блок 64 бит, ключ 256 бит")
    print("Источник: ЛАБКРИПТ_2022, Блок G, стр. 29")
    print("=" * 70)
    print()
    print("ВВОД:")
```

```
    print("  Открытый текст : hex-строка, кратная 16 символам (8 байт на блок)")
    print("  Ключ           : 64 шестнадцатеричных символа (256 бит)")
    print("  Пример ГОСТ    : PT=fedcba9876543210 → CT=4ee901e5c2d8ca3d")
    print()
    while True:
        print("1 — Зашифровать hex-блок")
        print("2 — Расшифровать hex-шифртекст")
        print("0 — Выход")
        choice = input("Выберите действие: ").strip()
        if choice == "0":
            break
        elif choice == "1":
            try:
                interactive_encrypt_hex()
            except (ValueError, UnicodeError) as exc:
                print(f"Ошибка ввода: {exc}")
        elif choice == "2":
            try:
                interactive_decrypt()
            except (ValueError, UnicodeError) as exc:
                print(f"Ошибка ввода: {exc}")
        else:
            print("Выберите 0, 1 или 2.")
        print()

if __name__ == "__main__":
    interactive_menu()
```

```
Выберите действие: 1
Введите открытый текст в hex (кратно 16 символам): fedcba9876543210
Ключ вводится как 64 шестнадцатеричных символа (256 бит).
Пример: ffeeddccbbaa99887766554433221100f0f1f2f3f4f5f6f7f8f9fafbfcfdfeff
Введите ключ: ffeeddccbbaa99887766554433221100f0f1f2f3f4f5f6f7f8f9fafbfcfdfeff
Шифртекст (hex): 4ee901e5c2d8ca3d
```

Результат работы программы МАГМА

20. Шифр Кузнечик

Файл программы: 20.KUZNECHIK.py

```
# -*- coding: utf-8 -*-
"""
20. Шифр КУЗНЕЧИК (ГОСТ Р 34.12-2015)
Реализация блочного шифра «Кузнечик».
Блок: 128 бит, ключ: 256 бит, 10 раундов.
Тестирование по ГОСТ Р 34.12-2015 (Приложение А).
"""
PI = [
    252, 238, 221, 17, 207, 110, 49, 22, 251, 196, 250, 218, 35, 197, 4, 77,
    233, 119, 240, 219, 147, 46, 153, 186, 23, 54, 241, 187, 20, 205, 95, 193,
    249, 24, 101, 90, 226, 92, 239, 33, 129, 28, 60, 66, 139, 1, 142, 79,
    5, 132, 2, 174, 227, 106, 143, 160, 6, 11, 237, 152, 127, 212, 211, 31,
    235, 52, 44, 81, 234, 200, 72, 171, 242, 42, 104, 162, 253, 58, 206, 204,
    181, 112, 14, 86, 8, 12, 118, 18, 191, 114, 19, 71, 156, 183, 93, 135,
    21, 161, 150, 41, 16, 123, 154, 199, 243, 145, 120, 111, 157, 158, 178, 177,
    50, 117, 25, 61, 255, 53, 138, 126, 109, 84, 198, 128, 195, 189, 13, 87,
    223, 245, 36, 169, 62, 168, 67, 201, 215, 121, 214, 246, 124, 34, 185, 3,
    224, 15, 236, 222, 122, 148, 176, 188, 220, 232, 40, 80, 78, 51, 10, 74,
    167, 151, 96, 115, 30, 0, 98, 68, 26, 184, 56, 130, 100, 159, 38, 65,
    173, 69, 70, 146, 39, 94, 85, 47, 140, 163, 165, 125, 105, 213, 149, 59,
    7, 88, 179, 64, 134, 172, 29, 247, 48, 55, 107, 228, 136, 217, 231, 137,
    225, 27, 131, 73, 76, 63, 248, 254, 141, 83, 170, 144, 202, 216, 133, 97,
    32, 113, 103, 164, 45, 43, 9, 91, 203, 155, 37, 208, 190, 229, 108, 82,
    89, 166, 116, 210, 230, 244, 180, 192, 209, 102, 175, 194, 57, 75, 99, 182,
]
PI_INV = [0] * 256
for i in range(256):
    PI_INV[PI[i]] = i
L_VEC = [148, 32, 133, 16, 194, 192, 1, 251, 1, 192, 194, 16, 133, 32, 148, 1]
def gf_mul(a, b):
    p = 0
    hi_bit_set = 0
    for _ in range(8):
        if b & 1:
            p ^= a
        hi_bit_set = a & 0x80
        a <<= 1
        if hi_bit_set:
            a ^= 0x1c3
        a &= 0xff
        b >>= 1
    return p
def S(block):
    return bytes(PI[b] for b in block)
def S_inv(block):
    return bytes(PI_INV[b] for b in block)
def R(block):
    a = 0
    for i in range(16):
        a ^= gf_mul(block[i], L_VEC[i])
    return bytes([a]) + block[:15]
def R_inv(block):
    a = block[0]
    block = block[1:] + bytes([0])
    result = block[:-1] + bytes([0])
    new_last = 0
    for i in range(15):
        new_last ^= gf_mul(result[i], L_VEC[i])
    new_last ^= gf_mul(a, L_VEC[15])
    return result[:15] + bytes([new_last])
def L(block):
    for _ in range(16):
        block = R(block)
    return block
def L_inv(block):
    for _ in range(16):
        block = R_inv(block)
    return block
```

```
def F(k, a1, a0):
    tmp = bytes(a ^ b for a, b in zip(k, a1))
    tmp = L(S(tmp))
    return tmp, bytes(a ^ b for a, b in zip(tmp, a0))

def C(i):
    c = bytes([0] * 15 + [i])
    return L(c)

def key_expansion(key):
    k1 = key[:16]
    k2 = key[16:]
    round_keys = [k1, k2]
    a1, a0 = k1, k2
    for i in range(1, 5):
        for j in range(8):
            c = C(8 * (i - 1) + j + 1)
            a1, a0 = F(c, a1, a0)
        round_keys.extend([a1, a0])
    return round_keys

def encrypt_block(block, round_keys):
    state = block
    for i in range(9):
        state = bytes(a ^ b for a, b in zip(state, round_keys[i]))
        state = S(state)
        state = L(state)
    state = bytes(a ^ b for a, b in zip(state, round_keys[9]))
    return state

def decrypt_block(block, round_keys):
    state = block
    state = bytes(a ^ b for a, b in zip(state, round_keys[9]))
    for i in range(8, -1, -1):
        state = L_inv(state)
        state = S_inv(state)
        state = bytes(a ^ b for a, b in zip(state, round_keys[i]))
    return state

def pad_pkcs7(data: bytes, block_size: int) -> bytes:
    pad_len = block_size - (len(data) % block_size)
    return data + bytes([pad_len] * pad_len)

def unpad_pkcs7(data: bytes) -> bytes:
    if not data:
        raise ValueError("Нельзя снять дополнение с пустых данных.")
    pad_len = data[-1]
    if pad_len < 1 or pad_len > 16 or data[-pad_len:] != bytes([pad_len]) * pad_len:
        raise ValueError("Неверный ключ или повреждённый шифртекст: дополнение некорректно.")
    return data[:-pad_len]

def kuznechik_ecb_encrypt(plaintext: bytes, key: bytes) -> bytes:
    round_keys = key_expansion(key)
    padded = pad_pkcs7(plaintext, 16)
    ciphertext = b''
    for i in range(0, len(padded), 16):
        block = padded[i:i+16]
        ciphertext += encrypt_block(block, round_keys)
    return ciphertext

def kuznechik_ecb_decrypt(ciphertext: bytes, key: bytes) -> bytes:
    round_keys = key_expansion(key)
    plaintext = b''
    for i in range(0, len(ciphertext), 16):
        block = ciphertext[i:i+16]
        plaintext += decrypt_block(block, round_keys)
    return unpad_pkcs7(plaintext)

def test_gost_vectors():
    print("=" * 70)
    print("ТЕСТ КУЗНЕЧИК по ГОСТ Р 34.12-2015 (Приложение А)")
    print("=" * 70)
    key = bytes.fromhex(
        "8899aabbccddeeff0011223344556677"
        "fedcba98765432100123456789abcdef"
    )
    plaintext = bytes.fromhex("1122334455667700ffeeddccbbaa9988")
    expected_ciphertext = bytes.fromhex("7f679d90bebc24305a468d42b9d4edcd")
    round_keys = key_expansion(key)
    ciphertext = encrypt_block(plaintext, round_keys)
    print(f"Ключ (hex): {key.hex()}")
```

```
    print(f"Открытый текст: {plaintext.hex()}")
    print(f"Ожидаемый шифртекст: {expected_ciphertext.hex()}")
    print(f"Полученный шифртекст: {ciphertext.hex()}")
    enc_ok = ciphertext == expected_ciphertext
    print(f"Тест шифрования пройден: {'ДА' if enc_ok else 'НЕТ'}")
    decrypted = decrypt_block(ciphertext, round_keys)
    print(f"Расшифрованный текст: {decrypted.hex()}")
    dec_ok = decrypted == plaintext
    print(f"Расшифрование верно: {'ДА' if dec_ok else 'НЕТ'}")
    print("\nПроверка S-преобразования (A.1.1):")
    s_input = bytes.fromhex("ffeeddccbbaa99881122334455667700")
    s_expected = bytes.fromhex("b66cd8887d38e8d77765aeea0c9a7efc")
    s_result = S(s_input)
    print(f"S({s_input.hex()}) = {s_result.hex()}")
    print(f"Ожидается: {s_expected.hex()}")
    s_ok = s_result == s_expected
    print(f"Тест S пройден: {'ДА' if s_ok else 'НЕТ'}")
    print("\nПроверка R-преобразования (A.1.2):")
    r_input = bytes.fromhex("00000000000000000000000000000100")
    r_expected = bytes.fromhex("94000000000000000000000000000001")
    r_result = R(r_input)
    print(f"R({r_input.hex()}) = {r_result.hex()}")
    print(f"Ожидается: {r_expected.hex()}")
    r_ok = r_result == r_expected
    print(f"Тест R пройден: {'ДА' if r_ok else 'НЕТ'}")
    print("\nПроверка L-преобразования (A.1.3):")
    l_input = bytes.fromhex("64a59400000000000000000000000000")
    l_expected = bytes.fromhex("d456584dd0e3e84cc3166e4b7fa2890d")
    l_result = L(l_input)
    print(f"L({l_input.hex()}) = {l_result.hex()}")
    print(f"Ожидается: {l_expected.hex()}")
    l_ok = l_result == l_expected
    print(f"Тест L пройден: {'ДА' if l_ok else 'НЕТ'}")
    print("=" * 70)
    return enc_ok and dec_ok

def test_long_text():
    print("\n" + "=" * 70)
    print("ТЕСТ НА ТЕКСТЕ 1000+ СИМВОЛОВ")
    print("=" * 70)
    long_text = """Криптография — наука о методах обеспечения конфиденциальности,
целостности данных, аутентификации и невозможности отказа от авторства.
Шифр «Кузнечик» (ГОСТ Р 34.12-2015) является одним из стандартных
блочных шифров Российской Федерации. Он использует SP-сеть с 10 раундами,
размером блока 128 бит и ключом 256 бит.
Шифр «Кузнечик» был разработан для замены устаревшего шифра «Магма»
(ГОСТ 28147-89) и обеспечивает более высокий уровень безопасности
благодаря увеличенному размеру блока.
Основные преобразования шифра:
1. S-преобразование (нелинейная подстановка)
2. R-преобразование (линейный сдвиговый регистр)
3. L-преобразование (16-кратное применение R)
4. Сложение с раундовым ключом по модулю 2 (XOR)
Ключевое расписание шифра «Кузнечик» основано на сети Фейстеля
и использует константы C_i, вычисляемые как L([i]).
Данный алгоритм широко применяется для защиты конфиденциальной информации
в государственных и коммерческих системах Российской Федерации.
The Kuznyechik cipher is a symmetric block cipher standardized in
GOST R 34.12-2015. It operates on 128-bit blocks with a 256-bit key
and uses a substitution-permutation network structure with 10 rounds.
""" * 2
    print(f"Длина текста: {len(long_text)} символов")
    key = bytes.fromhex(
        "8899aabbccddeeff0011223344556677"
        "fedcba98765432100123456789abcdef"
    )
    plaintext_bytes = long_text.encode('utf-8')
    print(f"Длина в байтах: {len(plaintext_bytes)}")
    ciphertext = kuznechik_ecb_encrypt(plaintext_bytes, key)
    print(f"Длина шифртекста: {len(ciphertext)} байт")
    print(f"Первые 64 байта шифртекста (hex): {ciphertext[:64].hex()}")
    decrypted = kuznechik_ecb_decrypt(ciphertext, key)
    decrypted_text = decrypted.decode('utf-8')
    print(f"Расшифрование успешно: {'ДА' if decrypted_text == long_text else 'НЕТ'}")
    print("=" * 70)
```

```
        return decrypted_text == long_text

    def demo_phrase():
        print("\n" + "=" * 70)
        print("ДЕМОНСТРАЦИЯ НА ФРАЗЕ ВАРИАНТА")
        print("=" * 70)
        phrase = "Леопард не может изменить своих пятен"
        print(f"Фраза: {phrase}")
        key = bytes.fromhex(
            "8899aabbccddeeff0011223344556677"
            "fedcba98765432100123456789abcdef"
        )
        plaintext_bytes = phrase.encode('utf-8')
        ciphertext = kuznechik_ecb_encrypt(plaintext_bytes, key)
        print(f"Шифртекст (hex): {ciphertext.hex()}")
        decrypted = kuznechik_ecb_decrypt(ciphertext, key)
        decrypted_text = decrypted.decode('utf-8')
        print(f"Расшифрованный текст: {decrypted_text}")
        print(f"Совпадение: {'ДА' if decrypted_text == phrase else 'НЕТ'}")
        print("=" * 70)

    def read_key_from_user() -> bytes:
        """Запрашивает ключ Кузнечика длиной 256 бит в hex-формате."""
        key_hex = input(
            "Введите ключ (64 hex-символа, пример "
            "8899aabbccddeeff0011223344556677fedcba98765432100123456789abcdef): "
        ).strip().replace(" ", "")
        if len(key_hex) != 64:
            raise ValueError("Ключ должен содержать ровно 64 hex-символа (32 байта).")
        try:
            return bytes.fromhex(key_hex)
        except ValueError as exc:
            raise ValueError("Ключ должен состоять только из символов 0-9 и A-F.") from exc

    def interactive_menu():
        print("Кузнечик использует ключ длиной 256 бит (32 байта).")
        print("Ключ вводится вручную как 64 шестнадцатеричных символа.")
        while True:
            print("\n1 - Зашифровать текст")
            print("2 - Расшифровать hex-шифртекст")
            print("3 - Показать тестовый пример ГОСТ")
            print("0 - Выход")
            choice = input("Выберите действие: ").strip()
            if choice == "0":
                break
            if choice == "3":
                test_gost_vectors()
                continue
            if choice not in ("1", "2"):
                print("Выберите 0, 1, 2 или 3.")
                continue
            try:
                if choice == "1":
                    plaintext = input("Введите открытый текст: ")
                    key = read_key_from_user()
                    ciphertext = kuznechik_ecb_encrypt(plaintext.encode("utf-8"), key)
                    print(f"Шифртекст (hex): {ciphertext.hex()}")
                else:
                    ciphertext_hex = input("Введите шифртекст в hex: ").strip().replace(" ", "")
                    if not ciphertext_hex:
                        raise ValueError("Шифртекст не должен быть пустым.")
                    if len(ciphertext_hex) % 32 != 0:
                        raise ValueError("Длина hex-шифртекста должна быть кратна 32 символам (16
байт).")
                    try:
                        ciphertext = bytes.fromhex(ciphertext_hex)
                    except ValueError as exc:
                        raise ValueError("Шифртекст должен состоять только из символов 0-9 и A-F.")
from exc
                    key = read_key_from_user()
                    plaintext = kuznechik_ecb_decrypt(ciphertext, key)
                    try:
                        print(f"Открытый текст: {plaintext.decode('utf-8')}")
                    except UnicodeDecodeError:
                        print(f"Результат (hex): {plaintext.hex()}")
            except ValueError as exc:
                print(f"Ошибка ввода: {exc}")

    if __name__ == "__main__":
```

```
print("\n" + "=" * 70)
print("ШИФР КУЗНЕЧИК (ГОСТ Р 34.12-2015)")
print("=" * 70)
interactive_menu()
```

27. Цифровая подпись ГОСТ Р 34.10-2012

Файл программы: 27.ГОСТ_Р_34.10-2012.py

```
"""
Лабораторная работа 3_2
Алгоритм 27: ЭЦП по ГОСТ Р 34.10-2012
Вариант: "Леопард не может изменить своих пятен"
Источник: ЛАБКРИПТ_2022, Блок J, стр. 37-39
Задание: "PDF/Задание по ГОСТ Р 34.10-2012..pdf"
Параметры из методички (пример):
  Кривая E_p(a,b): y² ≡ x³ + x + 1 (mod 11)
  a = 1, b = 1, p = 11
  Порядок подгруппы q = 7
  Генератор G = (0, 1)
Хэш-функция квадратичной свёртки (h_0 = 0):
  h_i = (h_{i-1} + M_i)² mod q
  где M_i — коды (ord) символов сообщения
Подпись (отправитель):
  1. Выбрать секретный ключ 0 < x_A < q
  2. Вычислить Y_A = x_A * G (открытый ключ)
  3. Вычислить e = h(сообщение) mod q, если e=0 → e=1
  4. Выбрать случайное 0 < k < q
  5. Вычислить C = k * G, r = x_C mod q
  6. s = (k * e + r * x_A) mod q
  7. Подпись: (r, s)
Проверка (получатель):
  1. Вычислить e = h(сообщение) mod q, если e=0 → e=1
  2. Проверить 0 < r < q и 0 < s < q
  3. v = e⁻¹ mod q
  4. z₁ = s * v mod q
  5. z₂ = -r * v mod q
  6. C' = z₁*G + z₂*Y_A
  7. r' = x_{C'} mod q; если r' = r — подпись верна
"""
import random
from math import isqrt
# ──────────────────────────────────────────────
#  Эллиптическая кривая
# ──────────────────────────────────────────────
class EllipticCurve:
    def __init__(self, p, a, b):
        self.p = p
        self.a = a
        self.b = b
    def is_on_curve(self, point):
        if point is None:
            return True
        x, y = point
        return (y * y - x ** 3 - self.a * x - self.b) % self.p == 0
    def neg(self, P):
        if P is None:
            return None
        return P[0], (-P[1]) % self.p
    def add(self, P, Q):
        if P is None:
            return Q
        if Q is None:
            return P
        x1, y1 = P
        x2, y2 = Q
        p = self.p
        if x1 == x2:
            if (y1 + y2) % p == 0:
                return None  # точка на бесконечности
            lam = (3 * x1 * x1 + self.a) * pow(2 * y1, p - 2, p) % p
        else:
            lam = (y2 - y1) * pow(x2 - x1, p - 2, p) % p
        x3 = (lam * lam - x1 - x2) % p
        y3 = (lam * (x1 - x3) - y1) % p
        return x3, y3
    def mul(self, k, P):
        """Скалярное умножение методом двоичного разложения."""
        result = None
```

```
        addend = P
        while k > 0:
            if k & 1:
                result = self.add(result, addend)
            addend = self.add(addend, addend)
            k >>= 1
        return result

def point_order(curve, P):
    """Вычисляет порядок точки P перебором кратных P.

    Для кривой над простым полем порядок группы точек не превосходит
    p + 1 + 2*sqrt(p) (граница Хассе), поэтому перебор имеет верхнюю границу.
    """
    if curve.p <= 1:
        raise ValueError("Модуль p должен быть больше 1.")
    if P is None or not curve.is_on_curve(P):
        raise ValueError("Точка G должна лежать на заданной кривой.")

    max_order = curve.p + 1 + isqrt(4 * curve.p)
    multiple = None
    for order in range(1, max_order + 1):
        multiple = curve.add(multiple, P)
        if multiple is None:
            return order
    raise ValueError("Не удалось вычислить порядок G; проверьте параметры кривой.")

# ──────────────────────────────────────────────
#  Хэш-функция квадратичной свёртки (ЛАБКРИПТ_2022, стр. 1)
#  h_0 = 0
#  h_i = (h_{i-1} + M_i)² mod p
# ──────────────────────────────────────────────
def hash_func(message: str, q: int) -> int:
    """
    Упрощённая хэш-функция квадратичной свёртки по методичке:
      h_0 = 0
      h_i = (h_{i-1} + ord(M_i))² mod q
    Если h = 0 — возвращаем 1 (защита от нулевого хэша).
    """
    h = 0
    for ch in message:
        h = (h + ord(ch)) ** 2 % q
    return h if h != 0 else 1

# ──────────────────────────────────────────────
#  Генерация ключей
# ──────────────────────────────────────────────
def keygen(curve, P, q):
    d = random.randint(1, q - 1)   # закрытый ключ
    Q = curve.mul(d, P)             # открытый ключ Q = d·P
    return d, Q

# ──────────────────────────────────────────────
#  Создание подписи
# ──────────────────────────────────────────────
def sign(message, curve, P, q, d, k=None):
    e = hash_func(message, q)
    if not 0 < d < q:
        raise ValueError(f"Закрытый ключ d должен быть в диапазоне 1..{q - 1}.")
    if k is not None:
        if not 0 < k < q:
            raise ValueError(f"Одноразовый ключ k должен быть в диапазоне 1..{q - 1}.")
        C = curve.mul(k, P)
        if C is None:
            raise ValueError("Выбранный k дал точку на бесконечности; введите другое k.")
        r = C[0] % q
        s = (k * e + r * d) % q
        if r == 0 or s == 0:
            raise ValueError("Для выбранного k получено r=0 или s=0; введите другое k.")
        return r, s, e, k
    while True:
        k = random.randint(1, q - 1)
        C = curve.mul(k, P)
        if C is None:
            continue
        r = C[0] % q
        if r == 0:
            continue
        s = (k * e + r * d) % q
        if s == 0:
            continue
        return r, s, e, k
```

```
# ──────────────────────────────────────────────
#  Проверка подписи
# ──────────────────────────────────────────────
def verify(message, r, s, curve, P, Q_pub, q):
    if not (0 < r < q and 0 < s < q):
        return False
    e = hash_func(message, q)
    v = pow(e, q - 2, q)       # e^{-1} mod q
    z1 = s * v % q
    z2 = (-r * v) % q
    C = curve.add(curve.mul(z1, P), curve.mul(z2, Q_pub))
    if C is None:
        return False
    return C[0] % q == r

# ──────────────────────────────────────────────
#  Главное меню
# ──────────────────────────────────────────────
# Параметры по умолчанию из примера методички (ЛАБКРИПТ_2022, стр. 37)
#   E_p(a,b): y² ≡ x³ + x + 1 (mod 11)
#   a=1, b=1, p=11, q=7, G=(0,1)
DEFAULT_P = 11
DEFAULT_A = 1
DEFAULT_B = 1
DEFAULT_G = (0, 1)
DEFAULT_Q = 7

def print_params(curve, G, q):
    print(f"  Кривая     : y² ≡ x³ + {curve.a}x + {curve.b}  (mod {curve.p})")
    print(f"  Генератор  : G = {G}")
    print(f"  Порядок    : q = {q}")

def main():
    print("=" * 60)
    print("ГОСТ Р 34.10-2012  —  Цифровая подпись (эллиптические кривые)")
    print("=" * 60)
    print("Параметры из методички (ЛАБКРИПТ_2022, Блок J, задание 27):")
    print(f"  Кривая E_p(a,b): y² ≡ x³ + ax + b  (mod p)")
    print(f"  Хэш: h_0=0, h_i=(h_{{i-1}}+ord(M_i))² mod q")
    print()
    # Загружаем параметры по умолчанию
    curve = EllipticCurve(p=DEFAULT_P, a=DEFAULT_A, b=DEFAULT_B)
    P_BASE = DEFAULT_G
    Q_ORDER = DEFAULT_Q
    while True:
        print("\n" + "-" * 60)
        print("Текущие параметры кривой:")
        print_params(curve, P_BASE, Q_ORDER)
        print("-" * 60)
        print("1 — Задать параметры кривой вручную")
        print("2 — Создать подпись")
        print("3 — Проверить подпись")
        print("4 — Показать пример из методички")
        print("0 — Выход")
        choice = input("Ваш выбор: ").strip()
        if choice == "0":
            break
        try:
            if choice == "1":
                print(f"\nВведите параметры кривой y² ≡ x³ + ax + b (mod p)")
                p = int(input("  p (простое): "))
                a = int(input("  a: "))
                b = int(input("  b: "))
                gx = int(input("  Gx (x-координата генератора): "))
                gy = int(input("  Gy (y-координата генератора): "))
                curve = EllipticCurve(p=p, a=a, b=b)
                P_BASE = (gx, gy)
                if not curve.is_on_curve(P_BASE):
                    print("  ОШИБКА: точка G не лежит на заданной кривой!")
                    # Возвращаем умолчания
                    curve = EllipticCurve(p=DEFAULT_P, a=DEFAULT_A, b=DEFAULT_B)
                    P_BASE = DEFAULT_G
                    Q_ORDER = DEFAULT_Q
                else:
                    Q_ORDER = point_order(curve, P_BASE)
                    print(f"  Порядок точки G вычислен автоматически: q = {Q_ORDER}.")
                    print("  Параметры установлены.")
            elif choice == "2":
```

```
                    print("\n--- Создание подписи ---")
                    message = input("Введите сообщение: ")
                    if not message:
                        raise ValueError("Сообщение не должно быть пустым.")
                    d = int(input(f"Введите закрытый ключ x_A (1..{Q_ORDER - 1}): "))
                    k = int(input(f"Введите одноразовый ключ k (1..{Q_ORDER - 1}): "))
                    if not (0 < d < Q_ORDER):
                        raise ValueError(f"x_A должен быть в диапазоне 1..{Q_ORDER - 1}.")
                    Q_pub = curve.mul(d, P_BASE)
                    r, s, e, _ = sign(message, curve, P_BASE, Q_ORDER, d, k)
                    print(f"\nХэш e = {e}")
                    print(f"Открытый ключ Y_A = {Q_pub}")
                    print(f"Подпись (r, s) = ({r}, {s})")
                    ok = verify(message, r, s, curve, P_BASE, Q_pub, Q_ORDER)
                    print(f"Проверка созданной подписи: {'ВЕРНА ✓' if ok else 'НЕ ВЕРНА ✗'}")
                elif choice == "3":
                    print("\n--- Проверка подписи ---")
                    message = input("Введите исходное сообщение: ")
                    r = int(input("Введите r: "))
                    s = int(input("Введите s: "))
                    qx = int(input("Введите координату Y_Ax: "))
                    qy = int(input("Введите координату Y_Ay: "))
                    Q_pub = (qx, qy)
                    if not curve.is_on_curve(Q_pub):
                        raise ValueError("Открытый ключ Y_A не принадлежит заданной эллиптической
кривой.")
                    valid = verify(message, r, s, curve, P_BASE, Q_pub, Q_ORDER)
                    print(f"Результат проверки: {'ПОДПИСЬ ВЕРНА ✓' if valid else 'ПОДПИСЬ НЕВЕРНА
✗'}")
                elif choice == "4":
                    print("\n" + "=" * 60)
                    print("ПРИМЕР ИЗ МЕТОДИЧКИ (ЛАБКРИПТ_2022, стр. 37)")
                    print("=" * 60)
                    print("Кривая E_p(a,b): y² ≡ x³ + x + 1 (mod 11)")
                    print("a = 1, b = 1, p = 11, q = 7, G = (0, 1)")
                    print()
                    demo_curve = EllipticCurve(p=11, a=1, b=1)
                    demo_G = (0, 1)
                    demo_q = 7
                    # Пример из PDF: x_A=4, k=5
                    demo_xA = 4
                    demo_k = 5
                    demo_msg = "ДЕВА"  # Из таблицы вариантов для примера
                    # Считаем хэш
                    demo_e = hash_func(demo_msg, demo_q)
                    demo_YA = demo_curve.mul(demo_xA, demo_G)
                    demo_C = demo_curve.mul(demo_k, demo_G)
                    demo_r = demo_C[0] % demo_q
                    demo_s = (demo_k * demo_e + demo_r * demo_xA) % demo_q
                    print(f"Сообщение   : {demo_msg!r}")
                    print(f"x_A (секрет): {demo_xA}")
                    print(f"k (разовый) : {demo_k}")
                    print(f"Хэш e       : {demo_e}")
                    print(f"Y_A = {demo_xA}·G = {demo_YA}")
                    print(f"C = {demo_k}·G = {demo_C}")
                    print(f"r = {demo_r}, s = {demo_s}")
                    print(f"Подпись (r, s) = ({demo_r}, {demo_s})")
                    ok = verify(demo_msg, demo_r, demo_s, demo_curve, demo_G, demo_YA, demo_q)
                    print(f"Проверка: {'ВЕРНА ✓' if ok else 'НЕ ВЕРНА ✗'}")
                    print("=" * 60)
                else:
                    print("Выберите 0, 1, 2, 3 или 4.")
            except ValueError as exc:
                print(f"Ошибка ввода: {exc}")
            except Exception as exc:
                print(f"Ошибка: {exc}")

    if __name__ == "__main__":
        main()
```

```
------------------------------------------------------------
1 — Задать параметры кривой вручную
2 — Создать подпись
3 — Проверить подпись
4 — Показать пример из методички
0 — Выход
Ваш выбор: 1

Введите параметры кривой y² ≡ x³ + ax + b (mod p)
  p (простое): 11
  a: 2
  b: 6
  Gx (x-координата генератора): 10
  Gy (y-координата генератора): 5
  Порядок точки G вычислен автоматически: q = 7.
  Параметры установлены.

------------------------------------------------------------
Текущие параметры кривой:
  Кривая     : y² ≡ x³ + 2x + 6  (mod 11)
  Генератор  : G = (10, 5)
  Порядок    : q = 7
```

```
Ваш выбор: 2

--- Создание подписи ---
Введите сообщение: БАЗА
Введите закрытый ключ x_A (1..6): 5
Введите одноразовый ключ k (1..6): 4

Хэш e = 1
Открытый ключ Y_A = (5, 8)
Подпись (r, s) = (1, 2)
Проверка созданной подписи: ВЕРНА ✓

------------------------------------------------------------
Текущие параметры кривой:
  Кривая     : y² ≡ x³ + 2x + 6  (mod 11)
  Генератор  : G = (10, 5)
  Порядок    : q = 7
```

```
------------------------------------------------------------
1 — Задать параметры кривой вручную
2 — Создать подпись
3 — Проверить подпись
4 — Показать пример из методички
0 — Выход
Ваш выбор: 3

--- Проверка подписи ---
Введите исходное сообщение: БАЗА
Введите r: 1
Введите s: 2
Введите координату Y_Ax: 5
Введите координату Y_Ay: 8
Результат проверки: ПОДПИСЬ ВЕРНА ✓

------------------------------------------------------------
Текущие параметры кривой:
  Кривая     : y² ≡ x³ + 2x + 6  (mod 11)
  Генератор  : G = (10, 5)
  Порядок    : q = 7
------------------------------------------------------------
```